

**PERBANDINGAN GENETIC ALGORITHM (GA) DAN ANT COLONY
OPTIMIZATION (ACO) PADA SIMULASI OPTIMASI PENJADWALAN
KORIDOR 5 BUS TRANS SULSEL RUTE UNHAS TAMALANREA – UNHAS
GOWA**



ABDUL HAFIZH HIDAYATULLAH M

H011191072



**PROGRAM STUDI MATEMATIKA
DEPARTEMEN MATEMATIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS HASANUDDIN
MAKASSAR**

2026

**PERBANDINGAN GENETIC ALGORITHM (GA) DAN ANT COLONY
OPTIMIZATION (ACO) PADA SIMULASI OPTIMASI PENJADWALAN
KORIDOR 5 BUS TRANS SULSEL RUTE UNHAS TAMALANREA – UNHAS
GOWA**

**ABDUL HAFIZH HIDAYATULLAH M
H011191072**



**PROGRAM STUDI MATEMATIKA
DEPARTEMEN MATEMATIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS HASANUDDIN
MAKASSAR
2026**

**PERBANDINGAN GENETIC ALGORITHM (GA) DAN ANT COLONY
OPTIMIZATION (ACO) PADA SIMULASI OPTIMASI PENJADWALAN
KORIDOR 5 BUS TRANS SULSEL RUTE UNHAS TAMALANREA – UNHAS
GOWA**

**ABDUL HAFIZH HIDAYATULLAH M
H011191072**



**Pembimbing Utama
Penguji**

**: Dr. Khaeruddin, M.Sc.
: 1. Dr. Firman, S.Si., M.Si.
: 2. Naimah Aris, S.Si., M.Math.**

**PROGRAM STUDI MATEMATIKA
DEPARTEMEN MATEMATIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS HASANUDDIN
MAKASSAR
2026**

**PERBANDINGAN GENETIC ALGORITHM (GA) DAN ANT COLONY
OPTIMIZATION (ACO) PADA SIMULASI OPTIMASI PENJADWALAN
KORIDOR 5 BUS TRANS SULSEL RUTE UNHAS TAMALANREA – UNHAS
GOWA**

**ABDUL HAFIZH HIDAYATULLAH M
H011191072**

Skripsi

sebagai salah satu syarat untuk mencapai gelar sarjana

Program Studi Matematika

pada

**PROGRAM STUDI MATEMATIKA
DEPARTEMEN MATEMATIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS HASANUDDIN
MAKASSAR
2026**

SKRIPSI

PERBANDINGAN GENETIC ALGORITHM (GA) DAN ANT COLONY OPTIMIZATION (ACO) PADA SIMULASI OPTIMASI PENJADWALAN KORIDOR 5 BUS TRANS SULSEL RUTE UNHAS TAMALANREA - UNHAS GOWA

ABDUL HAFIZH HIDAYATULLAH M
H011191072

Skripsi,

telah dipertahankan di depan Panitia Ujian Sarjana Matematika pada tanggal 03 Agustus 2026 dan dinyatakan telah memenuhi syarat kelulusan

pada

Program Studi Matematika
Departemen Matematika
Fakultas Matematika dan Ilmu Pengetahuan Alam
Universitas Hasanuddin
Makassar

Mengesahkan :**Pembimbing tugas akhir**

Dr. Khaeruddin, M.Sc.
NIP. 496509141991031003

Mengetahui :**Ketua Program Studi Matematika**

Dr. Firman, S.Si., M.Si.
NIP. 196804292002121001



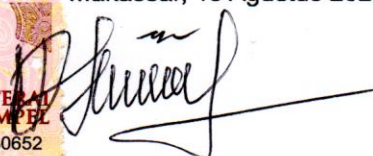
**PERNYATAAN KEASLIAN SKRIPSI
DAN PELIMPAHAN HAK CIPTA**

Dengan ini saya menyatakan bahwa, skripsi berjudul “PERBANDINGAN GENETIC ALGORITHM (GA) DAN ANT COLONY OPTIMIZATION (ACO) PADA SIMULASI OPTIMASI PENJADWALAN KORIDOR 5 BUS TRANS SULSEL RUTE UNHAS TAMALANREA – UNHAS GOWA” adalah benar karya saya dengan arahan dari pembimbing (Dr. Khaeruddin, M.Sc.). Karya ilmiah ini belum diajukan dan tidak sedang diajukan dalam bentuk apa pun kepada perguruan tinggi mana pun. Sumber informasi yang berasal atau dikutip dari karya yang diterbitkan maupun tidak diterbitkan dari penulis lain telah disebutkan dalam teks dan dicantumkan dalam Daftar Pustaka skripsi ini. Apabila di kemudian hari terbukti atau dapat dibuktikan bahwa sebagian atau keseluruhan skripsi ini adalah karya orang lain, maka saya bersedia menerima sanksi atas perbuatan tersebut berdasarkan aturan yang berlaku.

Dengan ini saya melimpahkan hak cipta (hak ekonomis) dari karya tulis saya berupa skripsi ini kepada Universitas Hasanuddin.



Makassar, 13 Agustus 2026


Abdul Hafizh Hidayatullah M
H011191072

UCAPAN TERIMA KASIH

Alhamdulillah, Puji syukur penulis panjatkan kehadirat Allah SWT atas segala rahmat dan karunia-nya, sehingga penulis dapat menyelesaikan skripsi yang berjudul **“PERBANDINGAN GENETIC ALGORITHM (GA) DAN ANT COLONY OPTIMIZATION (ACO) PADA SIMULASI OPTIMASI PENJADWALAN KORIDOR 5 BUS TRANS SULSEL RUTE UNHAS TAMALANREA – UNHAS GOWA”**. Skripsi ini tidak akan terwujud tanpa dukungan, bimbingan, dan bantuan dari berbagai pihak. Oleh karena itu, pada kesempatan kali ini penulis menyampaikan terima kasih

Dengan penuh kerendahan hati dan rasa hormat penulis menyampaikan terima kasih yang sebesar-besarnya dan penghargaan kepada orang tua penulis, Bapak **Pelda Makkatang** dan Ibu tersayang **Nursiah** yang telah sabar dan bekerja keras untuk membesarkan, mendidik, dan senantiasa memberikan doa, dukungan, motivasi, dan materi sehingga penulis bisa sampai di titik ini. Kepada keluarga yang senantiasa memberi dukungan doa dalam penyelesaian skripsi ini. Pada kesempatan kali ini pula, penulis ingin menyampaikan terima kasih kepada:

1. **Bapak Prof. Dr. Ir. Jamaluddin Jompa M.Sc.** selaku Rektor Universitas Hasanuddin beserta seluruh jajarannya, serta bapak **Dr.Sci. Muhammad Zakir, S.Si., M.Si.** selaku Dekan Fakultas Matematika dan Ilmu Pengetahuan Alam beserta jajarannya.
2. Secara khusus dan penuh hormat penulis menyampaikan rasa terima kasih kepada Bapak **Dr. Firman, S.Si., M.Si.** selaku Ketua Departemen Matematika Fakultas Matematika dan Ilmu Pengetahuan Alam Universitas Hasanuddin sekaligus selaku Dosen Penasihat Akademik dan Dosen Penguji yang sudah memberikan masukan dan kritikan kepada penulis yang bersifat membangun karena sesungguhnya skripsi ini tidak luput dari kekeliruan.
3. Secara khusus dengan penuh hormat penulis menyampaikan rasa terima kasih kepada Bapak **Dr. Khaeruddin, M.Sc.** selaku Dosen Pembimbing yang telah sabar, telaten dan penuh kesungguhan dalam membimbing penulis, memberikan saran, kritik, dan dorongan selama proses penyelesaian skripsi ini.
4. Secara khusus dengan penuh hormat penulis menyampaikan rasa terima kasih kepada Ibu **Naimah Aris, S.Si., M.Math.** selaku Dosen Penguji yang telah sabar dan penuh kesungguhan dalam memberikan dukungan berupa nasihat, masukan, dan kritikan kepada penulis yang bersifat membangun karena sesungguhnya skripsi ini tidak luput dari kekeliruan.
5. Terima kasih kepada seluruh **Dosen Departemen Matematika** terkhusus Ibu **Prof. Dr. Hasmawati, M.Si., Bapak Dr. Andi Muhammad Anwar, S.Si., M.Si., dan Bapak Mauliddin, S.Si., M.Si.** yang senantiasa mengingatkan dan memberi dukungan kepada penulis selama proses penyelesaian skripsi.
6. Terima kasih kepada **Staf Departemen Matematika** terkhusus Bapak **Nasir, S.Sos., Ibu Karmila, S.Si., dan Ibu Indah Puspitasiari, S.Kes.** yang telah sabar membantu dan memudahkan penulis dalam berbagai urusan administratif selama proses penyelesaian skripsi.

7. Terima kasih kepada **Staff Fakultas Matematika dan Ilmu Pengetahuan Alam** yang telah sabar membantu dan memudahkan penulis dalam berbagai urusan administratif selama proses penyelesaian skripsi.
8. Terima kasih kepada teman-teman seperjuangan Matematika 2019 yang telah berjuang bersama-sama selama proses penyelesaian skripsi.
9. Terima kasih saudara-saudara tercinta, yang selalu memberikan semangat, perhatian, dukungan, dan menjadi tempat berbagi dalam setiap proses yang penulis lalui.
10. Terakhir kepada diri saya sendiri Abdul Hafizh Hidayatullah M. Apresiasi yang sebesar-besarnya karena telah bertanggung jawab untuk menyelesaikan yang telah dimulai, terima kasih kepada hati yang tetap ikhlas, jiwa yang tetap kuat, raga yang terus melangkah meski tidak semua hal berjalan sesuai harapan. Terima kasih sudah kuat hingga detik ini, dan mampu mengendalikan diri dari tekanan luar. Terima kasih karena terus berusaha dan tidak menyerah, berbahagialah selalu dan jangan berhenti untuk berusaha untuk mengejar impian.

Penulis menyadari bahwa skripsi ini masih jauh dari kata sempurna. Oleh karena itu, penulis dengan terbuka menerima segala kritik dan saran yang membangun demi perbaikan dan penyempurnaan skripsi ini. Semoga skripsi ini dapat memberikan manfaat bagi pembaca serta menjadi sumbangsih yang bermanfaat bagi perkembangan ilmu pengetahuan.

Akhir kata, penulis mengucapkan terima kasih kepada seluruh pihak yang telah memberikan dukungan dan doa. Semoga segala kebaikan yang telah diberikan mendapatkan balasan yang terbaik dari Allah SWT.

Penulis,



Abdul Hafizh Hidayatullah M

ABSTRAK

Abdul Hafizh Hidayatullah M. **PERBANDINGAN GENETIC ALGORITHM (GA) DAN ANT COLONY OPTIMIZATION (ACO) PADA SIMULASI OPTIMASI PENJADWALAN KORIDOR 5 BUS TRANS SULSEL RUTE UNHAS TAMALANREA – UNHAS GOWA.**
(dibimbing oleh Dr. Khaeruddin, M.Sc.)

Latar belakang. Bus Trans Sulsel Koridor 5 yang melayani rute Kampus Universitas Hasanuddin Tamalanrea–Kampus Universitas Hasanuddin Gowa menghadapi permasalahan ketidakseimbangan antara kapasitas layanan dan fluktuasi permintaan penumpang antara jam sibuk dan non-sibuk, sehingga menimbulkan kepadatan penumpang pada jam sibuk serta rendahnya utilisasi armada pada jam non-sibuk. Penelitian ini bertujuan menentukan konfigurasi operasional Bus Trans Sulsel Koridor 5 yang optimal serta membandingkan kinerja Algoritma Genetika (Genetic Algorithm/GA) dan Ant Colony Optimization (ACO) dalam menyelesaikan permasalahan tersebut. Model optimasi dibangun berdasarkan penentuan jumlah bus aktif (xt), jumlah trip pergi-pulang (qt), dan headway (ht) pada setiap periode operasional, dengan fungsi objektif berupa weighted-sum ternormalisasi dari biaya operasional, waktu tunggu penumpang, dan jumlah kursi kosong, ditambah penalti shortage, dengan kendala kapasitas trip harian ($\sum qt \leq 60$) dan hubungan $qt \leq kt \cdot xt$. Kedua metode diimplementasikan menggunakan bahasa pemrograman Python dan diuji pada data operasional aktual Koridor 5 (45 halte arah pergi, 37 halte arah pulang, 15 armada, kapasitas 40 penumpang per bus). Hasil penelitian menunjukkan bahwa GA dan ACO menghasilkan konfigurasi trip PP yang identik, yaitu [3, 10, 7, 3, 8, 5] untuk enam periode operasional (total 36 trip per hari), dengan headway seragam 10 menit pada hampir seluruh periode. Konfigurasi ini menurunkan biaya operasional harian sebesar 40% (dari Rp8.569.020 menjadi Rp5.141.412) dan meningkatkan load factor rata-rata dari 55,79% menjadi 92,99%, tanpa menimbulkan shortage. GA konvergen pada generasi ke-37 dengan nilai fungsi objektif 1,3339 dan waktu komputasi 183,17 detik, sedangkan ACO konvergen pada iterasi ke-13 dengan nilai fungsi objektif 1,3575 dan waktu komputasi 223,93 detik. Berdasarkan perbandingan sepuluh indikator kinerja, GA terbukti sedikit lebih efektif dibandingkan ACO, dengan selisih nilai fungsi objektif sekitar 1,74%, meskipun ACO tetap menjadi alternatif yang layak, terutama ditinjau dari utilisasi armada. Analisis sensitivitas menunjukkan bahwa armada eksisting mampu menampung pertumbuhan demand hingga 20% tanpa menimbulkan shortage, sementara evaluasi arah pulang menunjukkan potensi penurunan waktu tunggu penumpang sekitar 39–41%.

Kata kunci: Bus Trans Sulsel, optimasi operasional, Genetic Algorithm, Ant Colony Optimization, penjadwalan transportasi publik.

ABSTRACT

Abdul Hafizh Hidayatullah M. **COMPARISON OF GENETIC ALGORITHM (GA) AND ANT COLONY OPTIMIZATION IN SCHEDULING SIMULATION OF CORRIDOR 5 BUS TRANS SULSEL ROUTE UNHAS TAMALANREA – UNHAS GOWA.** (Supervised by Dr. Khaeruddin, M.Sc.)

Corridor 5 of Bus Trans Sulsel, which serves the route between Universitas Hasanuddin Tamalanrea Campus and Universitas Hasanuddin Gowa Campus, faces an imbalance between service capacity and fluctuating passenger demand across peak and off-peak hours, resulting in passenger overcrowding during peak hours and low fleet utilization during off-peak hours. This study aims to determine the optimal operational configuration for Bus Trans Sulsel Corridor 5 and to compare the performance of the Genetic Algorithm (GA) and Ant Colony Optimization (ACO) in solving this problem. The optimization model determines the number of active buses (x_t), the number of round trips (q_t), and the headway (h_t) for each operational period, using a normalized weighted-sum objective function composed of operational cost, passenger waiting time, and the number of empty seats, plus a shortage penalty, subject to a daily trip capacity constraint ($\sum q_t \leq 60$) and the relation $q_t \leq k_t \cdot x_t$. Both methods were implemented using the Python programming language and tested on actual operational data from Corridor 5 (45 outbound stops, 37 inbound stops, 15 buses, a capacity of 40 passengers per bus). The results show that GA and ACO produced an identical round-trip configuration of [3, 10, 7, 3, 8, 5] across six operational periods (36 trips per day in total), with a uniform 10-minute headway in nearly all periods. This configuration reduced daily operational cost by 40% (from IDR 8,569,020 to IDR 5,141,412) and increased the average load factor from 55.79% to 92.99%, without causing any shortage. GA converged at generation 37 with an objective value of 1.3339 and a computation time of 183.17 seconds, while ACO converged at iteration 13 with an objective value of 1.3575 and a computation time of 223.93 seconds. Based on a comparison of ten performance indicators, GA proved slightly more effective than ACO, with an objective-value gap of about 1.74%, although ACO remains a viable alternative, particularly in terms of fleet utilization. Sensitivity analysis indicated that the existing fleet can accommodate up to a 20% growth in demand without shortage, while the return-trip evaluation showed a potential 39–41% reduction in passenger waiting time.

Keywords: Bus Trans Sulsel, operational optimization, Genetic Algorithm, Ant Colony Optimization, public transport scheduling.

DAFTAR ISI

	Halaman
HALAMAN JUDUL	ii
PERNYATAAN PENGAJUAN	iv
HALAMAN PENGESAHAN	v
PERNYATAAN KEASLIAN SKRIPSI.....	vi
UCAPAN TERIMA KASIH	vii
ABSTRAK.....	ix
ABSTRACT.....	x
DAFTAR ISI	xi
DAFTAR GAMBAR.....	xv
DAFTAR TABEL	xvi
DAFTAR NOTASI	xviii
DAFTAR ISTILAH.....	xx
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah.....	3
1.3 Batasan Masalah	3
1.4 Tujuan Penelitian	3
1.5 Manfaat Penelitian	4
1.5.1 Manfaat Teoritis	4
1.5.2 Manfaat Praktis	4
1.6 Landasan Teori	4
1.6.1 Sistem Transportasi Publik.....	4
1.6.2 Bus Trans Sulsel	6
1.6.3 Optimasi Operasional Bus.....	8
1.6.4 Algoritma Genetika (Genetic Algorithm).....	10
1.6.5 Ant Colony Optimization (ACO)	13
1.6.6 Penelitian Terdahulu	17
1.6.7 Kerangka Konseptual Penelitian	19
BAB II METODE PENELITIAN	20
2.1 Waktu dan Lokasi Penelitian.....	20
2.1.1 Waktu Penelitian.....	20
2.1.2 Lokasi Penelitian	20
2.2 Sumber dan Jenis Data	20
2.2.1 Sumber Data	20

2.2.2 Jenis Data	21
2.2.3 Periode Operasional dan Skenario Permintaan Penumpang	21
2.2.4 Ringkasan Data Penelitian	21
2.3 Variabel Penelitian	22
2.3.1 Variabel Input	22
2.3.2 Variabel Keputusan	22
2.3.3 Variabel Output.....	23
2.3.4 Definisi Operasional Variabel	23
2.3.5 Hubungan Bus Aktif (xt) dan Trip PP (qt).....	24
2.3.6 Hubungan Antar Variabel.....	25
2.4 Asumsi Penelitian	25
2.5 Kerangka Penelitian.....	26
2.6 Formulasi Model Optimasi	28
2.6.1 Himpunan (Sets)	28
2.6.2 Parameter.....	28
2.6.3 Variabel Keputusan	28
2.6.4 Fungsi Objektif	28
2.6.5 Komponen Fungsi Objektif	29
2.6.6 Kendala Model	30
2.6.7 Model Optimasi	31
2.7 Implementasi Genetic Algorithm	31
2.7.1 Representasi Kromosom	31
2.7.2 Inisialisasi Populasi	32
2.7.3 Fungsi Fitness	32
2.7.4 Seleksi.....	32
2.7.5 Crossover	32
2.7.6 Mutasi.....	32
2.7.7 Elitisme.....	33
2.7.8 Kriteria Terminasi.....	33
2.7.9 Parameter Genetic Algorithm	33
2.7.10 Alur Implementasi Genetic Algorithm.....	33
2.8 Implementasi Ant Colony Optimization	34
2.8.1 Representasi Solusi	34
2.8.2 Pembentukan Ruang Solusi	34
2.8.3 Inisialisasi Feromon.....	34
2.8.4 Informasi Heuristik.....	35

2.8.5 Probabilitas Pemilihan Solusi	35
2.8.6 Evaluasi Solusi	35
2.8.7 Pembaruan Feromon	36
2.8.8 Kriteria Terminasi.....	36
2.8.9 Parameter Ant Colony Optimization	36
2.8.10 Alur Implementasi Ant Colony Optimization	37
2.9 Kriteria Evaluasi Kinerja.....	37
2.9.1 Nilai Fungsi Objektif	37
2.9.2 Biaya Operasional	37
2.9.3 Waktu Tunggu Penumpang	38
2.9.4 Shortage.....	38
2.9.5 Tingkat Utilisasi Armada	38
2.9.6 Load Factor	38
2.9.7 Waktu Komputasi	38
2.9.8 Analisis Perbandingan	38
2.10 Tahapan Penelitian	38
BAB III HASIL PENELITIAN	41
3.1 Pengumpulan Data	41
3.1.1 Data Halte	41
3.1.2 Data Jarak Antar Halte	43
3.1.3 Data Armada Bus	46
3.1.4 Jadwal Keberangkatan	46
3.1.5 Data Biaya Operasional.....	46
3.2 Hasil.....	47
3.2.1 Hasil Persiapan Data Operasional	47
3.2.2 Hasil Pembangkitan Demand Penumpang.....	48
3.2.3 Hasil Evaluasi Kondisi Eksisting (Baseline).....	49
3.2.4 Hasil Optimasi Genetic Algorithm (GA).....	50
3.2.5 Hasil Optimasi Ant Colony Optimization (ACO)	51
3.2.6 Perbandingan Kinerja Existing, GA, dan ACO	52
3.2.7 Hasil Analisis Sensitivitas	56
3.2.8 Evaluasi Arah Pulang Berbasis Data Lapangan	58
3.3 Pembahasan	61
3.3.1 Efektivitas Optimasi.....	61
3.3.2 Perbandingan GA dan ACO.....	61
3.3.3 Keterbatasan Variabel Headway	61

3.3.4 Batas Kapasitas Armada terhadap Pertumbuhan Demand	61
3.3.5 Konsistensi Hasil dengan Kondisi Arah Pulang	62
3.3.6 Implikasi Praktis	62
BAB IV KESIMPULAN DAN SARAN	63
4.1 Kesimpulan	63
4.2 Saran	64
4.2.1 Saran bagi Pengelola Bus Trans Sulsel	64
4.2.2 Saran bagi Penelitian Selanjutnya	64
DAFTAR PUSTAKA	66
LAMPIRAN	68

DAFTAR GAMBAR

Nomor Urut	Halaman
Gambar 1 Peta Geografis Koridor 5 (Unhas Tamalanrea → Unhas Gowa) Bus Trans Sulsel	7
Gambar 2 Peta Geografis Koridor 5 (Unhas Gowa → Unhas Tamalanrea) Bus Trans Sulsel	8
Gambar 3 Kerangka Penelitian	27
Gambar 4 Flowchart Tahapan Penelitian	40
Gambar 5 Demand Penumpang per Periode Operasional	48
Gambar 6 Load Factor Bangkitan per Periode	49
Gambar 7 Konvergensi Genetic Algorithm	50
Gambar 8 Konvergensi Ant Colony Optimization	51
Gambar 9 Perbandingan Biaya Operasional Harian	54
Gambar 10 Perbandingan Load Factor	54
Gambar 11 Perbandingan Jumlah Bus Aktif per Periode	55
Gambar 12 Perbandingan Headway per Periode	55
Gambar 13 Perbandingan Empty Seat per Periode	56
Gambar 14 Sensitivitas Nilai Fungsi Objektif terhadap Perubahan Demand	57
Gambar 15 Sensitivitas Biaya Operasional terhadap Perubahan Demand	58
Gambar 16 Kontribusi Biaya Operasional Arah Pulang	59
Gambar 17 Perbandingan Waktu Tunggu Arah Pulang	60

DAFTAR TABEL

Nomor Urut	Halaman
Table 1 Penelitian Terdahulu	18
Table 2 Pembagian Periode Operasional dan Kategori Load Factor	21
Table 3 Data yang Digunakan dalam Penelitian	21
Table 4 Variabel Input Penelitian	22
Table 5 Variabel Keputusan	22
Table 6 Variabel Output	23
Table 7 Parameter yang digunakan dalam model	28
Table 8 Parameter yang digunakan dalam implementasi Genetic Algorithm	33
Table 9 Parameter yang digunakan dalam implementasi ACO	37
Table 10 Halte Pada Rute Perjalanan Kampus Universitas Hasanuddin Tamalanrea-Kampus Universitas Hasanuddin Gowa	41
Table 11 Halte Pada Rute Perjalanan Kampus Universitas Hasanuddin Gowa-Kampus Universitas Hasanuddin Tamalanrea	42
Table 12 Jarak Antar Halte Rute Perjalanan Kampus Universitas Hasanuddin Tamalanrea-Kampus Universitas Hasanuddin Gowa	43
Table 13 Jarak Antar Halte Kampus Universitas Hasanuddin Gowa- Kampus Universitas Hasanuddin Tamalanrea	45
Table 14 Karakteristik Armada	46
Table 15 Jadwal Keberangkatan	46
Table 16 Parameter Biaya Operasional	47
Table 17 Parameter Operasional Dasar Koridor 5 Bus Trans Sulsel	47
Table 18 Hasil Pembangkitan Demand Penumpang per Periode Operasional	48
Table 19 Hasil Evaluasi Kondisi Eksisting (Baseline)	49
Table 20 Ringkasan Hasil Genetic Algorithm	50
Table 21 Detail Alokasi per Periode - Genetic Algorithm	51
Table 22 Ringkasan Hasil Ant Colony Optimization	52
Table 23 Detail Alokasi per Periode - Ant Colony Optimization	52
Table 24 Ringkasan Perbandingan Existing, GA, dan ACO	53
Table 25 Perbandingan Konfigurasi Operasional per Periode	53
Table 26 Ringkasan Analisis Sensitivitas Perubahan Demand	56
Table 27 Kontribusi Biaya Operasional Arah Pulang	59
Table 28 Perbandingan Waktu Tunggu Arah Pulang	60

DAFTAR LAMPIRAN

Nomor Urut	Halaman
Lampiran 1 Program Python Data Preparation	69
Lampiran 2 Program Python Demand Generator	71
Lampiran 3 Program Python Objective Function	78
Lampiran 4 Program Python Genetic Algorithm.....	84
Lampiran 5 Program Python Ant Colony Optimization.....	92
Lampiran 6 Program Python Performance Evaluation.....	100
Lampiran 7 Program Python Sensitivity Analysis.....	107
Lampiran 8 Program Python Visualization.....	113
Lampiran 9 Program Python Analisis Rute Pulang	122
Lampiran 10 Program Python Evaluasi Rute Pulang.....	126

DAFTAR NOTASI

Notasi	Keterangan
H	Jumlah halte
N	Jumlah armada bus yang tersedia
T	Himpunan / jumlah periode operasional, $T = \{1, 2, \dots, m\}$
B	Himpunan armada bus yang tersedia, $B = \{1, 2, \dots, n\}$
m	Jumlah periode operasional dalam satu hari
n	Jumlah armada bus yang tersedia (anggota himpunan B)
dij	Jarak antara halte i dan halte j (km)
C	Kapasitas bus (penumpang)
Pt	Jumlah/permintaan penumpang pada periode t
Co	Biaya operasional per kilometer (rupiah/km)
R	Panjang rute perjalanan pergi-pulang (km)
xt	Jumlah bus aktif (armada yang dioperasikan) pada periode t
qt	Jumlah trip pergi-pulang (PP) yang diberangkatkan pada periode t
ht	Headway (interval keberangkatan) pada periode t
kt	Kapasitas trip maksimum per bus pada periode t
OC	Total biaya operasional (Operational Cost)
WT	Total waktu tunggu penumpang (Waiting Time)
ES	Total penalti kursi kosong (Empty Seat)
SH	Total shortage (permintaan penumpang yang tidak terlayani)
LF	Load factor rata-rata
AU	Tingkat utilisasi armada (Armada Utilization)
Z	Nilai fungsi objektif
w1, w2, w3	Bobot kepentingan masing-masing komponen fungsi objektif
OC0, WT0, ES0	Nilai referensi (baseline kondisi eksisting) untuk normalisasi komponen fungsi objektif
P	Penalti shortage
λ	Koefisien (konstanta) penalti shortage
hmin, hmax	Headway minimum dan headway maksimum yang diizinkan
K	Representasi kromosom pada Genetic Algorithm
S	Representasi solusi pada Ant Colony Optimization
$\tau_{ij}(0)$	Nilai feromon awal pada pilihan (i, j)
τ_0	Konstanta feromon awal
η_{ij}	Informasi heuristik pada pilihan solusi (i, j)
Zij	Estimasi nilai fungsi objektif lokal pada pilihan (i, j)

P_{ij}	Probabilitas pemilihan alternatif j oleh semut pada kondisi i
α	Parameter pengaruh feromon (Ant Colony Optimization)
β	Parameter pengaruh informasi heuristik (Ant Colony Optimization)
A_i	Himpunan alternatif yang dapat dipilih oleh semut
ρ	Laju evaporasi feromon, $0 < \rho < 1$
$\Delta\tau_{ij}$	Penambahan (deposit) nilai feromon pada pilihan (i, j)
Q	Konstanta deposit feromon
Z_{best}	Nilai fungsi objektif dari solusi terbaik pada suatu iterasi
CT	Waktu komputasi (Computational Time)

DAFTAR ISTILAH

Istilah	Keterangan
Genetic Algorithm (GA)	Algoritma optimasi metaheuristik yang meniru proses evolusi biologis (seleksi, crossover, mutasi) untuk mencari solusi terbaik
Ant Colony Optimization (ACO)	Algoritma optimasi metaheuristik yang meniru perilaku koloni semut dalam mencari jalur/solusi terbaik melalui mekanisme feromon
Kromosom	Representasi satu alternatif solusi lengkap dalam Genetic Algorithm
Gen	Bagian/elemen penyusun kromosom yang mewakili satu variabel keputusan
Populasi	Kumpulan kromosom yang ada pada satu generasi dalam Genetic Algorithm
Generasi	Satu iterasi dalam proses evolusi Genetic Algorithm
Fitness	Ukuran kualitas suatu kromosom; semakin kecil nilai fungsi objektif, semakin besar nilai fitness
Seleksi Tournament Selection	Metode pemilihan induk (parent) dengan membandingkan nilai fitness dari beberapa kromosom yang dipilih secara acak
Crossover	Proses pertukaran sebagian gen dari dua kromosom induk untuk menghasilkan kromosom anak (offspring)
Mutasi	Proses perubahan nilai gen tertentu secara acak berdasarkan peluang mutation rate
Elitisme	Mekanisme mempertahankan kromosom dengan fitness terbaik agar tetap ada pada generasi berikutnya
Offspring	Kromosom anak yang dihasilkan dari proses crossover
Feromon	Nilai yang merepresentasikan tingkat ketertarikan semut terhadap suatu pilihan solusi dalam ACO
Evaporasi Feromon	Proses pengurangan nilai feromon secara bertahap untuk mencegah algoritma terjebak pada solusi lokal (local optimum)
Informasi Heuristik	Estimasi efisiensi lokal suatu pilihan solusi yang digunakan untuk memandu pencarian semut
Semut (Ant)	Agen pencari solusi dalam Ant Colony Optimization yang membangun satu solusi lengkap pada setiap iterasi
Headway	Interval waktu antar keberangkatan bus/trip pada suatu periode operasional
Trip	Satu kali perjalanan bus pergi-pulang (PP) dari titik keberangkatan ke titik tujuan dan kembali
Pergi-Pulang (PP)	Istilah untuk satu siklus perjalanan bus dari titik awal ke titik akhir rute dan kembali ke titik awal

Load Factor	Rasio tingkat keterisian penumpang terhadap kapasitas armada yang dioperasikan
Shortage	Kondisi permintaan penumpang yang tidak dapat terlayani oleh kapasitas armada yang tersedia
Empty Seat	Kapasitas kursi bus yang tidak termanfaatkan (tidak terisi penumpang)
Utilisasi Armada	Tingkat pemanfaatan armada bus yang tersedia dalam operasional
Fungsi Objektif	Fungsi matematis yang menjadi ukuran utama dalam proses optimasi, dalam penelitian ini diminimumkan
Baseline (Kondisi Eksisting)	Kondisi operasional awal (sebelum optimasi) yang digunakan sebagai pembandingan hasil optimasi
Rush Hour	Periode operasional dengan tingkat keterisian penumpang tinggi (50%–62,5%)
Non Rush Hour	Periode operasional dengan tingkat keterisian penumpang rendah (12,5%–25%)
Konvergensi	Kondisi ketika nilai solusi (fungsi objektif) tidak lagi mengalami perubahan signifikan pada iterasi/generasi lanjutan
Repair Perbaikan Solusi	Proses penyesuaian solusi agar memenuhi kendala model, misalnya batas total trip per hari
Kriteria Terminasi	Kondisi yang menentukan berhentinya proses iterasi suatu algoritma optimasi
Waktu Komputasi	Lama waktu yang dibutuhkan suatu algoritma untuk memperoleh solusi terbaik
WITA	Waktu Indonesia Tengah

BAB I

PENDAHULUAN

1.1 Latar Belakang

Transportasi publik merupakan salah satu komponen penting dalam pembangunan wilayah dan mobilitas masyarakat. Sistem transportasi publik yang efektif tidak hanya berfungsi sebagai sarana perpindahan manusia dari satu lokasi ke lokasi lainnya, tetapi juga berkontribusi dalam mengurangi kemacetan lalu lintas, menekan konsumsi energi, mengurangi emisi gas rumah kaca, serta meningkatkan produktivitas ekonomi dan kualitas hidup masyarakat. Oleh karena itu, peningkatan kualitas layanan transportasi publik menjadi salah satu fokus utama dalam pengembangan sistem transportasi perkotaan yang berkelanjutan (Korzeń & Kruszyna, 2025).

Salah satu layanan transportasi publik yang saat ini dikembangkan oleh Pemerintah Provinsi Sulawesi Selatan adalah Bus Trans Sulsel. Sistem transportasi ini melayani berbagai koridor yang menghubungkan pusat-pusat aktivitas masyarakat, termasuk koridor yang menghubungkan Kampus Universitas Hasanuddin Tamalanrea dengan Kampus Universitas Hasanuddin Gowa melalui sejumlah titik strategis seperti Mall Panakkukang. Koridor ini memiliki peran penting karena melayani mobilitas mahasiswa, dosen, tenaga kependidikan, dan masyarakat umum yang melakukan perjalanan antarwilayah setiap hari.

Meskipun demikian, pengelolaan operasional transportasi publik menghadapi berbagai tantangan. Salah satu permasalahan yang sering dijumpai adalah ketidakseimbangan antara kapasitas layanan yang disediakan dengan permintaan penumpang yang berubah sepanjang waktu. Pada jam-jam sibuk (*rush hour*), seperti pagi hari saat aktivitas perkuliahan dimulai dan sore hari ketika aktivitas berakhir, tingkat keterisian bus cenderung sangat tinggi bahkan mendekati kapasitas maksimum. Sebaliknya, pada jam-jam non-sibuk (*non-rush hour*), jumlah penumpang menurun sehingga sebagian besar kursi yang tersedia tidak terisi secara optimal. Berdasarkan hasil observasi awal pada koridor Bus Trans Sulsel yang melayani rute Kampus Universitas Hasanuddin Tamalanrea–Kampus Universitas Hasanuddin Gowa, tingkat keterisian bus pada periode non-rush hour diperkirakan hanya berada pada kisaran 50% dari kapasitas yang tersedia.

Kondisi tersebut menunjukkan adanya potensi inefisiensi dalam pengoperasian armada. Pada satu sisi, frekuensi perjalanan yang terlalu rendah pada jam sibuk dapat meningkatkan waktu tunggu penumpang dan menurunkan kualitas layanan. Namun pada sisi lain, frekuensi perjalanan yang terlalu tinggi pada jam non-sibuk dapat menyebabkan rendahnya tingkat utilisasi armada dan meningkatkan biaya operasional. Oleh karena itu, diperlukan suatu strategi optimasi yang mampu menyeimbangkan antara kebutuhan pelayanan penumpang dan efisiensi penggunaan sumber daya transportasi.

Permasalahan optimasi transportasi publik merupakan salah satu permasalahan yang kompleks karena melibatkan banyak faktor yang saling berinteraksi, seperti jumlah armada yang tersedia, jadwal keberangkatan, waktu tempuh perjalanan, kapasitas kendaraan, serta pola permintaan penumpang yang bersifat dinamis. Menurut Liu dan Ceder (2023), optimasi penjadwalan kendaraan dan pengaturan layanan transportasi publik dapat memberikan dampak yang signifikan terhadap efisiensi operasional sekaligus meningkatkan kualitas pelayanan kepada pengguna. Oleh karena itu, berbagai metode optimasi telah dikembangkan untuk menyelesaikan permasalahan tersebut.

Salah satu kelompok metode yang banyak digunakan adalah metode metaheuristik. Metode ini mampu menghasilkan solusi yang mendekati optimum dalam waktu komputasi yang relatif efisien, terutama pada permasalahan optimasi berskala besar yang sulit diselesaikan menggunakan metode eksak. Di antara berbagai metode metaheuristik yang tersedia, Algoritma Genetika (*Genetic Algorithm* atau GA) dan Ant Colony Optimization (ACO) merupakan dua metode yang banyak diterapkan pada berbagai permasalahan transportasi dan logistik.

Algoritma Genetika merupakan metode optimasi yang terinspirasi dari mekanisme evolusi biologis melalui proses seleksi, crossover, dan mutasi. Metode ini telah banyak digunakan dalam optimasi penjadwalan kendaraan dan transportasi karena kemampuannya dalam menjelajahi ruang solusi yang luas serta menghasilkan solusi berkualitas tinggi (Wang et al., 2024). Sementara itu, Ant Colony Optimization merupakan metode yang terinspirasi dari perilaku koloni semut dalam menemukan jalur terpendek menuju sumber makanan melalui mekanisme komunikasi berbasis feromon. ACO dikenal efektif dalam menyelesaikan permasalahan pencarian jalur, penugasan kendaraan, dan optimasi jaringan transportasi (Korzeń & Kruszyna, 2025).

Meskipun berbagai penelitian telah menunjukkan keberhasilan penerapan Algoritma Genetika maupun Ant Colony Optimization dalam bidang transportasi, kajian yang membandingkan kedua metode tersebut pada sistem operasional Bus Trans Sulsel, khususnya pada koridor yang menghubungkan Kampus Universitas Hasanuddin Tamalanrea dan Kampus Universitas Hasanuddin Gowa, masih sangat terbatas. Selain itu, karakteristik permintaan penumpang yang berbeda antara jam sibuk dan jam non-sibuk memberikan peluang untuk mengevaluasi kemampuan kedua metode dalam menghasilkan solusi operasional yang efisien.

Berdasarkan uraian tersebut, penelitian ini dilakukan untuk membandingkan kinerja Algoritma Genetika dan Ant Colony Optimization dalam menyelesaikan permasalahan optimasi operasional Bus Trans Sulsel pada koridor Kampus Universitas Hasanuddin Tamalanrea–Kampus Universitas Hasanuddin Gowa. Hasil penelitian diharapkan dapat memberikan rekomendasi metode yang lebih efektif dalam mengoptimalkan pengoperasian armada sehingga mampu menekan biaya operasional, meningkatkan utilisasi kendaraan, serta mempertahankan kualitas layanan kepada pengguna transportasi publik.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, maka rumusan masalah dalam penelitian ini adalah sebagai berikut:

1. Bagaimana menentukan konfigurasi operasional Bus Trans Sulsel yang optimal pada koridor Kampus Universitas Hasanuddin Tamalanrea–Kampus Universitas Hasanuddin Gowa sehingga dapat meminimalkan biaya operasional dan meningkatkan tingkat utilisasi armada?
2. Bagaimana kinerja Algoritma Genetika (*Genetic Algorithm*) dalam memperoleh solusi optimasi operasional Bus Trans Sulsel?
3. Bagaimana kinerja metode Ant Colony Optimization (ACO) dalam memperoleh solusi optimasi operasional Bus Trans Sulsel?
4. Metode manakah yang lebih efektif dalam menyelesaikan permasalahan optimasi operasional Bus Trans Sulsel ditinjau dari kualitas solusi dan efisiensi komputasi?

1.3 Batasan Masalah

Agar penelitian lebih terarah dan sesuai dengan tujuan yang ingin dicapai, maka batasan masalah dalam penelitian ini adalah sebagai berikut:

1. Penelitian difokuskan pada layanan Bus Trans Sulsel yang melayani koridor Kampus Universitas Hasanuddin Tamalanrea–Kampus Universitas Hasanuddin Gowa.
2. Data rute, halte, jarak antarhalte, dan jadwal operasional bus diperoleh dari data Bus Trans Sulsel yang tersedia pada koridor penelitian.
3. Permintaan penumpang pada setiap halte dan periode waktu tertentu diperoleh berdasarkan hasil observasi lapangan serta data simulasi yang dibangun berdasarkan karakteristik operasional Bus Trans Sulsel.
4. Penelitian tidak membahas perubahan jalur atau penambahan rute baru, melainkan hanya melakukan optimasi terhadap aspek operasional pada rute yang telah ditetapkan.
5. Faktor-faktor eksternal seperti kecelakaan lalu lintas, kondisi cuaca ekstrem, kerusakan kendaraan, demonstrasi, dan gangguan operasional lainnya tidak diperhitungkan dalam model.
6. Kapasitas setiap armada bus diasumsikan tetap selama periode pengamatan.
7. Parameter biaya operasional dihitung berdasarkan asumsi biaya perjalanan yang seragam untuk setiap armada yang beroperasi.
8. Metode optimasi yang digunakan dalam penelitian ini dibatasi pada Algoritma Genetika (*Genetic Algorithm*) dan Ant Colony Optimization (ACO).
9. Perbandingan kinerja kedua metode dilakukan berdasarkan nilai fungsi objektif yang diperoleh, tingkat utilisasi armada, serta waktu komputasi yang dibutuhkan untuk mencapai solusi.
10. Implementasi algoritma dan simulasi dilakukan menggunakan perangkat lunak Python.

1.4 Tujuan Penelitian

1. Menentukan konfigurasi operasional Bus Trans Sulsel yang optimal pada koridor Kampus Universitas Hasanuddin Tamalanrea–Kampus Universitas Hasanuddin

Gowa sehingga dapat meningkatkan utilisasi armada dan menurunkan biaya operasional.

2. Memperoleh solusi terbaik operasional Bus Trans Sulsel menggunakan Algoritma Genetika (*Genetic Algorithm*).
3. Memperoleh solusi terbaik operasional Bus Trans Sulsel menggunakan metode Ant Colony Optimization (ACO).
4. Menganalisis dan membandingkan performa Algoritma Genetika dan Ant Colony Optimization berdasarkan nilai fungsi objektif, tingkat utilisasi armada, konvergensi solusi, dan waktu komputasi.

1.5 Manfaat Penelitian

Penelitian ini diharapkan dapat memberikan manfaat baik secara teoritis maupun praktis sebagai berikut:

1.5.1 Manfaat Teoritis

1. Menambah kajian ilmiah dalam bidang optimasi, khususnya penerapan metode metaheuristik pada permasalahan transportasi publik.
2. Memberikan kontribusi dalam pengembangan aplikasi Algoritma Genetika (*Genetic Algorithm*) dan Ant Colony Optimization (ACO) untuk menyelesaikan permasalahan optimasi operasional transportasi.
3. Menjadi referensi bagi penelitian selanjutnya yang berkaitan dengan optimasi penjadwalan, alokasi armada, dan sistem transportasi publik menggunakan metode metaheuristik.

1.5.2 Manfaat Praktis

1. Bagi pengelola Bus Trans Sulsel, penelitian ini dapat memberikan alternatif solusi dalam pengaturan operasional armada yang lebih efisien sehingga mampu meningkatkan kualitas layanan dan utilisasi kendaraan.
2. Bagi Pemerintah Provinsi Sulawesi Selatan dan pemangku kepentingan di bidang transportasi, hasil penelitian ini dapat menjadi bahan pertimbangan dalam pengembangan sistem transportasi publik yang lebih efektif dan berkelanjutan.
3. Bagi masyarakat pengguna Bus Trans Sulsel, hasil penelitian ini diharapkan dapat mendukung peningkatan kualitas layanan melalui pengurangan waktu tunggu dan peningkatan kenyamanan perjalanan.
4. Bagi mahasiswa dan peneliti, penelitian ini dapat menjadi contoh penerapan metode optimasi metaheuristik pada permasalahan nyata yang berkaitan dengan transportasi publik.
5. Bagi penulis, penelitian ini menjadi sarana untuk memperdalam pemahaman mengenai pemodelan matematika, optimasi, serta implementasi Algoritma Genetika dan Ant Colony Optimization dalam menyelesaikan permasalahan dunia nyata.

1.6 Landasan Teori

1.6.1 Sistem Transportasi Publik

Transportasi publik merupakan salah satu komponen utama dalam sistem transportasi perkotaan yang berfungsi untuk menyediakan layanan mobilitas bagi masyarakat secara efisien dan berkelanjutan. Menurut Ceder (2015), transportasi publik adalah sistem transportasi yang dirancang untuk melayani pergerakan penumpang secara massal melalui rute, jadwal, dan kapasitas layanan yang telah ditentukan. Keberadaan transportasi publik tidak hanya berfungsi sebagai sarana perpindahan penumpang, tetapi juga berperan dalam mengurangi penggunaan kendaraan pribadi, mengurangi kemacetan lalu lintas, menekan konsumsi energi, dan mendukung pembangunan kota yang berkelanjutan.

Vuchic (2007) menjelaskan bahwa sistem transportasi publik yang efektif harus mampu mencapai keseimbangan antara kualitas pelayanan kepada pengguna dan efisiensi operasional penyelenggara transportasi. Kualitas pelayanan umumnya diukur melalui kenyamanan, aksesibilitas, keandalan jadwal, waktu perjalanan, serta waktu tunggu yang dialami oleh penumpang. Sementara itu, efisiensi operasional berkaitan dengan penggunaan armada, biaya operasional, dan kemampuan sistem dalam memenuhi permintaan perjalanan masyarakat.

Dalam proses perencanaan dan pengelolaan transportasi publik, beberapa indikator kinerja sering digunakan untuk mengevaluasi kualitas layanan dan efisiensi sistem. Menurut Ceder (2015), indikator tersebut meliputi waktu tunggu (*waiting time*), interval kedatangan kendaraan (*headway*), waktu perjalanan (*travel time*), tingkat keterisian kendaraan (*load factor*), utilisasi armada, dan biaya operasional. Indikator-indikator tersebut saling berkaitan dan menjadi dasar dalam pengambilan keputusan operasional pada sistem transportasi publik.

1.6.1.1 Waktu Tunggu (Waiting Time)

Waktu tunggu merupakan selang waktu yang diperlukan penumpang sejak tiba di halte hingga kendaraan datang untuk melayani perjalanan. Menurut Vuchic (2007), waktu tunggu merupakan salah satu faktor yang paling memengaruhi persepsi kualitas layanan transportasi publik karena secara langsung dirasakan oleh pengguna. Semakin kecil waktu tunggu, semakin tinggi tingkat kepuasan penumpang terhadap layanan yang diberikan.

1.6.1.2 Headway

Headway adalah selang waktu antara dua kendaraan yang beroperasi secara berurutan pada rute yang sama. Menurut Ceder (2015), headway merupakan parameter penting dalam perencanaan transportasi publik karena berpengaruh langsung terhadap kapasitas layanan dan waktu tunggu penumpang. Penentuan headway yang terlalu besar dapat meningkatkan waktu tunggu, sedangkan headway yang terlalu kecil dapat meningkatkan biaya operasional akibat bertambahnya jumlah armada yang harus dioperasikan.

1.6.1.3 Load Factor

Load factor menunjukkan tingkat pemanfaatan kapasitas kendaraan yang dihitung sebagai perbandingan antara jumlah penumpang dengan kapasitas kendaraan yang tersedia. Menurut Yu dan Wang (2018), load factor merupakan indikator penting dalam mengevaluasi efisiensi operasional transportasi publik karena dapat menggambarkan tingkat utilisasi armada yang digunakan. Nilai load factor yang terlalu rendah menunjukkan adanya kapasitas yang tidak termanfaatkan secara optimal, sedangkan nilai yang terlalu tinggi dapat mengurangi kenyamanan penumpang.

1.6.2 Bus Trans Sulsel

Bus Trans Sulsel merupakan sistem transportasi publik berbasis bus yang dikelola oleh Pemerintah Provinsi Sulawesi Selatan untuk melayani mobilitas masyarakat di kawasan Mamminasata (Makassar, Maros, Sungguminasa, dan Takalar). Layanan ini mulai beroperasi pada tahun 2025 sebagai bagian dari upaya pemerintah daerah untuk mempertahankan dan meningkatkan layanan angkutan massal setelah berakhirnya skema subsidi Program Teman Bus dari Kementerian Perhubungan (PT JAMKRIDA SULSEL, 2025).

Menurut Zona Redaksi (2026), pengembangan Bus Trans Sulsel bertujuan untuk menyediakan layanan transportasi yang aman, nyaman, terjangkau, dan terintegrasi antarwilayah di kawasan Mamminasata. Selain meningkatkan mobilitas masyarakat, keberadaan Bus Trans Sulsel juga diharapkan dapat mengurangi ketergantungan terhadap kendaraan pribadi sehingga mendukung sistem transportasi yang lebih berkelanjutan.

Pada tahap awal operasionalnya, Bus Trans Sulsel melayani beberapa koridor utama. Koridor 1 menghubungkan Panakkukang Square dengan Pelabuhan Galesong di Kabupaten Takalar. Koridor 2 melayani rute Kampus Universitas Hasanuddin Tamalanrea menuju Stasiun Kereta Api Mandai melalui Bandara Internasional Sultan Hasanuddin. Selain itu, terdapat Koridor 5 yang menghubungkan Kampus Universitas Hasanuddin Tamalanrea dengan Kampus Teknik Universitas Hasanuddin di Kabupaten Gowa melalui sejumlah titik strategis di Kota Makassar, termasuk kawasan Mall Panakkukang.

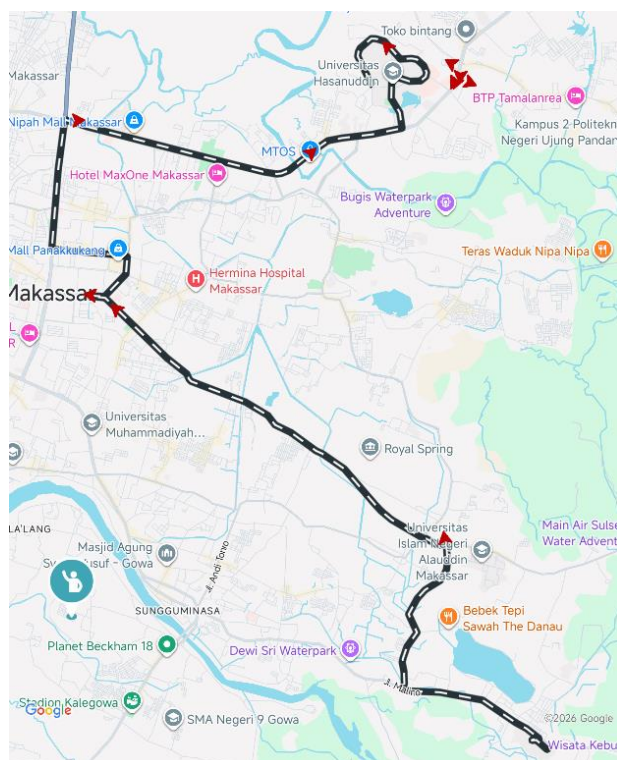
Koridor 5 memiliki karakteristik yang unik dibandingkan koridor lainnya karena melayani mobilitas harian mahasiswa, dosen, tenaga kependidikan, dan masyarakat umum yang melakukan perjalanan antara Kota Makassar dan Kabupaten Gowa. Berdasarkan informasi dari Dinas Perhubungan Provinsi Sulawesi Selatan, koridor ini merupakan salah satu jalur yang memiliki peranan penting dalam mendukung konektivitas kawasan pendidikan di wilayah Mamminasata.

Layanan Bus Trans Sulsel beroperasi setiap hari mulai pukul 05.30 hingga 21.00 WITA. Pada tahap awal implementasinya, jumlah armada yang beroperasi pada masing-masing koridor masih disesuaikan dengan ketersediaan kendaraan dan kebutuhan layanan. Pemerintah Provinsi Sulawesi Selatan menargetkan pengaturan interval

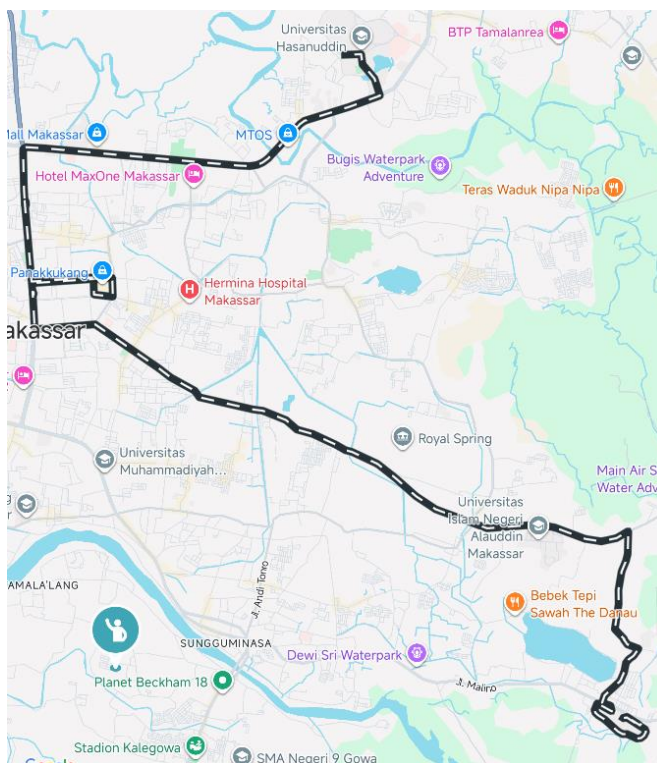
keberangkatan (*headway*) yang lebih baik seiring dengan bertambahnya jumlah armada yang beroperasi.

Sebagai sistem transportasi publik, keberhasilan operasional Bus Trans Sulsel sangat dipengaruhi oleh kemampuan pengelola dalam menyesuaikan kapasitas layanan dengan permintaan penumpang. Ketidaksesuaian antara jumlah armada yang beroperasi, jadwal keberangkatan, dan jumlah penumpang dapat menyebabkan terjadinya inefisiensi operasional. Pada kondisi tertentu, armada dapat mengalami kepadatan penumpang pada jam-jam sibuk (*rush hour*), sedangkan pada jam-jam non-sibuk (*non-rush hour*) tingkat keterisian kendaraan dapat menurun secara signifikan. Fenomena semacam ini umum dijumpai pada sistem transportasi publik dan menjadi salah satu alasan penting dilakukannya optimasi operasional transportasi (Saw et al., 2018).

Dalam penelitian ini, objek kajian difokuskan pada Koridor 5 Bus Trans Sulsel yang melayani rute Kampus Universitas Hasanuddin Tamalanrea–Kampus Teknik Universitas Hasanuddin Gowa. Data yang digunakan meliputi informasi halte, jarak antarhalte, jadwal keberangkatan, jumlah armada, serta skenario permintaan penumpang yang digunakan untuk membangun model optimasi operasional. Fokus utama penelitian adalah menentukan konfigurasi operasional yang lebih efisien melalui penerapan Algoritma Genetika (*Genetic Algorithm*) dan Ant Colony Optimization (ACO).



Gambar 1 Peta Geografis Koridor 5 (Unhas Tamalanrea→Unhas Gowa) Bus Trans Sulsel



Gambar 2 Peta Geografis Koridor 5 (Unhas Gowa→Unhas Tamalanrea) Bus Trans Sulsel

1.6.3 Optimasi Operasional Bus

Optimasi operasional bus merupakan salah satu aspek penting dalam pengelolaan sistem transportasi publik. Tujuan utama optimasi adalah memperoleh tingkat pelayanan yang baik bagi penumpang dengan penggunaan sumber daya yang efisien. Menurut Ceder (2015), proses perencanaan operasional transportasi publik umumnya terdiri atas empat tahap utama, yaitu perancangan jaringan rute (*network design*), penyusunan jadwal (*timetabling*), penjadwalan kendaraan (*vehicle scheduling*), dan penjadwalan awak kendaraan (*crew scheduling*). Setiap tahap saling berkaitan dan berpengaruh terhadap kualitas pelayanan maupun biaya operasional sistem transportasi publik.

Dalam praktiknya, operator transportasi publik menghadapi tantangan untuk menyeimbangkan antara kebutuhan pelayanan penumpang dan efisiensi biaya operasional. Peningkatan frekuensi keberangkatan dapat mengurangi waktu tunggu penumpang, namun pada saat yang sama akan meningkatkan jumlah armada yang harus dioperasikan serta biaya operasional yang ditanggung operator. Sebaliknya,

pengurangan frekuensi keberangkatan dapat menekan biaya operasional tetapi berpotensi meningkatkan waktu tunggu dan kepadatan penumpang. Oleh karena itu,

diperlukan pendekatan optimasi yang mampu menghasilkan kompromi terbaik antara kedua tujuan tersebut (Liu dan Ceder., 2023).

Menurut Mertens et al. (2023), optimasi pada sistem transportasi publik umumnya bertujuan untuk meminimalkan biaya operasi, meminimalkan jumlah armada yang digunakan, meningkatkan keteraturan jadwal, serta meningkatkan kualitas pelayanan kepada penumpang. Oleh karena itu, berbagai model optimasi telah dikembangkan untuk membantu pengambilan keputusan operasional dalam sistem transportasi publik.

1.6.3.1 Fleet Allocation Problem

Permasalahan lain yang berkaitan dengan operasional bus adalah *Fleet Allocation Problem*, yaitu menentukan jumlah armada yang harus dioperasikan pada periode waktu tertentu. Jumlah armada yang terlalu sedikit dapat menyebabkan kepadatan penumpang dan meningkatnya waktu tunggu, sedangkan jumlah armada yang terlalu banyak dapat menyebabkan rendahnya tingkat utilisasi kendaraan dan meningkatnya biaya operasional.

Menurut Wang et al. (2024), pola permintaan penumpang yang berbeda antara jam sibuk (*peak hour*) dan jam non-sibuk (*non-peak hour*) menyebabkan kebutuhan armada juga berubah sepanjang waktu. Oleh karena itu, jumlah armada dan interval keberangkatan perlu disesuaikan dengan pola permintaan penumpang agar sistem dapat beroperasi secara efisien.

1.6.3.2 Fungsi Objektif dalam Optimasi Operasional Bus

Dalam berbagai penelitian optimasi transportasi publik, fungsi objektif yang digunakan umumnya melibatkan satu atau lebih tujuan berikut:

1. Meminimalkan biaya operasional kendaraan.
2. Meminimalkan jumlah armada yang digunakan.
3. Meminimalkan waktu tunggu penumpang.
4. Meminimalkan tingkat kepadatan kendaraan.
5. Memaksimalkan tingkat utilisasi armada.
6. Memaksimalkan kualitas pelayanan transportasi.

Sui et al. (2025) menyatakan bahwa sebagian besar penelitian optimasi jadwal bus menggunakan tujuan berupa minimisasi biaya operasional, minimisasi jumlah kendaraan, dan peningkatan kapasitas pelayanan kepada penumpang.

Pada penelitian ini, optimasi operasional Bus Trans Sulsel difokuskan pada pengaturan jadwal keberangkatan dan pemanfaatan armada dengan mempertimbangkan variasi permintaan penumpang antara jam sibuk dan jam non-sibuk. Tujuan optimasi adalah memperoleh konfigurasi operasional yang mampu meningkatkan utilisasi armada dan menekan biaya operasional tanpa mengurangi kualitas pelayanan kepada pengguna.

1.6.3.5 Kompleksitas Permasalahan Optimasi Operasional Bus

Permasalahan optimasi operasional bus termasuk ke dalam kelompok permasalahan optimasi kombinatorial (*combinatorial optimization*) yang memiliki ruang solusi sangat besar. Setiap perubahan jadwal keberangkatan, alokasi kendaraan, maupun pengaturan armada dapat menghasilkan kombinasi solusi yang berbeda. Seiring bertambahnya jumlah halte, armada, dan perjalanan yang harus dilayani, kompleksitas perhitungan juga meningkat secara signifikan.

Basak et al. (2019) menjelaskan bahwa permasalahan optimasi jadwal transportasi publik umumnya sulit diselesaikan menggunakan metode eksak pada ukuran permasalahan yang besar sehingga banyak penelitian menggunakan metode metaheuristik seperti Genetic Algorithm, Particle Swarm Optimization, dan Ant Colony Optimization untuk memperoleh solusi yang mendekati optimum dalam waktu komputasi yang lebih efisien.

Berdasarkan uraian tersebut, optimasi operasional Bus Trans Sulsel pada penelitian ini dipandang sebagai permasalahan optimasi penjadwalan dan alokasi armada yang akan diselesaikan menggunakan dua metode metaheuristik, yaitu Genetic Algorithm (GA) dan Ant Colony Optimization (ACO), untuk memperoleh konfigurasi operasional yang lebih efisien.

1.6.4 Algoritma Genetika (Genetic Algorithm)

Algoritma Genetika (*Genetic Algorithm* atau GA) merupakan salah satu metode optimasi metaheuristik yang dikembangkan berdasarkan prinsip evolusi biologis dan seleksi alam yang diperkenalkan oleh Holland pada tahun 1975. Menurut Holland (1975), proses evolusi dalam suatu populasi terjadi melalui mekanisme seleksi individu terbaik yang memiliki peluang lebih besar untuk bertahan dan menghasilkan keturunan. Konsep ini kemudian diadaptasi ke dalam bidang optimasi untuk mencari solusi terbaik dari suatu permasalahan yang memiliki ruang solusi sangat besar dan kompleks.

Goldberg (1989) mendefinisikan Algoritma Genetika sebagai metode pencarian dan optimasi yang bekerja menggunakan sekumpulan solusi kandidat yang disebut populasi (*population*), kemudian melakukan proses seleksi, persilangan (*crossover*), dan mutasi (*mutation*) secara berulang hingga diperoleh solusi yang mendekati optimum. Berbeda dengan metode optimasi konvensional yang biasanya bergerak dari satu solusi ke solusi lainnya, Algoritma Genetika melakukan pencarian secara paralel melalui banyak solusi sekaligus sehingga memiliki kemampuan eksplorasi yang lebih baik.

Algoritma Genetika telah banyak digunakan dalam berbagai bidang, seperti optimasi transportasi, penjadwalan produksi, optimasi jaringan, penentuan rute kendaraan, hingga kecerdasan buatan. Menurut Wang et al. (2024), Genetic Algorithm mampu memberikan solusi yang efektif pada permasalahan penjadwalan transportasi publik yang memiliki banyak variabel keputusan dan kendala operasional.

1.6.4.1 Komponen Dasar Algoritma Genetika

Menurut Goldberg (1989), terdapat beberapa komponen utama yang membentuk Algoritma Genetika, yaitu kromosom, gen, populasi, dan fungsi fitness.

1. Gen (*Gene*)

Gen merupakan unit informasi terkecil yang merepresentasikan suatu variabel keputusan dalam permasalahan optimasi. Dalam konteks optimasi operasional bus, gen dapat merepresentasikan jadwal keberangkatan, alokasi armada, atau interval waktu antar keberangkatan (*headway*).

2. Kromosom (*Chromosome*)

Kromosom adalah kumpulan gen yang membentuk satu solusi kandidat. Setiap kromosom mewakili satu alternatif solusi yang akan dievaluasi kualitasnya menggunakan fungsi fitness.

Sebagai contoh, sebuah kromosom dapat direpresentasikan sebagai:

$$C = 10,12,15,8,11$$

yang menunjukkan jumlah armada atau konfigurasi jadwal pada beberapa periode operasi tertentu.

3. Populasi (*Population*)

Populasi merupakan sekumpulan kromosom yang digunakan pada satu generasi tertentu. Menurut Mitchell (1998), semakin besar ukuran populasi, semakin beragam solusi yang dapat dieksplorasi, namun kebutuhan komputasi juga akan meningkat.

4. Fungsi Fitness (*Fitness Function*)

Fungsi fitness digunakan untuk mengukur kualitas setiap solusi yang dihasilkan. Nilai fitness menjadi dasar dalam menentukan kromosom yang memiliki peluang lebih besar untuk dipilih pada proses reproduksi.

Dalam permasalahan optimasi operasional bus, fungsi fitness dapat dirancang berdasarkan kombinasi beberapa indikator, seperti:

$$Fitness = f(biaya\ operasional, waktu\ tunggu, utilisasi\ armada)$$

Semakin baik solusi yang diperoleh, semakin tinggi nilai fitness yang dihasilkan.

1.6.4.2 Tahapan Algoritma Genetika

Menurut Holland (1975) dan Goldberg (1989), Algoritma Genetika bekerja melalui beberapa tahapan utama sebagai berikut.

1. Inisialisasi Populasi

Proses diawali dengan membangkitkan sejumlah kromosom secara acak sebagai populasi awal.

2. Evaluasi Fitness

Setiap kromosom dievaluasi menggunakan fungsi fitness untuk menentukan kualitas solusi yang dihasilkan.

3. Seleksi (*Selection*)

Seleksi dilakukan untuk memilih kromosom yang memiliki kualitas terbaik sebagai induk (*parent*).

Beberapa metode seleksi yang sering digunakan adalah:

1. Roulette Wheel Selection
2. Tournament Selection
3. Rank Selection

Menurut Goldberg (1989), proses seleksi bertujuan mempertahankan solusi yang baik sehingga memiliki peluang lebih besar untuk menghasilkan generasi berikutnya.

4. Persilangan (*Crossover*)

Persilangan dilakukan dengan menggabungkan informasi dari dua kromosom induk untuk menghasilkan kromosom baru (*offspring*).

Sebagai contoh:

$$\begin{aligned} Parent_1 &= 1,2,3,4,5 \\ Parent_2 &= 6,7,8,9,10 \end{aligned}$$

Jika dilakukan *single-point crossover*, maka dapat diperoleh:

$$Child = 1,2,3,9,10$$

Crossover berfungsi meningkatkan eksplorasi ruang solusi.

5. Mutasi (*Mutation*)

Mutasi dilakukan dengan mengubah nilai gen tertentu secara acak.

Contoh:

$$1,2,3,4,5$$

menjadi

$$1,2,8,4,5$$

Mutasi bertujuan menjaga keberagaman populasi dan mencegah algoritma terjebak pada solusi lokal (*local optimum*).

6. Pembentukan Generasi Baru

Kromosom hasil seleksi, crossover, dan mutasi membentuk populasi baru yang akan dievaluasi kembali pada iterasi berikutnya.

7. Kriteria Terminasi

Proses iterasi dihentikan apabila salah satu kondisi berikut terpenuhi:

1. jumlah generasi maksimum tercapai;
2. nilai fitness telah konvergen;
3. solusi yang diinginkan telah diperoleh.

1.6.4.3 Kelebihan dan Kekurangan Algoritma Genetika

Menurut Mitchell (1998), beberapa kelebihan Algoritma Genetika adalah:

1. Mampu menangani permasalahan optimasi yang kompleks dan nonlinier.
2. Tidak memerlukan informasi turunan fungsi objektif.
3. Memiliki kemampuan eksplorasi global yang baik.
4. Dapat diterapkan pada berbagai jenis permasalahan optimasi.

Namun demikian, Algoritma Genetika juga memiliki beberapa kelemahan, yaitu:

1. Sensitif terhadap pemilihan parameter seperti ukuran populasi, probabilitas crossover, dan probabilitas mutasi.
2. Membutuhkan waktu komputasi yang relatif besar untuk populasi yang besar.
3. Tidak selalu menjamin diperolehnya solusi optimum global.

1.6.4.4 Penerapan Algoritma Genetika pada Optimasi Transportasi

Algoritma Genetika telah banyak diterapkan pada berbagai permasalahan transportasi, seperti optimasi rute kendaraan (*Vehicle Routing Problem*), penjadwalan transportasi publik, optimasi jaringan transportasi, serta pengaturan armada kendaraan. Wang et al. (2024) menunjukkan bahwa Genetic Algorithm mampu menghasilkan jadwal bus yang lebih efisien dengan mempertimbangkan variasi permintaan penumpang pada jam sibuk dan jam non-sibuk.

Dalam penelitian ini, Algoritma Genetika digunakan untuk menentukan konfigurasi operasional Bus Trans Sulsei yang optimal melalui pengaturan jadwal keberangkatan dan pemanfaatan armada. Kinerja metode ini selanjutnya akan dibandingkan dengan metode Ant Colony Optimization (ACO) berdasarkan kualitas solusi yang dihasilkan, tingkat utilisasi armada, dan waktu komputasi.

1.6.5 Ant Colony Optimization (ACO)

Ant Colony Optimization (ACO) merupakan salah satu metode optimasi metaheuristik yang terinspirasi oleh perilaku koloni semut dalam mencari jalur terpendek antara sarang dan sumber makanan. Metode ini pertama kali diperkenalkan oleh Dorigo (1992) dalam disertasinya mengenai optimasi kombinatorial. Menurut Dorigo dan Stützle (2004), semut mampu menemukan jalur terpendek secara kolektif melalui mekanisme komunikasi tidak langsung yang dikenal sebagai *stigmergy*, yaitu komunikasi melalui jejak feromon (*pheromone trail*) yang ditinggalkan pada lintasan yang dilalui.

Ketika semut bergerak dari sarang menuju sumber makanan, semut akan meninggalkan sejumlah feromon pada jalur yang dilaluinya. Jalur yang lebih pendek akan dilalui lebih sering sehingga akumulasi feromonnya menjadi lebih besar dibandingkan

jalur yang lebih panjang. Akibatnya, semut-semut lain cenderung memilih jalur dengan konsentrasi feromon yang lebih tinggi. Melalui proses berulang, seluruh koloni secara bertahap akan mengarahkan pergerakannya menuju jalur terbaik. Prinsip inilah yang menjadi dasar pengembangan Ant Colony Optimization untuk menyelesaikan berbagai permasalahan optimasi.

Menurut Dorigo dan Gambardella (1997), ACO sangat efektif digunakan untuk menyelesaikan permasalahan optimasi kombinatorial yang dapat direpresentasikan dalam bentuk graf, seperti *Travelling Salesman Problem* (TSP), *Vehicle Routing Problem* (VRP), penjadwalan transportasi, optimasi jaringan komunikasi, dan optimasi logistik. Kemampuan ACO dalam mengeksplorasi ruang solusi secara adaptif menjadikannya salah satu metode metaheuristik yang banyak digunakan dalam bidang transportasi.

1.6.5.1 Konsep Dasar Ant Colony Optimization

Dalam ACO, suatu permasalahan direpresentasikan sebagai graf yang terdiri atas himpunan simpul (*nodes*) dan sisi (*edges*). Setiap semut buatan (*artificial ant*) akan membangun solusi dengan bergerak dari satu simpul ke simpul lainnya berdasarkan informasi feromon dan informasi heuristik yang tersedia.

Menurut Dorigo dan Stützle (2004), terdapat dua komponen utama yang memengaruhi keputusan semut dalam memilih jalur, yaitu:

1. Feromon (Pheromone)

Feromon merupakan informasi kolektif yang diperoleh dari pengalaman semut sebelumnya. Semakin besar nilai feromon pada suatu jalur, semakin besar peluang jalur tersebut dipilih oleh semut berikutnya.

Nilai feromon pada sisi (i, j) biasanya dinyatakan sebagai:

$$\tau_{ij}$$

dengan:

$$\tau_{ij} > 0$$

Semakin besar nilai (τ_{ij}), semakin menarik jalur tersebut bagi semut.

2. Informasi Heuristik (Heuristic Information)

Informasi heuristik menggambarkan kualitas lokal suatu jalur. Pada permasalahan transportasi, informasi heuristik biasanya didasarkan pada jarak atau waktu tempuh.

Secara umum:

$$\eta_{ij} = \frac{1}{d_{ij}}$$

dengan:

(η_{ij}) = nilai heuristik pada sisi ((i, j)),

(d_{ij}) = jarak atau biaya perjalanan dari simpul (i) ke simpul (j).

Semakin pendek jarak yang ditempuh, semakin besar nilai heuristik yang dimiliki.

1.6.5.2 Probabilitas Pemilihan Jalur

Menurut Dorigo dan Gambardella (1997), probabilitas semut ke-(k) berpindah dari simpul (i) menuju simpul (j) diberikan oleh:

$$P_{ij}^{(k)} = \frac{(\tau_{ij})^\alpha (\eta_{ij})^\beta}{\sum_{l \in N_i} (\tau_{il})^\alpha (\eta_{il})^\beta}$$

dengan:

$(P_{ij}^{(k)})$ = probabilitas pemilihan jalur,

(τ_{ij}) = intensitas feromon,

(η_{ij}) = informasi heuristik,

(α) = pengaruh feromon,

(β) = pengaruh informasi heuristik,

(N_i) = himpunan simpul yang dapat dikunjungi dari simpul (i).

Parameter (α) dan (β) berperan penting dalam menentukan keseimbangan antara eksploitasi pengalaman sebelumnya dan eksplorasi solusi baru.

1.6.5.3 Pembaruan Feromon

Setelah seluruh semut menyelesaikan solusi pada suatu iterasi, nilai feromon diperbarui. Menurut Dorigo dan Stützle (2004), proses pembaruan feromon dilakukan melalui dua mekanisme:

1. Evaporasi Feromon

Feromon akan berkurang secara bertahap agar algoritma tidak terlalu bergantung pada solusi lama.

Persamaan evaporasi:

$$\tau_{ij} \leftarrow (1 - \rho)\tau_{ij}$$

dengan:

(ρ) = laju evaporasi feromon,

$(0 < \rho < 1)$.

2. Penambahan Feromon

Jalur yang menghasilkan solusi lebih baik akan memperoleh tambahan feromon.

$$\tau_{ij} \leftarrow \tau_{ij} + \Delta\tau_{ij}$$

dengan:

$$\Delta\tau_{ij} = \sum_{k=1}^m \Delta\tau_{ij}^{(k)}$$

di mana (m) adalah jumlah semut yang digunakan.

1.6.5.4 Tahapan Algoritma Ant Colony Optimization

Menurut Dorigo dan Stützle (2004), tahapan utama ACO adalah sebagai berikut:

1. Inisialisasi parameter algoritma.
2. Inisialisasi nilai feromon awal.
3. Setiap semut membangun solusi berdasarkan probabilitas pemilihan jalur.
4. Evaluasi kualitas solusi yang diperoleh.
5. Melakukan evaporasi feromon.
6. Melakukan pembaruan feromon berdasarkan solusi terbaik.
7. Mengulangi proses hingga memenuhi kriteria terminasi.
8. Menentukan solusi terbaik yang diperoleh.

1.6.5.5 Kelebihan dan Kekurangan Ant Colony Optimization

Menurut Dorigo dan Stützle (2004), beberapa kelebihan Ant Colony Optimization adalah:

1. Sangat efektif untuk permasalahan jaringan dan rute.
2. Memiliki kemampuan pencarian global yang baik.
3. Dapat beradaptasi terhadap perubahan kondisi permasalahan.
4. Mampu menghasilkan solusi berkualitas tinggi pada permasalahan kombinatorial.

Namun demikian, ACO juga memiliki beberapa kelemahan, yaitu:

1. Sensitif terhadap pemilihan parameter (α), (β), dan (ρ).
2. Dapat mengalami stagnasi apabila feromon terlalu terkonsentrasi pada jalur tertentu.
3. Membutuhkan waktu komputasi yang relatif besar pada ukuran masalah yang sangat besar.

1.6.5.6 Penerapan Ant Colony Optimization pada Sistem Transportasi

Ant Colony Optimization telah banyak digunakan dalam berbagai permasalahan transportasi. Korzeń dan Kruszyna (2025) menunjukkan bahwa ACO mampu menghasilkan solusi yang efektif dalam perencanaan transportasi publik dengan mempertimbangkan efisiensi jaringan dan kebutuhan pengguna. Selain itu, Dorigo dan Gambardella (1997) menunjukkan bahwa ACO memberikan performa yang sangat baik pada permasalahan pencarian rute dan optimasi jaringan transportasi.

Dalam penelitian ini, Ant Colony Optimization digunakan untuk menentukan konfigurasi operasional Bus Trans Sulsel yang optimal berdasarkan data rute, jadwal perjalanan, dan permintaan penumpang. Kinerja metode ini akan dibandingkan dengan

Algoritma Genetika berdasarkan nilai fungsi objektif, tingkat utilisasi armada, dan waktu komputasi yang diperlukan untuk memperoleh solusi terbaik.

1.6.6 Penelitian Terdahulu

Berbagai penelitian telah dilakukan untuk menyelesaikan permasalahan optimasi transportasi publik menggunakan metode metaheuristik. Metode yang paling banyak digunakan antara lain Genetic Algorithm (GA) dan Ant Colony Optimization (ACO) karena keduanya mampu menangani permasalahan optimasi kombinatorial yang kompleks dan memiliki ruang solusi yang besar.

Wang et al. (2024) menerapkan Genetic Algorithm untuk mengoptimalkan penjadwalan bus dengan mempertimbangkan variasi permintaan penumpang pada jam sibuk dan jam non-sibuk. Hasil penelitian menunjukkan bahwa Genetic Algorithm mampu menurunkan biaya operasional sekaligus meningkatkan tingkat pelayanan kepada penumpang melalui penyesuaian jadwal keberangkatan dan jumlah armada yang beroperasi.

Basak et al. (2019) mengembangkan pendekatan berbasis data (*data-driven optimization*) untuk optimasi jadwal transportasi publik. Penelitian tersebut menunjukkan bahwa metode optimasi mampu meningkatkan efisiensi sistem transportasi melalui pemanfaatan data permintaan penumpang secara lebih efektif.

Korzeń dan Kruszyna (2025) menggunakan Ant Colony Optimization untuk menyelesaikan permasalahan perencanaan transportasi publik. Hasil penelitian menunjukkan bahwa ACO mampu menghasilkan jaringan transportasi yang lebih efisien dengan mempertimbangkan kebutuhan pengguna dan karakteristik jaringan transportasi yang tersedia.

Dorigo dan Gambardella (1997) menerapkan Ant Colony System pada permasalahan Travelling Salesman Problem yang menjadi dasar berbagai pengembangan aplikasi ACO dalam bidang transportasi. Penelitian tersebut menunjukkan bahwa ACO mampu menghasilkan solusi berkualitas tinggi pada permasalahan pencarian rute yang kompleks.

Liu dan Ceder (2023) meneliti integrasi antara penjadwalan kendaraan dan koordinasi jadwal layanan transportasi publik. Hasil penelitian menunjukkan bahwa integrasi kedua aspek tersebut dapat meningkatkan efisiensi operasional dan menurunkan biaya transportasi tanpa mengurangi kualitas layanan kepada penumpang.

Blum (2005) melakukan kajian komprehensif mengenai perkembangan Ant Colony Optimization dan menunjukkan bahwa metode ini sangat efektif untuk menyelesaikan berbagai permasalahan optimasi kombinatorial, termasuk perencanaan transportasi dan optimasi jaringan.

Mitchell (1998) menjelaskan bahwa Genetic Algorithm memiliki kemampuan eksplorasi global yang baik sehingga banyak digunakan pada permasalahan optimasi

yang memiliki banyak solusi lokal. Karakteristik ini menjadikan Genetic Algorithm sebagai salah satu metode yang banyak diterapkan dalam optimasi transportasi.

Mavrovouniotis et al. (2018) menunjukkan bahwa Ant Colony Optimization memiliki kemampuan adaptasi yang baik terhadap perubahan lingkungan sehingga cocok digunakan pada permasalahan transportasi yang bersifat dinamis.

Berdasarkan berbagai penelitian tersebut dapat disimpulkan bahwa Genetic Algorithm dan Ant Colony Optimization memiliki kemampuan yang baik dalam menyelesaikan permasalahan optimasi transportasi. Namun sebagian besar penelitian terdahulu hanya berfokus pada penerapan salah satu metode secara terpisah. Penelitian yang secara khusus membandingkan kinerja Genetic Algorithm dan Ant Colony Optimization pada sistem operasional Bus Trans Sulsei, khususnya Koridor 5 yang menghubungkan Kampus Universitas Hasanuddin Tamalanrea dan Kampus Teknik Universitas Hasanuddin Gowa, masih sangat terbatas.

Oleh karena itu, penelitian ini dilakukan untuk membandingkan kinerja Genetic Algorithm dan Ant Colony Optimization dalam menyelesaikan permasalahan optimasi operasional Bus Trans Sulsei dengan mempertimbangkan jadwal perjalanan, utilisasi armada, dan biaya operasional. Hasil penelitian diharapkan dapat memberikan rekomendasi metode yang lebih efektif untuk diterapkan pada sistem transportasi publik yang memiliki karakteristik serupa.

Table 1 Penelitian Terdahulu

No	Peneliti	Tahun	Metode	Kasus	Hasil Utama
1	Dorigo & Gambardella	1997	ACO	Travelling Salesman Problem	ACO menghasilkan solusi rute yang efisien
2	Mitchell	1998	GA	Optimasi umum	GA memiliki kemampuan eksplorasi global yang baik
3	Basak et al.	2019	Data-driven Optimization	Jadwal transportasi publik	Meningkatkan efisiensi jadwal
4	Liu & Ceder	2023	Timetabling & Scheduling	Transportasi publik	Menurunkan biaya operasional
5	Wang et al.	2024	GA	Penjadwalan bus	Meningkatkan kualitas layanan
6	Korzeń & Kruszyna	2025	ACO	Transportasi publik	Menghasilkan jaringan yang lebih efisien
7	Mavrovouniotis et al.	2018	ACO	Dynamic Optimization Bus Trans	Adaptif terhadap perubahan kondisi
8	Penelitian ini	2026	GA vs ACO	Sulsei Koridor 5	Membandingkan performa kedua metode

Berdasarkan hasil telaah literatur, penelitian mengenai optimasi transportasi publik menggunakan Genetic Algorithm maupun Ant Colony Optimization telah banyak dilakukan. Namun penelitian yang secara khusus membandingkan kedua metode tersebut pada sistem operasional Bus Trans Sulsei, khususnya Koridor 5 yang menghubungkan Kampus Universitas Hasanuddin Tamalanrea dan Kampus Teknik Universitas Hasanuddin Gowa, belum ditemukan. Selain itu, karakteristik permintaan penumpang yang berbeda antara jam sibuk dan jam non-sibuk pada koridor ini memberikan konteks yang menarik untuk mengevaluasi efektivitas kedua metode dalam menghasilkan solusi operasional yang efisien. Oleh karena itu, penelitian ini diharapkan dapat mengisi kesenjangan tersebut dan memberikan kontribusi pada pengembangan optimasi transportasi publik berbasis metaheuristik.

1.6.7 Kerangka Konseptual Penelitian

Operasional Bus Trans Sulsei pada Koridor 5 yang menghubungkan Kampus Universitas Hasanuddin Tamalanrea dan Kampus Teknik Universitas Hasanuddin Gowa menghadapi tantangan dalam menjaga keseimbangan antara kualitas pelayanan dan efisiensi operasional. Berdasarkan hasil observasi awal, tingkat keterisian armada cenderung tinggi pada jam sibuk (*rush hour*) dan relatif rendah pada jam non-sibuk (*non-rush hour*). Kondisi ini mengindikasikan adanya peluang untuk melakukan optimasi terhadap jadwal keberangkatan dan pemanfaatan armada.

Penelitian ini menggunakan data operasional Bus Trans Sulsei yang meliputi data halte, jarak antarhalte, jadwal keberangkatan, jumlah armada, dan skenario permintaan penumpang. Data tersebut digunakan untuk membangun model optimasi operasional bus yang bertujuan meningkatkan utilisasi armada dan menurunkan biaya operasional tanpa mengurangi kualitas pelayanan kepada pengguna.

Model optimasi yang dibangun selanjutnya diselesaikan menggunakan dua metode metaheuristik, yaitu Genetic Algorithm (GA) dan Ant Colony Optimization (ACO). Kedua metode digunakan secara terpisah untuk memperoleh solusi terbaik berdasarkan fungsi objektif yang telah ditetapkan.

Selanjutnya dilakukan analisis perbandingan terhadap kedua metode berdasarkan beberapa indikator kinerja, yaitu nilai fungsi objektif, tingkat utilisasi armada, tingkat keterisian kendaraan (*load factor*), serta waktu komputasi yang diperlukan untuk memperoleh solusi. Hasil perbandingan tersebut digunakan untuk menentukan metode yang lebih efektif dalam menyelesaikan permasalahan optimasi operasional Bus Trans Sulsei pada koridor yang diteliti.

BAB II

METODE PENELITIAN

2.1 Waktu dan Lokasi Penelitian

2.1.1 Waktu Penelitian

Penelitian ini dilaksanakan pada periode Februari 2026 hingga Juli 2026. Kegiatan penelitian meliputi pengumpulan dan pengolahan data, pembangunan model optimasi, implementasi algoritma, analisis hasil, serta penyusunan laporan penelitian.

2.1.2 Lokasi Penelitian

Penelitian ini menggunakan data operasional Bus Trans Sulsel pada Koridor 5 yang melayani rute Kampus Universitas Hasanuddin Tamalanrea-Kampus Teknik Universitas Hasanuddin Gowa melalui beberapa titik pemberhentian di wilayah Kota Makassar dan Kabupaten Gowa.

Secara administratif, wilayah penelitian mencakup: (1) Kampus Universitas Hasanuddin Tamalanrea, Kota Makassar; (2) kawasan Mall Panakkukang, Kota Makassar; (3) kawasan Samata, Kabupaten Gowa; dan (4) Kampus Teknik Universitas Hasanuddin Gowa.

Penelitian ini bersifat komputasional dan berbasis simulasi sehingga seluruh proses pemodelan, implementasi algoritma, dan analisis dilakukan di Laboratorium Komputasi Departemen Matematika Fakultas Matematika dan Ilmu Pengetahuan Alam Universitas Hasanuddin.

2.2 Sumber dan Jenis Data

Penelitian ini menggunakan data sekunder dan data simulasi yang berkaitan dengan operasional Bus Trans Sulsel pada Koridor 5 yang melayani rute Kampus Universitas Hasanuddin Tamalanrea–Kampus Teknik Universitas Hasanuddin Gowa. Data yang digunakan merupakan kombinasi antara data operasional aktual dan data yang dibangun berdasarkan asumsi penelitian untuk mendukung proses optimasi.

2.2.1 Sumber Data

Sumber data dalam penelitian ini terdiri atas tiga kelompok, yaitu data operasional Bus Trans Sulsel (halte, jarak antarhalte, jadwal keberangkatan, jumlah armada, jumlah trip), data hasil observasi lapangan mengenai pola keterisian penumpang, dan data simulasi yang dibangun karena data historis jumlah penumpang per halte dan per periode waktu belum tersedia secara lengkap.

Berdasarkan hasil observasi awal, tingkat keterisian armada cenderung tinggi pada jam sibuk (rush hour) dan menurun pada jam non-sibuk (non-rush hour), dengan

keterisian pada periode non-sibuk diperkirakan berada pada kisaran 50% dari kapasitas armada. Menurut Law (2015), penggunaan simulasi merupakan pendekatan yang umum digunakan dalam penelitian sistem transportasi ketika data aktual tidak tersedia secara lengkap, dengan tetap memperhatikan kesesuaian asumsi terhadap kondisi nyata.

2.2.2 Jenis Data

Berdasarkan sifatnya, data yang digunakan terdiri atas data kuantitatif (jarak antarhalte, panjang rute, jumlah armada, jumlah trip, jadwal keberangkatan, kapasitas kendaraan, jumlah penumpang, load factor, biaya operasional) dan data kualitatif (karakteristik rute, karakteristik operasional armada, klasifikasi periode operasional, dan pola permintaan penumpang berdasarkan waktu perjalanan).

2.2.3 Periode Operasional dan Skenario Permintaan Penumpang

Layanan Bus Trans Sulsel Koridor 5 beroperasi mulai pukul 05.00 hingga 22.00 WITA. Untuk keperluan pemodelan, hari operasional dibagi menjadi enam periode (P1-P6) yang masing-masing dikategorikan sebagai Rush Hour, Normal, atau Non Rush Hour berdasarkan karakteristik keterisian penumpang yang teramati di lapangan, sebagaimana disajikan pada Tabel 2.

Table 2 Pembagian Periode Operasional dan Kategori Load Factor

Periode	Waktu	Kategori	Rentang Load Factor
P1	05.00 - 06.30	Non Rush Hour	12,5% - 25%
P2	06.00 - 09.00	Rush Hour	50% - 62,5%
P3	09.00 - 12.00	Normal	25% - 50%
P4	12.00 - 15.00	Non Rush Hour	12,5% - 25%
P5	15.00 - 18.00	Rush Hour	50% - 62,5%
P6	18.00 - 21.00	Normal	25% - 50%

2.2.4 Ringkasan Data Penelitian

Ringkasan data yang digunakan dalam penelitian ini disajikan pada Tabel 3.

Table 3 Data yang Digunakan dalam Penelitian

No	Data	Sumber	Kegunaan
1	Daftar halte	Bus Trans Sulsel	Pembentukan jaringan rute
2	Jarak antarhalte	Bus Trans Sulsel	Perhitungan jarak perjalanan
3	Jadwal keberangkatan	Bus Trans Sulsel	Pembanding baseline (eksisting)
4	Jumlah armada	Bus Trans Sulsel	Kendala model
5	Jumlah trip	Bus Trans Sulsel	Kendala model
6	Kapasitas armada	Bus Trans Sulsel	Evaluasi utilisasi
7	Permintaan penumpang	Simulasi	Fungsi objektif

No	Data	Sumber	Kegunaan
8	Load factor	Simulasi	Evaluasi solusi
9	Biaya operasional	Asumsi penelitian	Fungsi objektif

2.3 Variabel Penelitian

Variabel yang digunakan dalam penelitian ini terdiri atas variabel input, variabel keputusan, dan variabel output yang digunakan dalam proses optimasi operasional Bus Trans Sulsel pada Koridor 5 yang menghubungkan Kampus Universitas Hasanuddin Tamalanrea dan Kampus Teknik Universitas Hasanuddin Gowa.

2.3.1 Variabel Input

Variabel input merupakan data yang digunakan sebagai masukan dalam model optimasi.

Table 4 Variabel Input Penelitian

Simbol	Keterangan	Satuan
H	Jumlah halte	halte
N	Jumlah armada bus	unit
T	Jumlah periode operasional	periode
dij	Jarak antara halte i dan halte j	km
C	Kapasitas bus	penumpang
Pt	Jumlah penumpang pada periode t	orang
Co	Biaya operasional per kilometer	rupiah/km
R	Panjang rute perjalanan (pergi-pulang)	km

2.3.2 Variabel Keputusan

Variabel keputusan merupakan variabel yang nilainya akan ditentukan melalui proses optimasi.

Table 5 Variabel Keputusan

Simbol	Keterangan
xt	Jumlah armada yang dioperasikan pada periode t
qt	Jumlah trip PP (pergi-pulang) yang diberangkatkan pada periode t
ht	Headway pada periode t

Variabel keputusan utama pada penelitian ini adalah jumlah bus aktif (xt), jumlah trip PP yang dijalankan (qt), dan interval keberangkatan/headway (ht) pada setiap periode operasional. xt dan qt sengaja dibedakan karena keduanya memiliki makna berbeda: xt menyatakan berapa unit armada yang digunakan pada periode t, sedangkan qt menyatakan berapa kali trip PP benar-benar diberangkatkan pada periode tersebut. Satu bus dapat menjalankan lebih dari satu trip PP pada periode yang sama apabila durasi periode memungkinkan, sehingga qt dan xt tidak selalu bernilai sama

2.3.3 Variabel Output

Variabel output merupakan hasil yang diperoleh setelah proses optimasi dilakukan.

Table 6 Variabel Output

Simbol	Keterangan
OC	Total biaya operasional
WT	Total waktu tunggu penumpang
ES	Total penalti kursi kosong (empty seat)
SH	Total shortage (penumpang yang tidak terlayani)
LF	Load factor rata-rata
AU	Tingkat utilisasi armada
Z	Nilai fungsi objektif

2.3.4 Definisi Operasional Variabel

1. Headway (h_t)

Headway adalah selang waktu antara dua keberangkatan bus yang berurutan pada periode operasional tertentu. Headway dinyatakan dalam satuan menit.

2. Load Factor (LF)

Load factor merupakan rasio antara jumlah penumpang yang menggunakan layanan dengan kapasitas kendaraan yang tersedia. Menurut Yu dan Wang (2018), load factor dapat dihitung menggunakan persamaan:

$$LF = \frac{\text{Jumlah Penumpang}}{\text{Kapasitas Bus}} \times 100\% \quad (1)$$

3. Utilisasi Armada (AU)

Mengacu pada konsep utilisasi armada sebagai indikator kinerja transportasi publik menurut Ceder (2015), utilisasi armada pada penelitian ini dirumuskan sebagai:

$$AU = \frac{\text{Jumlah Armada Aktif}}{\text{Jumlah Armada Tersedia}} \times 100\% \quad (2)$$

4. Biaya Operasional (OC)

Biaya operasional merupakan total biaya yang dikeluarkan untuk menjalankan trip PP (pergi-pulang) selama periode operasional. Perlu ditegaskan bahwa biaya operasional dihitung berdasarkan jumlah trip PP yang benar-benar dijalankan (q_t), bukan berdasarkan jumlah bus aktif (x_t), karena satu bus dapat menjalankan lebih dari satu trip pada periode yang sama. Biaya satu trip PP didefinisikan sebagai $c_{trip} = C_o \times R$, sehingga biaya operasional total dirumuskan sebagai:

$$OC = c_{trip} \sum_t q_t = C_o \times R \times \sum_t q_t \quad (3)$$

dengan:

C_o = biaya operasional per kilometer,

R = panjang rute (pergi-pulang),

c_{trip} = biaya satu trip PP,

q_t = jumlah trip PP yang diberangkatkan pada periode t .

5. Waktu Tunggu Penumpang (WT)

Mengikuti pendekatan Ceder (2015), dengan asumsi kedatangan penumpang di halte bersifat acak, rata-rata waktu tunggu penumpang diperkirakan sebagai:

$$WT = \frac{h_t}{2} \quad (3)$$

dengan h_t = headway pada periode t (menit).

6. Shortage (SH)

Jumlah penumpang yang tidak dapat terlayani terjadi apabila kapasitas total trip PP pada suatu periode ($40q_t$) lebih kecil daripada permintaan pada periode tersebut, dihitung sebagai:

$$SH = \sum_t \max(0, P_t - 40q_t) \quad (4)$$

dengan:

P_t = permintaan penumpang pada periode

t , & q_t = jumlah trip PP pada periode t (sehingga $40q_t$ adalah total kapasitas layanan pada periode tersebut).

2.3.5 Hubungan Bus Aktif (x_t) dan Trip PP (q_t)

Sebelum menetapkan hubungan antarvariabel secara umum, perlu ditegaskan terlebih dahulu hubungan antara jumlah bus aktif (x_t) dan jumlah trip PP (q_t) pada periode yang sama, karena keduanya sering tertukar maknanya.

Lima periode operasional (Tabel 2) hanyalah pembagian waktu berdasarkan tingkat kesibukan, bukan enam kelompok armada yang masing-masing berisi 15 bus, sehingga jumlah periode tidak boleh langsung dikalikan dengan jumlah armada ($m \times N$). Jumlah trip PP harian yang tersedia dibatasi oleh jumlah armada dan jumlah trip maksimum per bus, yaitu:

$$\sum_t q_t \leq N \times 4 = 15 \times 4 = 60 \quad (5)$$

dengan N = jumlah armada tersedia (15 unit), dan 4 = jumlah trip PP maksimum yang dapat dijalankan satu bus per hari.

Secara umum, jumlah trip pada periode t dibatasi oleh jumlah bus aktif pada periode tersebut dikalikan kapasitas trip maksimum satu bus pada periode itu (k_t):

$$q_t \leq k_t x_t \quad (6)$$

dengan $k_t = \left\lfloor \frac{L_t}{T_{cycle}} \right\rfloor$, L_t adalah lama periode t (menit), dan T_{cycle} adalah waktu yang dibutuhkan untuk menyelesaikan satu trip PP (termasuk waktu perjalanan dan waktu berhenti). Karena durasi satu trip PP pada penelitian ini berlangsung selama beberapa jam, satu bus pada umumnya hanya dapat menjalankan paling banyak satu trip PP dalam satu periode ($k_t = 1$), sehingga secara praktis berlaku:

$$q_t \leq x_t \quad (7)$$

dengan q_t = jumlah trip PP dan x_t = jumlah bus aktif pada periode t yang sama (berlaku pada kasus $k_t = 1$).

Apabila setiap bus yang aktif pada periode t dipastikan menjalankan tepat satu trip pada periode tersebut, maka $q_t = x_t$; namun kesamaan ini tidak boleh diasumsikan begitu saja dan harus diperiksa terhadap data jadwal aktual (lama periode dan durasi trip). Biaya operasional, kapasitas layanan (kursi kosong dan shortage), dan headway seluruhnya dihitung berdasarkan q_t , sedangkan x_t hanya digunakan untuk menghitung utilisasi armada (AU) dan sebagai batas ketersediaan bus ($x_t \leq N$).

2.3.6 Hubungan Antar Variabel

Dalam penelitian ini, jumlah bus aktif, jumlah trip PP, dan headway merupakan variabel keputusan yang akan dioptimalkan menggunakan Genetic Algorithm (GA) dan Ant Colony Optimization (ACO). Variabel tersebut memengaruhi biaya operasional, load factor, utilisasi armada, dan waktu tunggu penumpang yang selanjutnya digunakan untuk mengevaluasi kualitas solusi yang dihasilkan. Hubungan antar variabel dalam penelitian ini dapat digambarkan sebagai berikut:

$$(x_t, q_t, h_t) \rightarrow (OC, WT, ES, SH, LF, AU) \rightarrow Z \quad (8)$$

dengan x_t, q_t, h_t = variabel keputusan (bus aktif, trip PP, headway); OC, WT, ES, SH, LF, AU = indikator kinerja yang dihitung dari ketiga variabel keputusan tersebut (biaya operasional, waktu tunggu, kursi kosong, shortage, load factor, dan utilisasi armada); dan Z menyatakan nilai fungsi objektif yang akan dioptimalkan.

2.4 Asumsi Penelitian

Untuk mempermudah proses pemodelan dan optimasi operasional Bus Trans Sulsel, penelitian ini menggunakan beberapa asumsi sebagai berikut:

1. Rute perjalanan Bus Trans Sulsel pada Koridor 5 yang menghubungkan Kampus Universitas Hasanuddin Tamalanrea dan Kampus Teknik Universitas Hasanuddin Gowa dianggap tetap selama periode penelitian.
2. Seluruh halte yang berada pada koridor penelitian diasumsikan beroperasi normal dan dapat melayani proses naik serta turun penumpang.
3. Kapasitas setiap armada bus dianggap konstan selama periode operasional.
4. Seluruh armada yang tersedia diasumsikan berada dalam kondisi layak operasi dan tidak mengalami kerusakan selama periode simulasi.

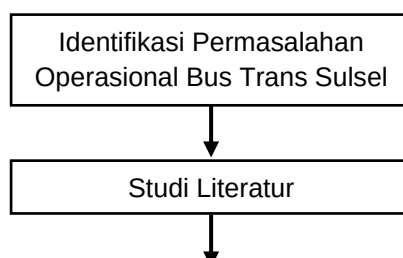
5. Kondisi lalu lintas dianggap normal sehingga waktu tempuh antarhalte mengikuti nilai rata-rata yang digunakan dalam penelitian.
6. Faktor-faktor eksternal seperti kecelakaan lalu lintas, bencana alam, cuaca ekstrem, demonstrasi, dan gangguan operasional lainnya tidak diperhitungkan dalam model.
7. Permintaan penumpang bersifat dinamis dan berbeda antarperiode operasional (Tabel 2).
8. Data permintaan penumpang dibangun melalui simulasi berdasarkan karakteristik operasional Bus Trans Sulsel dan hasil observasi lapangan.
9. Kedatangan penumpang di halte diasumsikan bersifat acak sehingga rata-rata waktu tunggu penumpang dapat didekati menggunakan setengah dari nilai headway (Ceder, 2015).
10. Biaya operasional setiap armada diasumsikan proporsional terhadap jarak tempuh perjalanan (pergi-pulang) dan seragam untuk setiap armada.
11. Tidak terjadi perubahan tarif selama periode penelitian.
12. Setiap armada menyelesaikan satu perjalanan sesuai jadwal yang telah ditentukan sebelum memulai perjalanan berikutnya.
13. Jumlah armada yang tersedia selama periode penelitian bersifat tetap (15 Unit), dengan setiap bus menjalankan paling banyak 4 trip PP per hari, sehingga total trip PP yang tersedia dalam sehari dibatasi sebanyak $15 \times 4 = 60$ trip.
14. Evaluasi kinerja algoritma dilakukan berdasarkan nilai fungsi objektif, tingkat utilisasi armada, load factor, dan waktu komputasi.
15. Permintaan penumpang pada setiap periode dibangkitkan mengikuti distribusi seragam (uniform) dalam rentang load factor sesuai kategori periode pada Tabel 2.

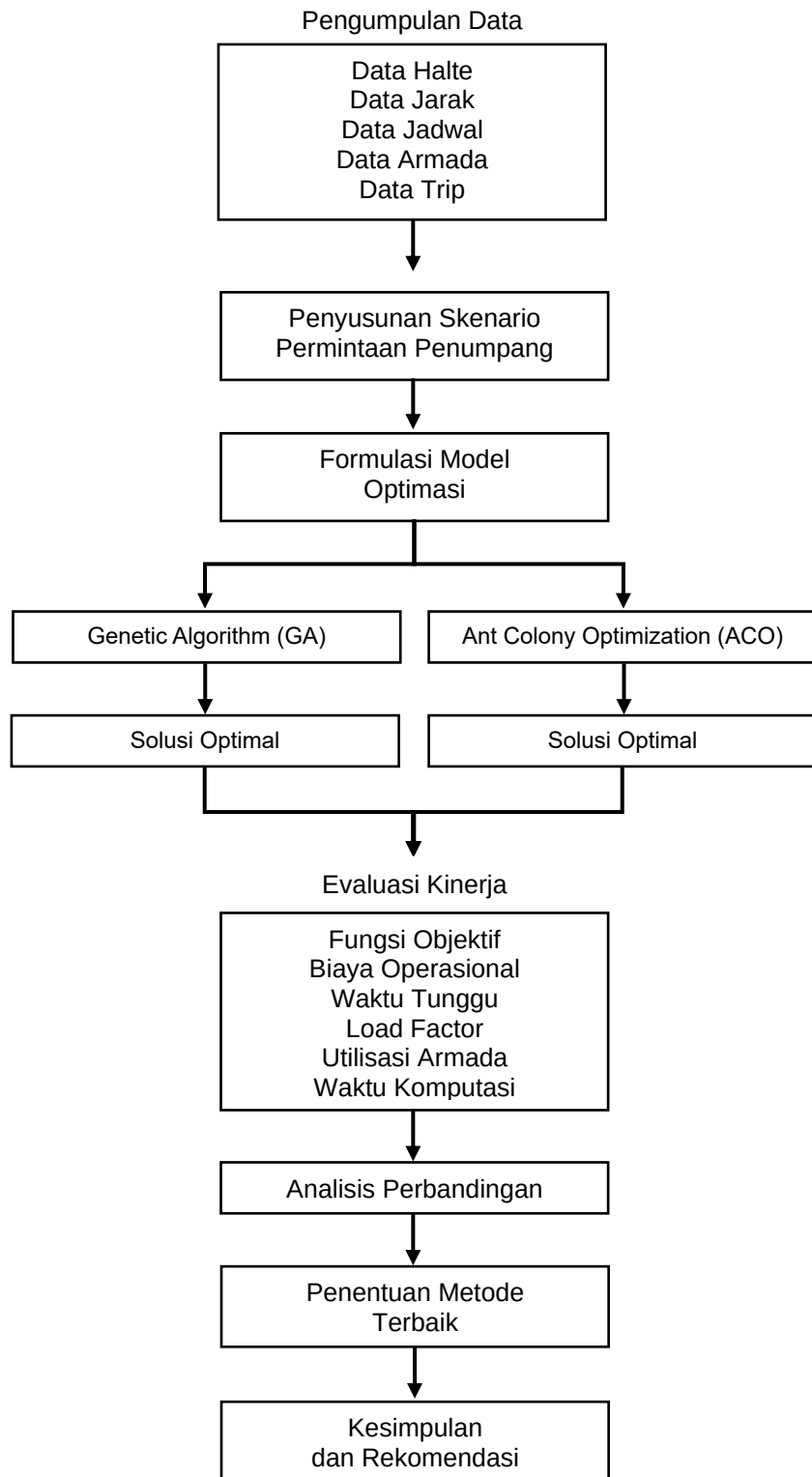
2.5 Kerangka Penelitian

Penelitian ini bertujuan membandingkan kinerja Genetic Algorithm (GA) dan Ant Colony Optimization (ACO) dalam menyelesaikan permasalahan optimasi operasional Bus Trans Sulsel pada Koridor 5.

Tahapan penelitian diawali dengan pengumpulan dan pengolahan data operasional (halte, jarak antarhalte, jadwal keberangkatan, jumlah armada, jumlah trip), dilanjutkan dengan penyusunan skenario permintaan penumpang. Data tersebut digunakan untuk membangun model optimasi yang memuat variabel keputusan, fungsi objektif, serta kendala operasional. Model yang telah dibangun kemudian diselesaikan menggunakan GA dan ACO secara terpisah pada model yang sama, sehingga hasil keduanya dapat dibandingkan secara objektif berdasarkan nilai fungsi objektif, biaya operasional, waktu tunggu, load factor, utilisasi armada, dan waktu komputasi. Tahap akhir adalah analisis perbandingan untuk menentukan metode dengan kinerja terbaik.

Kerangka penelitian yang digunakan dalam penelitian ini ditunjukkan pada Gambar 3.





Gambar 3 Kerangka Penelitian

2.6 Formulasi Model Optimasi

2.6.1 Himpunan (Sets)

Dalam penelitian ini digunakan himpunan sebagai berikut:

$T = \{1, 2, \dots, m\}$ himpunan periode operasional.

$B = \{1, 2, \dots, n\}$ himpunan armada bus yang tersedia.

2.6.2 Parameter

Parameter yang digunakan dalam model adalah:

Table 7 Parameter yang digunakan dalam model

Parameter	Keterangan
P_t	Permintaan penumpang pada periode t
C	Kapasitas bus (penumpang)
R	Panjang rute pergi-pulang (km)
Co	Biaya operasional per km
N	Jumlah armada tersedia
h_{min}, h_{max}	Headway minimum dan maksimum
OC_0, WT_0, ES_0	Nilai referensi (baseline kondisi eksisting) untuk normalisasi

2.6.3 Variabel Keputusan

Variabel keputusan yang digunakan adalah:

x_t = jumlah bus aktif pada periode (t).

q_t = jumlah trip PP yang diberangkatkan pada periode (t).

h_t = headway pada periode (t).

Ketiga variabel ini dioptimalkan secara bersama-sama, dengan hubungan $q_t \leq k_t x_t$ sebagaimana dijelaskan pada Subbab 2.3.5.

2.6.4 Fungsi Objektif

Penelitian ini bertujuan memperoleh konfigurasi operasional yang menekan biaya operasional, mengurangi waktu tunggu penumpang, dan mengurangi kapasitas yang tidak termanfaatkan, sambil memberikan penalti besar terhadap solusi yang menyebabkan shortage (permintaan tidak terlayani). Setiap komponen biaya, waktu tunggu, dan kursi kosong dinormalisasi terhadap nilai referensi kondisi eksisting (OC_0 , WT_0 , ES_0) agar berada pada skala yang sebanding sebelum dijumlahkan, sehingga fungsi objektif gabungan dirumuskan sebagai:

$$\min_z = w_1 \frac{OC}{OC_0} + w_2 \frac{WT}{WT_0} + w_3 \frac{ES}{ES_0} \quad (9)$$

dengan:

- OC, WT, ES = biaya operasional, waktu tunggu, dan kursi kosong dari solusi yang dievaluasi,
- OC_0, WT_0, ES_0 = nilai referensi (baseline) kondisi eksisting untuk masing-masing komponen,
- w_1, w_2, w_3 = bobot kepentingan tiap komponen.

$w_1 = w_2 = w_3 = 1$ (bobot kepentingan yang sama untuk ketiga komponen pada penelitian ini; dapat disesuaikan untuk penelitian lanjutan), dan P adalah penalti shortage:

$$P = \lambda \cdot SH \quad (10)$$

dengan λ = koefisien penalti (bernilai besar, ditetapkan PENALTY PER SHORTAGE PASSENGER = 1000) dan SH = total shortage sesuai Subbab 2.3.4.

2.6.5 Komponen Fungsi Objektif

a. Biaya Operasional

Biaya operasional dihitung berdasarkan jumlah trip PP yang benar-benar dijalankan pada setiap periode (q_t), bukan berdasarkan jumlah bus aktif (x_t), karena satu bus aktif dapat menjalankan lebih dari satu trip pada periode yang sama (Subbab 2.3.5). Biaya operasional dihitung sebagai:

$$OC = \sum_{t \in T} C_o R q_t = c_{trip} \sum_{t \in T} q_t \quad (11)$$

dengan:

C_o = biaya operasional per km,

R = panjang rute pergi-pulang,

$c_{trip} = C_o R$ = biaya satu trip PP,

q_t = jumlah trip PP pada periode t .

Semakin banyak trip PP yang dijalankan maka semakin besar biaya operasional.

b. Waktu Tunggu Penumpang

Mengikuti pendekatan Ceder (2015), waktu tunggu rata-rata penumpang dapat diperkirakan sebagai:

$$WT = \sum_{t \in T} \frac{h_t}{2} \quad (12)$$

dengan h_t = headway pada periode t (menit), dijumlahkan untuk seluruh periode $t \in T$.

Asumsi ini berlaku apabila kedatangan penumpang bersifat acak.

c. Penalti Kursi Kosong (kapasitas tidak termanfaatkan)

Untuk mengurangi penyediaan trip PP yang berlebihan dibandingkan permintaan digunakan penalti, dihitung dari kapasitas total trip pada periode tersebut (Cq_t):

$$ES = \sum_{t \in T} \max(0, Cq_t - P_t) \quad (13)$$

dengan:

C = kapasitas satu bus (40 penumpang),

q_t = jumlah trip PP pada periode t (sehingga Cq_t adalah total kapasitas layanan periode tersebut),

P_t = permintaan penumpang pada periode t .

Besaran ini menggambarkan kapasitas yang tidak termanfaatkan.

d. Shotage (permintaan tidak terlayani)

$$SH = \sum_{t \in T} \max(0, P_t - Cq_t) \quad (14)$$

dengan P_t = permintaan penumpang pada periode t , dan Cq_t = total kapasitas layanan pada periode tersebut, sesuai definisi pada Subbab 2.3.4.

2.6.6 Kendala Model

a. Kendala jumlah armada

Jumlah bus aktif pada setiap periode tidak boleh melebihi armada yang tersedia.

$$1 \leq x_t \leq N, \text{ untuk setiap } t \in T \quad (15)$$

b. Kendala jumlah trip PP

Jumlah trip PP pada periode t dibatasi oleh jumlah bus aktif dan kapasitas trip maksimum per bus pada periode itu (k_t), dan total trip PP dalam sehari dibatasi oleh jumlah armada dikalikan jumlah trip maksimum per bus (4 trip/bus/hari):

$$q_t \leq k_t x_t, \quad \sum_{t \in T} q_t \leq N \times 4 = 60 \quad (16)$$

dengan k_t = kapasitas trip maksimum satu bus pada periode t (Subbab 2.3.5), x_t = jumlah bus aktif, dan N = jumlah armada tersedia (15 unit).

c. Kendala kapasitas pelayanan

Kapasitas armada harus mampu melayani permintaan penumpang.

$$x_t C \geq P_t, \text{ (idealnya terpenuhi; pelanggaran dikenai penalti } \lambda \cdot SH)$$

d. Kendala headway

$$h_{\min} \leq h_t \leq h_{\max} \quad (17)$$

dengan $h_{\min}$ dan $h_{\max}$ = headway minimum dan maksimum yang diizinkan, dan h_t = headway pada periode t .

e. Kendala bilangan bulat dan non-negatif

$$x_t \in Z^+, h_t > 0 \quad (18)$$

2.6.7 Model Optimasi

Berdasarkan fungsi objektif dan kendala yang telah dirumuskan, model optimasi operasional Bus Trans Sulsel dapat dinyatakan sebagai:

$$\min Z = w_1 \sum_{t \in T} C_o R q_t + w_2 \sum_{t \in T} \frac{h_t}{2} + w_3 \sum_{t \in T} \max(0, C q_t - P_t) \quad (19)$$

dengan Z = nilai fungsi objektif gabungan yang diminimalkan (dalam implementasi, ketiga suku ternormalisasi terhadap OC_0, WT_0, ES_0 sesuai Subbab 2.6.4); C_o, R, q_t, h_t, C, P_t mengikuti definisi pada Subbab 2.3.4 dan 2.6.2; dan w_1, w_2, w_3 = bobot kepentingan tiap komponen. Dan kendala: $1 \leq x_t \leq N, q_t \leq k_t x_t, \sum_{t \in T} q_t \leq 60, h_{\min} \leq h_t \leq h_{\max}, x_t, q_t \in Z^+, h_t > 0$ untuk setiap $t \in T$, kendala $C q_t \geq P_t$ diterapkan sebagai kendala lunak melalui penalti $\lambda \cdot SH$.

2.7 Implementasi Genetic Algorithm

Genetic Algorithm (GA) digunakan untuk memperoleh solusi optimasi operasional Bus Trans Sulsel berdasarkan model yang telah dirumuskan pada subbab 2.6. Pada penelitian ini, GA diterapkan untuk menentukan kombinasi jumlah armada aktif dan headway pada setiap periode operasional sehingga diperoleh nilai fungsi objektif minimum.

2.7.1 Representasi Kromosom

Dalam penelitian ini, satu kromosom merepresentasikan satu alternatif solusi operasional Bus Trans Sulsel. Setiap kromosom terdiri atas tiga bagian utama, yaitu jumlah bus aktif (x_t), jumlah trip PP (q_t), dan headway (h_t) pada setiap periode operasional. Gen q_t dipisahkan secara eksplisit dari x_t agar biaya operasional dan kendala kapasitas layanan dihitung dari jumlah trip yang sebenarnya dijalankan, bukan dari jumlah bus aktif (Subbab 2.3.5).

Misalkan terdapat (m) periode operasional dalam satu hari. Maka kromosom dapat direpresentasikan sebagai:

$$K = (x_1, x_2, \dots, x_m, q_1, q_2, \dots, q_m, h_1, h_2, \dots, h_m) \quad (20)$$

dengan m = jumlah periode operasional, $x_{1..m}$ = gen jumlah bus aktif, $q_{1..m}$ = gen jumlah trip PP, dan $h_{1..m}$ = gen headway untuk setiap periode.

Sebagai contoh, untuk 6 periode operasional (P1-P6 sesuai Tabel 2), satu kromosom dapat ditulis sebagai $K = (14, 12, 8, 15, 10, 10, 14, 12, 8, 15, 10, 10, 10, 10, 10, 10, 10, 10)$, yang berarti jumlah bus aktif (14, 12, 8, 15, 10, 10), jumlah trip PP (14, 12, 8, 15, 10, 10), dan headway seragam 10 menit pada seluruh periode. Total jumlah

trip PP pada seluruh kromosom dibatasi tidak melebihi 60 trip/hari ($\sum q_t \leq 60$) dan setiap gen q_t dibatasi tidak melebihi gen x_t pada periode yang sama ($q_t \leq x_t$).

2.7.2 Inisialisasi Populasi

Populasi awal dibangkitkan secara acak: untuk setiap periode t , nilai x_t dibangkitkan pada rentang 1 sampai N , nilai q_t dibangkitkan pada rentang 1 sampai x_t , sedangkan h_t dibangkitkan dari himpunan diskrit headway yang diizinkan. Setelah pembangkitan, dilakukan proses perbaikan (*repair*) untuk memastikan total trip pada seluruh periode tidak melebihi batas harian ($\sum_t q_t \leq 60$); apabila total trip melebihi batas, kelebihan trip dikurangi secara proporsional dari periode-periode dengan alokasi trip terbesar.

2.7.3 Fungsi Fitness

Fungsi fitness digunakan untuk mengevaluasi kualitas setiap kromosom. Karena model optimasi bertujuan meminimalkan fungsi objektif Z , maka nilai fitness dapat didefinisikan sebagai:

$$Fitness = \frac{1}{1+Z} \quad (21)$$

dengan Z = nilai fungsi objektif kromosom yang dievaluasi (Subbab 2.6.7).

Semakin kecil nilai Z , semakin besar nilai fitness, sehingga kromosom dengan nilai fungsi objektif lebih kecil memiliki peluang lebih besar terpilih pada proses seleksi. Penalti shortage (Subbab 2.6.5) sudah tercakup di dalam Z , sehingga tidak diperlukan penalti tambahan terpisah untuk kromosom yang melanggar kendala kapasitas pelayanan.

2.7.4 Seleksi

Seleksi induk (parent) dilakukan menggunakan metode tournament selection: beberapa kromosom dipilih secara acak dari populasi, kemudian kromosom dengan nilai fitness terbaik di antaranya dipilih sebagai induk.

2.7.5 Crossover

Crossover dilakukan menggunakan metode single-point crossover, yaitu menukar sebagian gen dari dua kromosom induk untuk menghasilkan kromosom anak (offspring), dengan peluang terjadinya crossover mengikuti parameter crossover rate.

2.7.6 Mutasi

Mutasi dilakukan dengan mengubah nilai gen tertentu secara acak berdasarkan peluang mutation rate: gen jumlah bus aktif (x_t) diganti bilangan bulat acak pada rentang 1 sampai N , gen jumlah trip (q_t) diganti bilangan bulat acak pada rentang 1 sampai x_t , dan gen headway diganti nilai acak dari himpunan headway yang diizinkan. Setelah mutasi, fungsi

perbaikan (*repair*) pada Subbab 2.7.2 kembali diterapkan agar kendala $\sum_t q_t \leq 60$ dan $q_t \leq x_t$ tetap terpenuhi.

2.7.7 Elitisme

Sejumlah kromosom dengan nilai fitness terbaik dipertahankan langsung ke generasi berikutnya (elitisme), agar kualitas solusi terbaik tidak menurun selama proses evolusi.

2.7.8 Kriteria Terminasi

Proses iterasi Genetic Algorithm dihentikan apabila salah satu kondisi berikut terpenuhi:

1. Jumlah generasi maksimum telah tercapai.
2. Nilai fungsi objektif tidak mengalami perubahan signifikan selama beberapa generasi berturut-turut.
3. Solusi terbaik telah memenuhi kriteria yang ditetapkan.

2.7.9 Parameter Genetic Algorithm

Parameter yang digunakan dalam implementasi Genetic Algorithm meliputi:

Table 8 Parameter yang digunakan dalam implementasi Genetic Algorithm

Parameter	Nilai yang Digunakan
Ukuran populasi	100 individu
Jumlah generasi maksimum	300
Probabilitas crossover	0,80
Probabilitas mutasi	0,10
Ukuran tournament	3
Jumlah elit	2 individu terbaik

Nilai parameter ditentukan melalui percobaan awal (*trial and error*) untuk memperoleh performa algoritma yang stabil.

2.7.10 Alur Implementasi Genetic Algorithm

Tahapan implementasi Genetic Algorithm dalam penelitian ini adalah sebagai berikut:

1. Menentukan parameter algoritma.
2. Membangkitkan populasi awal secara acak.
3. Menghitung nilai fungsi objektif dan fitness setiap kromosom.
4. Melakukan seleksi induk.
5. Melakukan crossover.
6. Melakukan mutasi.
7. Menerapkan elitisme.
8. Mengulangi proses hingga kriteria terminasi terpenuhi.
9. Menentukan kromosom terbaik sebagai solusi optimal.

Solusi terbaik yang diperoleh dari Genetic Algorithm selanjutnya dibandingkan dengan solusi yang diperoleh menggunakan Ant Colony Optimization berdasarkan nilai fungsi objektif, biaya operasional, waktu tunggu penumpang, load factor, utilisasi armada, dan waktu komputasi.

2.8 Implementasi Ant Colony Optimization

Ant Colony Optimization (ACO) digunakan sebagai metode pembandingan terhadap Genetic Algorithm dalam menyelesaikan permasalahan optimasi operasional Bus Trans Susel. Pada penelitian ini, ACO diterapkan untuk menentukan kombinasi jumlah armada aktif dan headway pada setiap periode operasional sehingga diperoleh nilai fungsi objektif minimum.

2.8.1 Representasi Solusi

Dalam penelitian ini, satu solusi ACO merepresentasikan konfigurasi operasional Bus Trans Susel yang terdiri atas jumlah armada aktif dan headway pada setiap periode operasional. Jika terdapat (m) periode operasional, maka solusi dapat dituliskan sebagai:

$$S = (x_1, x_2, \dots, x_m, h_1, h_2, \dots, h_m) \quad (22)$$

dengan m = jumlah periode operasional, $x_{1..m}$ = jumlah bus aktif, $q_{1..m}$ = jumlah trip PP, dan $h_{1..m}$ = headway untuk setiap periode.

Representasi ini dibuat sama dengan representasi kromosom pada Genetic Algorithm (Subbab 2.7.1) agar hasil kedua metode dapat dibandingkan secara adil.

2.8.2 Pembentukan Ruang Solusi

Ruang solusi dibentuk dari seluruh kombinasi (jumlah bus aktif, jumlah trip, headway) yang mungkin pada setiap periode: x_t pada rentang 1 sampai N, q_t pada rentang 1 sampai x_t , dan h_t dari himpunan headway diskrit yang diizinkan. Setiap semut membangun satu solusi lengkap dengan memilih satu kombinasi pada setiap periode, kemudian solusi yang terbentuk diperbaiki (*repair*) agar total trip pada seluruh periode tidak melebihi batas harian ($\sum_t q_t \leq 60$), sebagaimana dijelaskan pada Subbab 2.7.2.

2.8.3 Inisialisasi Feromon

Nilai feromon awal diberikan pada setiap kemungkinan pilihan jumlah armada dan headway. Feromon menunjukkan tingkat ketertarikan semut terhadap suatu pilihan solusi. Nilai feromon awal dinyatakan sebagai:

$$\tau_{ij}(0) = \tau_0 \quad (23)$$

dengan:

$\tau_{ij}(0)$ = nilai feromon awal,

τ_0 = konstanta feromon awal.

Nilai feromon awal dibuat sama untuk seluruh pilihan agar setiap alternatif memiliki peluang yang sama untuk dipilih pada iterasi awal.

2.8.4 Informasi Heuristik

Informasi heuristik dibangun dari estimasi efisiensi lokal setiap kombinasi (biaya, waktu tunggu, dan kursi kosong lokal), dengan bentuk umum:

$$\eta_{ij} = \frac{1}{1+Z_{ij}} \quad (24)$$

dengan:

η_{ij} = informasi heuristik pada pilihan solusi,

Z_{ij} = estimasi nilai fungsi objektif lokal.

Semakin kecil nilai Z_{ij} , semakin besar nilai heuristiknya, sehingga pilihan tersebut memiliki peluang lebih besar untuk dipilih.

2.8.5 Probabilitas Pemilihan Solusi

Setiap semut memilih alternatif solusi berdasarkan kombinasi antara nilai feromon dan informasi heuristik. Probabilitas pemilihan alternatif (j) oleh semut pada kondisi (i) dinyatakan sebagai:

$$P_{ij} = \frac{(\tau_{ij})^\alpha (\eta_{ij})^\beta}{\sum_{\{k \in A_i\}} (\tau_{ik})^\alpha (\eta_{ik})^\beta} \quad (25)$$

dengan:

P_{ij} = probabilitas pemilihan alternatif j,

(τ_{ij}) = nilai feromon,

(η_{ij}) = informasi heuristik,

(α) = parameter pengaruh feromon,

(β) = parameter pengaruh informasi heuristik,

(A_i) = himpunan alternatif yang dapat dipilih.

Parameter (α) dan (β) menentukan keseimbangan antara pengalaman kolektif semut dan informasi lokal dari solusi yang sedang dibangun.

2.8.6 Evaluasi Solusi

Setelah setiap semut membentuk satu solusi lengkap, solusi tersebut dievaluasi menggunakan fungsi objektif yang sama dengan model pada Subbab 2.6:

$$Z = w_1 \frac{OC}{OC_0} + w_2 \frac{WT}{WT_0} + w_3 \frac{ES}{ES_0} + \lambda \cdot SH \quad (26)$$

dengan OC, WT, ES = biaya operasional, waktu tunggu, dan kursi kosong dari solusi; OC_0, WT_0, ES_0 = nilai referensi baseline; w_1, w_2, w_3 = bobot tiap komponen; dan $\lambda.SH$ = penalti shortage, seluruhnya mengikuti definisi pada Subbab 2.6.4-2.6.5.

2.8.7 Pembaruan Feromon

Setelah seluruh semut menyelesaikan pembentukan solusi, dilakukan pembaruan feromon. Pembaruan feromon terdiri atas dua tahap, yaitu evaporasi feromon dan penambahan feromon.

1. Evaporasi Feromon

Evaporasi dilakukan untuk mengurangi pengaruh solusi lama dan mencegah algoritma terjebak pada solusi lokal.

$$\tau_{ij} \leftarrow (1 - \rho)\tau_{ij} \quad (27)$$

dengan: ρ = laju evaporasi feromon, ($0 < \rho < 1$)

2. Penambahan Feromon

Solusi yang lebih baik diberi tambahan feromon lebih besar. Penambahan feromon dapat dirumuskan sebagai:

$$\tau_{ij} \leftarrow \tau_{ij} + \Delta\tau_{ij} \quad (28)$$

dengan tambahan feromon dihitung berdasarkan kualitas solusi terbaik:

$$\Delta\tau_{ij} = \frac{Q}{1+Z_{best}} \quad (29)$$

dengan:

Q = konstanta deposit feromon,

Z_{best} = nilai fungsi objektif dari solusi terbaik pada iterasi tersebut. Semakin kecil Z_{best} , semakin besar tambahan feromon yang diberikan pada jalur solusi terbaik.

2.8.8 Kriteria Terminasi

Proses iterasi ACO dihentikan apabila salah satu kondisi berikut terpenuhi:

1. Jumlah iterasi maksimum telah tercapai.
2. Nilai fungsi objektif terbaik tidak mengalami perubahan signifikan selama beberapa iterasi.
3. Solusi terbaik telah memenuhi kriteria yang ditetapkan.

2.8.9 Parameter Ant Colony Optimization

Parameter yang digunakan dalam implementasi ACO meliputi:

Table 9 Parameter yang digunakan dalam implementasi ACO

Parameter	Nilai yang Digunakan
Jumlah semut	80
Jumlah iterasi maksimum	300
Alpha - pengaruh feromon	1,0
Beta - pengaruh heuristik	2,0
Rho - laju evaporasi	0,50
Q - konstanta deposit feromon	1,0
Tau0 - feromon awal	1,0

2.8.10 Alur Implementasi Ant Colony Optimization

Tahapan implementasi ACO dalam penelitian ini adalah sebagai berikut:

1. Menentukan parameter ACO.
2. Menghitung matriks heuristik.
3. Menginisialisasi nilai feromon awal.
4. Setiap semut membangun solusi berupa kombinasi (x_t) dan (h_t).
5. Mengevaluasi setiap solusi menggunakan fungsi objektif.
6. Menentukan solusi terbaik pada setiap iterasi.
7. Melakukan evaporasi feromon.
8. Menambahkan feromon pada komponen solusi terbaik.
9. Mengulangi proses hingga kriteria terminasi terpenuhi.
10. Menentukan solusi terbaik akhir.

Solusi terbaik yang diperoleh dari Ant Colony Optimization selanjutnya dibandingkan dengan solusi terbaik dari Genetic Algorithm berdasarkan nilai fungsi objektif, biaya operasional, waktu tunggu penumpang, load factor, utilisasi armada, dan waktu komputasi.

2.9 Kriteria Evaluasi Kinerja

Kriteria utama yang digunakan adalah nilai fungsi objektif Z sebagaimana dirumuskan pada Subbab 2.6.4. Metode yang menghasilkan nilai Z lebih kecil dianggap memberikan solusi yang lebih baik.

2.9.1 Nilai Fungsi Objektif

Kriteria utama yang digunakan adalah nilai fungsi objektif Z sebagaimana dirumuskan pada Subbab 2.6.4. Metode yang menghasilkan nilai Z lebih kecil dianggap memberikan solusi yang lebih baik.

2.9.2 Biaya Operasional

Biaya operasional (OC, Subbab 2.6.5) digunakan untuk mengukur efisiensi penggunaan armada. Metode yang menghasilkan biaya operasional lebih rendah dianggap lebih efisien.

2.9.3 Waktu Tunggu Penumpang

Waktu tunggu penumpang (WT, Subbab 2.6.5) digunakan sebagai indikator kualitas pelayanan. Semakin kecil waktu tunggu, semakin baik kualitas pelayanan yang diberikan.

2.9.4 Shortage

Shortage (SH, Subbab 2.3.4) digunakan untuk mengukur sejauh mana suatu solusi gagal memenuhi seluruh permintaan penumpang. Solusi dengan shortage lebih kecil (idealnya nol) dianggap lebih baik dari sisi kualitas layanan, meskipun mungkin memerlukan biaya operasional yang lebih tinggi.

2.9.5 Tingkat Utilisasi Armada

Utilisasi armada (AU, Subbab 2.3.4) digunakan untuk mengukur tingkat pemanfaatan armada yang tersedia. Nilai utilisasi yang tinggi menunjukkan armada digunakan secara lebih optimal.

2.9.6 Load Factor

Load factor (LF, Subbab 2.3.4) digunakan untuk mengevaluasi kesesuaian antara kapasitas armada yang dioperasikan dengan permintaan penumpang.

2.9.7 Waktu Komputasi

Waktu komputasi (CT) menunjukkan lama waktu yang dibutuhkan algoritma untuk memperoleh solusi terbaik, dihitung sebagai selisih waktu selesai dan waktu mulai proses. Metode yang menghasilkan solusi berkualitas tinggi dengan waktu komputasi lebih singkat dianggap lebih efisien secara komputasional.

2.9.8 Analisis Perbandingan

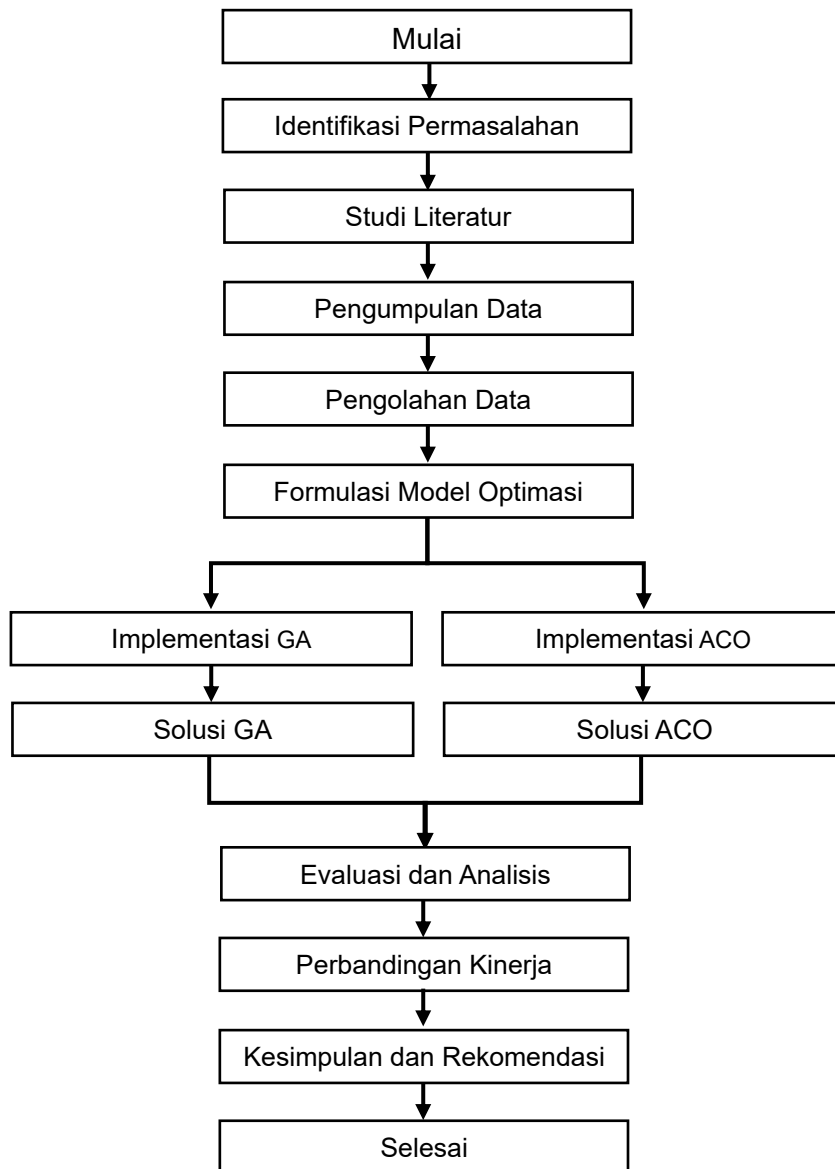
Perbandingan kinerja GA dan ACO dilakukan berdasarkan seluruh indikator evaluasi di atas, diringkas dalam format tabel perbandingan sebagaimana disajikan pada BAB III (Tabel 24). Metode yang menghasilkan kombinasi terbaik antara kualitas solusi dan efisiensi komputasi direkomendasikan sebagai metode yang lebih efektif untuk permasalahan optimasi operasional Bus Trans Sulsel pada Koridor 5.

2.10 Tahapan Penelitian

Penelitian ini dilaksanakan melalui beberapa tahapan yang dilakukan secara sistematis mulai dari identifikasi permasalahan hingga penarikan kesimpulan. Tahapan penelitian yang dilakukan adalah sebagai berikut.

1. Identifikasi Permasalahan: mengidentifikasi permasalahan operasional Bus Trans Sulsel pada Koridor 5 melalui studi literatur dan observasi terhadap karakteristik layanan yang tersedia.

2. Studi Literatur: mengkaji referensi mengenai sistem transportasi publik, optimasi operasional bus, Genetic Algorithm, Ant Colony Optimization, dan penelitian terdahulu yang relevan.
3. Pengumpulan Data: mengumpulkan data halte, jarak antarhalte, jadwal keberangkatan, jumlah armada, jumlah trip, dan karakteristik permintaan penumpang.
4. Pengolahan dan Persiapan Data: verifikasi data, penyusunan basis data penelitian, penentuan parameter model, dan penyusunan skenario permintaan penumpang (Tabel 2).
5. Formulasi Model Optimasi: membangun model optimasi (variabel keputusan, parameter, fungsi objektif, dan kendala operasional) sebagaimana dirumuskan pada Subbab 2.6.
6. Implementasi Genetic Algorithm: menyelesaikan model menggunakan GA (Subbab 2.7), menghasilkan konfigurasi operasional terbaik menurut GA.
7. Implementasi Ant Colony Optimization: menyelesaikan model yang sama menggunakan ACO (Subbab 2.8), menghasilkan konfigurasi operasional terbaik menurut ACO.
8. Evaluasi dan Analisis Hasil: mengevaluasi solusi dari kedua metode berdasarkan kriteria pada Subbab 2.9, dan membandingkan kinerja kedua metode.
9. Penarikan Kesimpulan dan Penyusunan Rekomendasi: menginterpretasikan hasil penelitian untuk menentukan metode dengan kinerja terbaik, serta menyusun kesimpulan dan rekomendasi bagi pengelolaan operasional Bus Trans Sulsel.



Gambar 4 Flowchart Tahapan Penelitian

BAB III

HASIL PENELITIAN

3.1 Pengumpulan Data

Data yang digunakan pada penelitian ini terdiri atas data operasional yang diperoleh dari hasil observasi lapangan, dengan objek penelitian yang berlokasi di koridor 5 Bus Trans Sulsel. Data yang digunakan meliputi data Halte, jarak antar halte, jadwal kerangkatan, jumlah armada, kapasitas armada, jumlah perjalanan (trip), serta scenario permintaan penumpang.

3.1.1 Data Halte

Koridor 5 Bus Trans Sulsel melayani perjalanan dari Kampus Universitas Hasanuddin Tamalanrea menuju Kampus Universitas Hasanuddin Gowa dengan beberapa halte pemberhentian. Penelitian ini terdapat 45 halte pada rute perjalanan Kampus Universitas Hasanuddin Tamalanrea-Kampus Universitas Hasanuddin Gowa dan 37 halte pada rute perjalanan Kampus Universitas Hasanuddin Gowa- Kampus Universitas Hasanuddin Tamalanrea.

Table 10 Halte Pada Rute Perjalanan Kampus Universitas Hasanuddin Tamalanrea-Kampus Universitas Hasanuddin Gowa

NO. HALTE	NAMA HALTE
1	Kampus Unhas Tamalanrea
2	Halte A Gor Unhas
3	Halte Hotel Unhas
4	Halte Masjid Ikhtiar Unhas
5	Halte Litha Co
6	Halte Kantor Gubernur 2
7	Halte Telkom Panakkukang
8	Halte Alianz Pettarani
9	Halte Topaz
10	Halte Pizza Ria Café
11	Halte Mall Panakkukang
12	Halte BCA Panakkukang
13	Halte Politeknik STIA LAN Makassar
14	Halte Kimia Farma Hertasning
15	Halte Markobar Hertasning
16	Halte RS Grestelina
17	Halte Bawaslu Makassar
18	Halte Mega Rezky 2
19	Halte Aeroppala Food Court
20	Halte Perumahan Belmont

NO. HALTE	NAMA HALTE
21	Halte Komplek Blok AB Minasa Upa
22	Halte Mcd Hertasning
23	Halte Satu Sama Hertasning
24	Halte Kampus Patria Artha
25	Halte Royal Spring
26	Halte Grand Toserba Hertasning 2
27	Halte RS Muhammadiyah 2
28	Halte Mr DIY Hertasning
29	Halte UIN Samata Pintu 1
30	Halte UIN Samata Pintu 2
31	Halte Pesantren Guppi Samata
32	Halte Mutiara Indah Samata
33	Halte Masjid Baiturrahman Gowa
34	Halte Samata Residence
35	Halte Griya Kenari
36	Halte Politeknik Pertanian Pembangunan Gowa
37	Halte PDAM Borongloe
38	Halte H Teknik Unhas (Technomart)
39	Halte G Teknik Unhas (Asrama Fakultas Teknik)
40	Halte F Teknik Unhas (Masjid Al Ilmi Ikatek)
41	Halte E Teknik Unhas (Teknik Sipil)
42	Halte D Teknik Unhas (Teknik Elektrikal)
43	Halte C Teknik Unhas (Teknik Perkapalan)
44	Halte B Teknik Unhas (Teknik Geologi)
45	Kampus Teknik Unhas Gowa

Table 11 Halte Pada Rute Perjalanan Kampus Universitas Hasanuddin Gowa-Kampus Universitas Hasanuddin Tamalanrea

NO. HALTE	NAMA HALTE
1	Kampus Teknik Unhas Gowa
2	Halte CSA Unhas Teknik 1
3	Halte Titik 0 Unhas Teknik
4	Halte Asrama Rindam
5	Halte Kantor Lurah Mawang
6	Halte R3 Sentosa Land
7	Halte Griya Putra Sejahtera
8	Halte Masjid Al Insan
9	Halte Green Allian Residence
10	Halte Ammar Food Court
11	Halte RS Muhammadiyah 1

NO. HALTE	NAMA HALTE
12	Halte Grand Toserba Hertasning 1
13	Bumi Aroepala
14	Halte Daihatsu Hertasning
15	Halte Pao Pao Permai
16	Halte Citra Land Hertasning
17	BRI Aroepala
18	Balai Serbaguna Aroepala
19	Halte Mega Rezky 1
20	Halte PT Pp Persero
21	Halte SD Katolik Santo Aloysius
22	Halte Hotel Continent
23	Halte Mall Panakkukang
24	Halte BCA Panakkukang
25	Halte Taman Pakui Pettarani
26	Halte Kantor Gubernur 1
27	Halte Perum Bulog
28	Halte GPIB Mangamaseang
29	Halte Hotel Unhas 2
30	Halte G Fakultas Ilmu Budaya
31	Halte RS Pendidikan
32	Halte F Fakultas Hukum
33	Halte E Fakultas Sospol
34	Halte D Fakultas Kedokteran
35	Halte C Fakultas Pertanian/Peternakan
36	Halte B Fakultas Kehutanan
37	Halte Unhas Tamalanre

3.1.2 Data Jarak Antar Halte

Jarak antar halte digunakan untuk menghitung total Panjang rute dan biaya operasional kendaraan

Table 12 Jarak Antar Halte Rute Perjalanan Kampus Universitas Hasanuddin Tamalanrea-Kampus Universitas Hasanuddin Gowa

NO. HALTE	NAMA HALTE	JARAK
1	Kampus Unhas Tamalanrea	55
2	Halte A Gor Unhas	359
3	Halte Hotel Unhas	404
4	Halte Masjid Ikhtiar Unhas	2772
5	Halte Litha Co	1984
6	Halte Kantor Gubernur 2	1593

NO. HALTE	NAMA HALTE	JARAK
7	Halte Telkom Panakukang	452
8	Halte Alianz Pettarani	1307
9	Halte Topaz	395
10	Halte Pizza Ria Café	1426
11	Halte Mall Panakkukang	824
12	Halte BCA Panakkukang	378
13	Halte Politeknik STIA LAN Makassar	834
14	Halte Kimia Farma Hertasning	216
15	Halte Markobar Hertasning	534
16	Halte RS Grestelina	888
17	Halte Bawaslu Makassar	407
18	Halte Mega Rezky 2	320
19	Halte Aeroppala Food Court	397
20	Halte Perumahan Belmont	662
21	Halte Komplek Blok AB Minasa Upa	458
22	Halte Mcd Hertasning	582
23	Halte Satu Sama Hertasning	808
24	Halte Kampus Patria Artha	558
25	Halte Royal Spring	768
26	Halte Grand Toserba Hertasning 2	411
27	Halte RS Muhammadiyah 2	394
28	Halte Mr DIY Hertasning	651
29	Halte UIN Samata Pintu 1	377
30	Halte UIN Samata Pintu 2	673
31	Halte Pesantren Guppi Samata	476
32	Halte Mutiara Indah Samata	400
33	Halte Masjid Baiturrahman Gowa	794
34	Halte Samata Residence	588
35	Halte Griya Kenari	355
36	Halte Politeknik Pertanian Pembangunan Gowa	1005
37	Halte PDAM Borongloe	306
38	Halte H Teknik Unhas (Technomart)	351
39	Halte G Teknik Unhas (Asrama Fakultas Teknik)	188
40	Halte F Teknik Unhas (Masjid Al Ilmi Ikatek)	185
41	Halte E Teknik Unhas (Teknik Sipil)	148
42	Halte D Teknik Unhas (Teknik Elektrikal)	381
43	Halte C Teknik Unhas (Teknik Perkapalan)	98
44	Halte B Teknik Unhas (Teknik Geologi)	666
45	Kampus Teknik Unhas Gowa	

Table 13 Jarak Antar Halte Kampus Universitas Hasanuddin Gowa- Kampus Universitas Hasanuddin Tamalanrea

NO. HALTE	NAMA HALTE	JARAK
1	Kampus Teknik Unhas Gowa	
2	Halte CSA Unhas Teknik 1	439
3	Halte Titik 0 Unhas Teknik	329
4	Halte Asrama Rindam	514
5	Halte Kantor Lurah Mawang	635
6	Halte R3 Sentosa Land	747
7	Halte Griya Putra Sejahtera	902
8	Halte Masjid Al Insan	516
9	Halte Green Allian Residence	573
10	Halte Ammar Food Court	934
11	Halte RS Muhammadiyah 1	376
12	Halte Grand Toserba Hertasning 1	433
13	Bumi Aroepala	662
14	Halte Daihatsu Hertasning	712
15	Halte Pao Pao Permai	759
16	Halte Citra Land Hertasning	830
17	BRI Aroepala	366
18	Balai Serbaguna Aroepala	558
19	Halte Mega Rezky 1	700
20	Halte PT Pp Persero	563
21	Halte SD Katolik Santo Aloysius	498
22	Halte Hotel Continent	1117
23	Halte Mall Panakkukang	715
24	Halte BCA Panakkukang	825
25	Halte Taman Pakui Pettarani	660
26	Halte Kantor Gubernur 1	3.3
27	Halte Perum Bulog	654
28	Halte GPIB Mangamaseang	1092
29	Halte Hotel Unhas 2	3033
30	Halte G Fakultas Ilmu Budaya	467
31	Halte RS Pendidikan	288
32	Halte F Fakultas Hukum	255
33	Halte E Fakultas Sospol	160
34	Halte D Fakultas Kedokteran	590
35	Halte C Fakultas Pertanian/Peternakan	495
36	Halte B Fakultas Kehutanan	272
37	Halte Unhas Tamalanre	400

3.1.3. Data Armada Bus

Bus Trans Sulsel pada koridor 5 beroperasi mulai pukul 05.00 WITA sampai pukul 22.00 WITA, menggunakan armada dengan spesifikasi yang relatif seragam.

Table 14 Karakteristik Armada

Parameter	Nilai
Jumlah Armada Bus	15 Unit
Kapasitas Tiap Bus	40 Penumpang
Trip Pulang-Pergi per Bus	4 kali/hari
Panjang Rute Pergi	27,83 km
Panjang Rute Pulang	26,37 km

Jumlah armada tersebut merupakan batas maksimum yang dapat digunakan dalam proses optimasi.

3.1.4 Jadwal Keberangkatan

Jadwal Keberangkatan bus pada koridor 5 memiliki headway yang berbeda tiap trip.

Table 15 Jadwal Keberangkatan

KODE BUS	JAM			
	TRIP 1	TRIP 2	TRIP 3	TRIP 4
1	5:00 AM	8:15 AM	12:00 AM	3:45 PM
2	5:10 AM	8:30 AM	12:15 AM	4:00 PM
3	5:20 AM	8:45 AM	12:30 AM	4:20 PM
4	5:30 AM	9:00 AM	12:45 AM	4:40 PM
5	5:40 AM	9:15 AM	1:00 AM	5:00 PM
6	5:50 AM	9:30 AM	1:15 AM	5:20 PM
7	6:00 AM	9:45 AM	1:30 AM	5:40 PM
8	6:15 AM	10:00 AM	1:45 AM	6:00 PM
9	6:30 AM	10:15 AM	2:00 AM	6:20 PM
10	6:45 AM	10:30 AM	2:15 AM	6:40 PM
11	7:00 AM	10:45 AM	2:30 AM	7:00 PM
12	7:15 AM	11:00 AM	2:45 AM	7:20 PM
13	7:30 AM	11:15 AM	3:00 AM	7:40 PM
14	7:45 AM	11:30 AM	3:15 AM	8:00 PM
15	8:00 AM	11:45 AM	3:30 AM	9:00 PM

3.1.5 Data Biaya Operasional

Dalam penelitian ini biaya operasional dihitung berdasarkan biaya perjalanan per kilometer. 216,8

Table 16 Parameter Biaya Operasional

Parameter	Nilai
Biaya Operasional	Rp2.635/Km
Panjang Rute Pulang-Pergi	54,2 Km
Biaya satu trip	Rp142.800/trip
Biaya satu hari	Rp571.200/hari

Biaya operasional ini digunakan dalam perhitungan fungsi objektif yang bertujuan meminimalkan total biaya operasional.

3.2 Hasil

3.2.1 Hasil Persiapan Data Operasional

Tahap persiapan data menghasilkan parameter operasional dasar Bus Trans Sulsel Koridor 5 yang menjadi acuan seluruh tahap perhitungan berikutnya. Parameter ini mencakup jumlah armada, kapasitas kendaraan, jumlah trip per bus, jarak tempuh, serta komponen biaya operasional per kilometer. Ringkasan parameter yang dihasilkan ditampilkan pada Tabel 17.

Table 17 Parameter Operasional Dasar Koridor 5 Bus Trans Sulsel

Parameter	Nilai
Jumlah Halte (Arah Unhas Tamalanrea → Unhas Gowa)	45 halte
Jumlah Halte (Arah Unhas Gowa → Unhas Tamalanrea)	37 halte
Jumlah Bus	15 unit
Kapasitas Bus	40 penumpang
Trip per Bus	4 trip/hari
Total Trip Harian	60 trip
Jarak Pergi	27,83 km
Jarak Pulang (asumsi awal)	26,37 km
Jarak Pergi-Pulang (PP)	54,20 km
Biaya per Km	Rp 2.635
Biaya per Trip	Rp 142.847
Biaya per Bus per Hari	Rp 571.388
Total Biaya Operasional Harian (Eksisting)	Rp 8.570.820

Parameter di atas menjadi dasar perhitungan biaya pada seluruh skenario optimasi (kondisi eksisting, Genetic Algorithm, dan Ant Colony Optimization) yang dibahas pada subbab berikutnya.

Layanan Bus Trans Sulsel Koridor 5 beroperasi mulai pukul 05.00 hingga 21.00 WITA. Hari operasional dibagi menjadi enam periode (P1-P6) berdasarkan karakteristik jam sibuk dan non-sibuk yang teramati di lapangan. Pembagian ini merupakan hasil revisi akhir yang memastikan jumlah bus aktif eksisting (xt) sama dengan jumlah trip

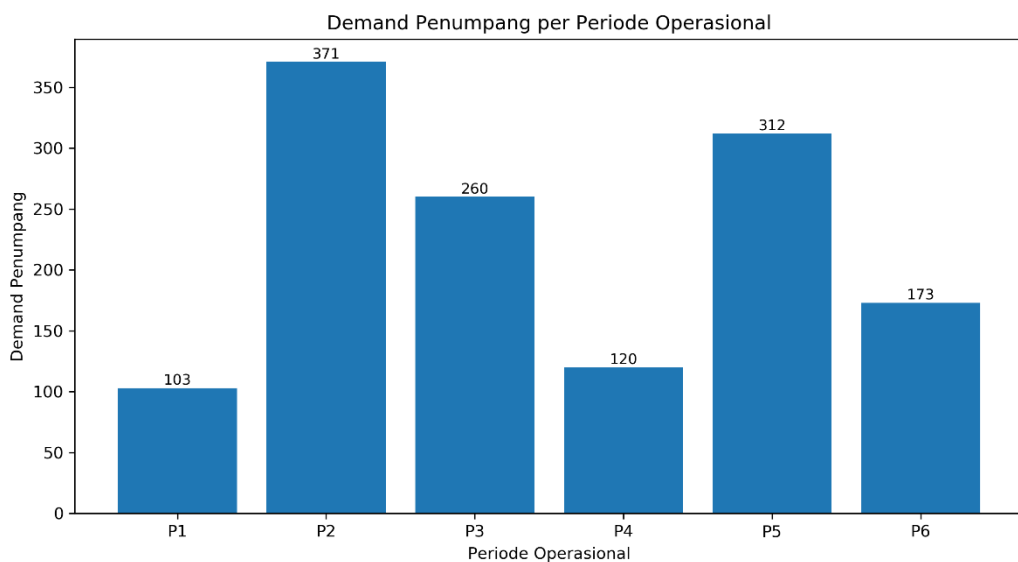
eksisting (qt) pada setiap periode, sehingga tidak ada bus yang harus menjalankan lebih dari satu trip PP dalam periode yang sama.

3.2.2 Hasil Pembangkitan Demand Penumpang

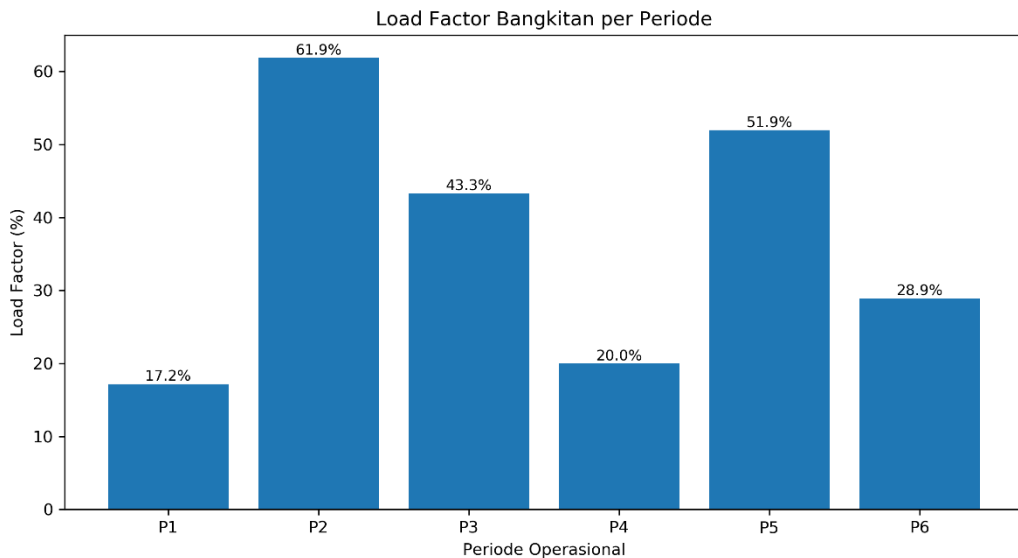
Demand penumpang dibangkitkan untuk enam periode operasional menggunakan load factor acak yang dibatasi pada rentang tiap kategori. Rentang load factor yang digunakan telah disesuaikan dengan hasil observasi langsung di lapangan (Rush Hour 50-62,5%, Normal 25-50%, Non Rush Hour 12,5-25%), lebih rendah dibandingkan asumsi awal penelitian yang cenderung melebih-lebihkan keterisian bus. Hasil pembangkitan demand untuk tiap periode disajikan pada Tabel 18.

Table 18 Hasil Pembangkitan Demand Penumpang per Periode Operasional

Periode	Waktu	Kategori	LF Min-Maks (%)	LF Acak (%)	Demand (penumpang)
P1	05.00-06.30	Non Rush Hour	12,5 - 25,0	17,18	103
P2	06.30-09.00	Rush Hour	50,0 - 62,5	61,88	371
P3	09.00-12.00	Normal	25,0 - 50,0	43,30	260
P4	12.00-15.00	Non Rush Hour	12,5 - 25,0	19,98	120
P5	15.00-18.00	Rush Hour	50,0 - 62,5	51,95	312
P6	18.00-21.00	Normal	25,0 - 50,0	28,90	173



Gambar 5 Demand Penumpang per Periode Operasional



Gambar 6 Load Factor Bangkitan per Periode

Total demand harian yang dibangkitkan adalah 1.339 penumpang, dengan rata-rata load factor bangkitan 37,2%. Demand tertinggi terjadi pada periode P2 (06.30-09.00, Rush Hour) sebesar 371 penumpang, dan terendah pada periode P1 (05.00-06.30, Non Rush Hour) sebesar 103 penumpang, sejalan dengan pola perjalanan komuter yang umum terjadi pada koridor Bus Rapid Transit (BRT) perkotaan.

3.2.3 Hasil Evaluasi Kondisi Eksisting (Baseline)

Kondisi eksisting dievaluasi menggunakan jumlah bus aktif dan headway yang saat ini diterapkan di lapangan pada setiap periode ($x_t = q_t = 8, 10, 12, 12, 10, 8$ sesuai Tabel 17), sebagai pembandingan terhadap hasil optimasi GA dan ACO. Hasil evaluasi baseline ditampilkan pada Tabel 19.

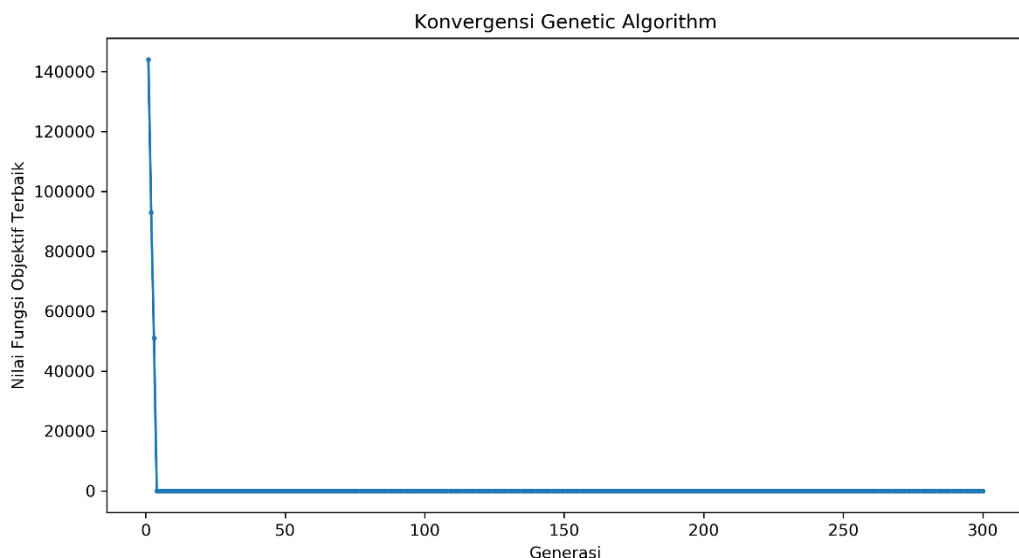
Table 19 Hasil Evaluasi Kondisi Eksisting (Baseline)

Indikator	Nilai
Nilai Fungsi Objektif (Z)	2,2947
Fitness	0,3035
Biaya Operasional Harian	Rp 8.569.020
Waktu Tunggu Total	42,50 menit
Empty Seat	1.061 kursi
Shortage	0 penumpang
Load Factor	55,79%
Utilisasi Armada	125,00%
Total Trip PP Harian	60 trip

Kondisi eksisting menunjukkan headway yang tidak menyesuaikan fluktuasi demand antarperiode (10-20 menit), sehingga terjadi kelebihan kapasitas (empty seat) yang besar terutama pada periode dengan demand rendah dan headway lebar, seperti P1 (headway 15 menit, load factor hanya 32,2%) dan P4 (headway 20 menit, load factor hanya 25,0%). Meskipun tidak terjadi shortage, load factor rata-rata kondisi eksisting hanya 55,79%, mengindikasikan bahwa armada eksisting over-provisioned relatif terhadap demand riil pada sebagian besar periode.

3.2.4 Hasil Optimasi Genetic Algorithm (GA)

Algoritma GA dijalankan dengan parameter sesuai Subbab 2.7.9 (populasi 100 individu, 300 generasi, crossover 0,80, mutasi 0,10, tournament 3, elitisme 2 individu). Grafik konvergensi nilai fungsi objektif terbaik terhadap generasi ditampilkan pada Gambar 7.



Gambar 7 Konvergensi Genetic Algorithm

GA mencapai nilai fungsi objektif terbaik pada generasi ke-37 dan tidak mengalami perbaikan lebih lanjut hingga generasi ke-300, menunjukkan konvergensi yang stabil jauh sebelum batas generasi maksimum tercapai. Ringkasan hasil optimasi GA ditampilkan pada Tabel 20, dan rincian per periode pada Tabel 21.

Table 20 Ringkasan Hasil Genetic Algorithm

Indikator	Nilai
Nilai Fungsi Objektif (Z)	1,3339
Fitness	0,4285
Biaya Operasional Harian	Rp 5.141.412
Waktu Tunggu Total	30,00 menit
Empty Seat	101 kursi

Indikator	Nilai
Shortage	0 penumpang
Load Factor	92,99%
Utilisasi Armada	112,50%
Total Trip PP Harian	36 trip
Waktu Komputasi	183,17 detik

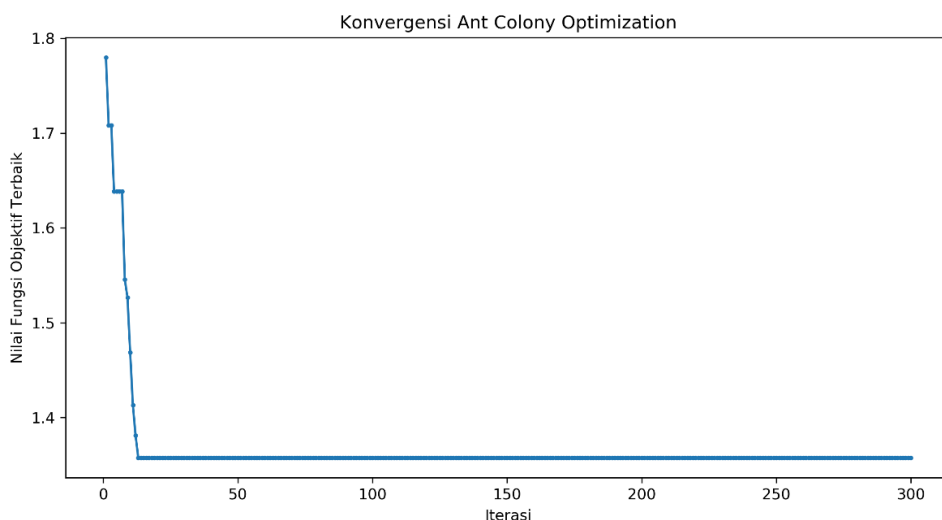
Table 21 Detail Alokasi per Periode - Genetic Algorithm

Periode	Demand	Bus Aktif	Trip PP	Headway	Load Factor (%)	Empty Seat	Biaya (Rp)
P1	103	4	3	10	85,83	17	428.451
P2	371	13	10	10	92,75	29	1.428.170
P3	260	12	7	10	92,86	20	999.719
P4	120	4	3	10	100,00	0	428.451
P5	312	10	8	10	97,50	8	1.142.536
P6	173	11	5	10	86,50	27	714.085

GA menurunkan total trip PP harian dari 60 menjadi 36 trip (turun 40%), dengan biaya operasional turun dari Rp8.569.020 menjadi Rp5.141.412 (hemat 40,00%), tanpa menimbulkan shortage pada periode manapun. Headway pada seluruh periode konvergen ke nilai minimum yang diizinkan (10 menit).

3.2.5 Hasil Optimasi Ant Colony Optimization (ACO)

ACO dijalankan dengan parameter sesuai Subbab 2.8.9 (80 semut, 300 iterasi, $\alpha=1,0$, $\beta=2,0$, $\rho=0,50$, $Q=1,0$, $\tau_0=1,0$). Grafik konvergensi ditampilkan pada Gambar 8.



Gambar 8 Konvergensi Ant Colony Optimization

ACO mencapai nilai fungsi objektif terbaik lebih cepat, yaitu pada iterasi ke-13, dan bertahan stabil hingga iterasi ke-300. Ringkasan hasil ACO ditampilkan pada Tabel 22, dan rincian per periode pada Tabel 23.

Table 22 Ringkasan Hasil Ant Colony Optimization

Indikator	Nilai
Nilai Fungsi Objektif (Z)	1,3575
Fitness	0,4242
Biaya Operasional Harian	Rp 5.141.412
Waktu Tunggu Total	31,00 menit
Empty Seat	101 kursi
Shortage	0 penumpang
Load Factor	92,99%
Utilisasi Armada	158,33%
Total Trip PP Harian	36 trip
Waktu Komputasi	223,93 detik

Table 23 Detail Alokasi per Periode - Ant Colony Optimization

Periode	Demand	Bus Aktif	Trip PP	Headway	Load Factor (%)	Empty Seat	Biaya (Rp)
P1	103	14	3	10	85,83	17	428.451
P2	371	10	10	10	92,75	29	1.428.170
P3	260	12	7	10	92,86	20	999.719
P4	120	15	3	10	100,00	0	428.451
P5	312	10	8	10	97,50	8	1.142.536
P6	173	15	5	12	86,50	27	714.085

ACO memperoleh total trip dan biaya operasional yang identik dengan GA (36 trip, Rp5.141.412), karena kedua metode sama-sama mencapai kombinasi trip optimal yang memenuhi seluruh demand tanpa shortage. Perbedaan utama terletak pada distribusi jumlah bus aktif per periode (misalnya P1: GA=4 bus vs ACO=14 bus, sama-sama menjalankan 3 trip) dan headway pada P6 (ACO=12 menit, sedikit di atas minimum).

3.2.6 Perbandingan Kinerja Existing, GA, dan ACO

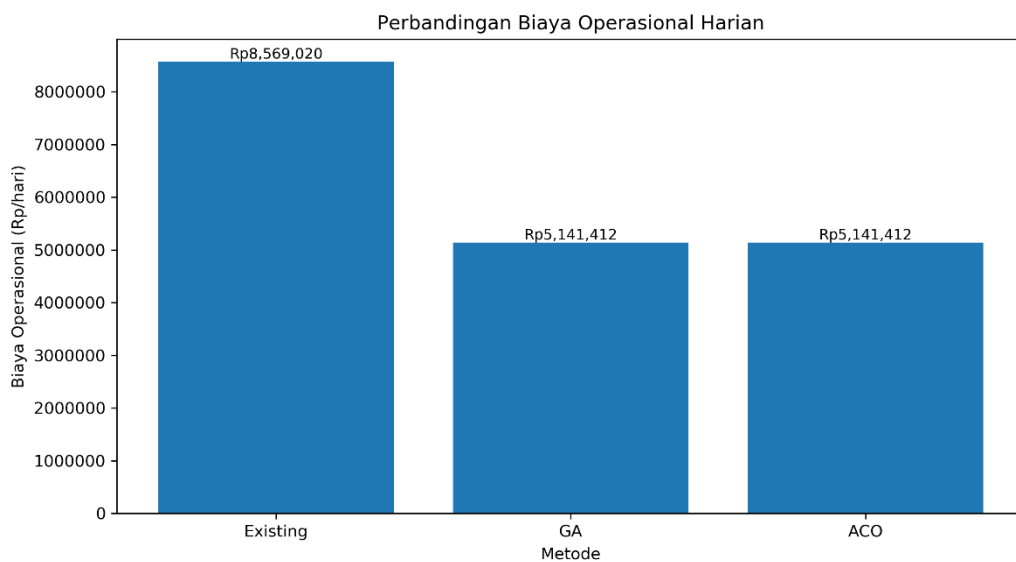
Perbandingan menyeluruh antara kondisi eksisting, GA, dan ACO disajikan pada Tabel 24 (ringkasan) dan Tabel 25 (rincian per periode).

Table 24 Ringkasan Perbandingan Existing, GA, dan ACO

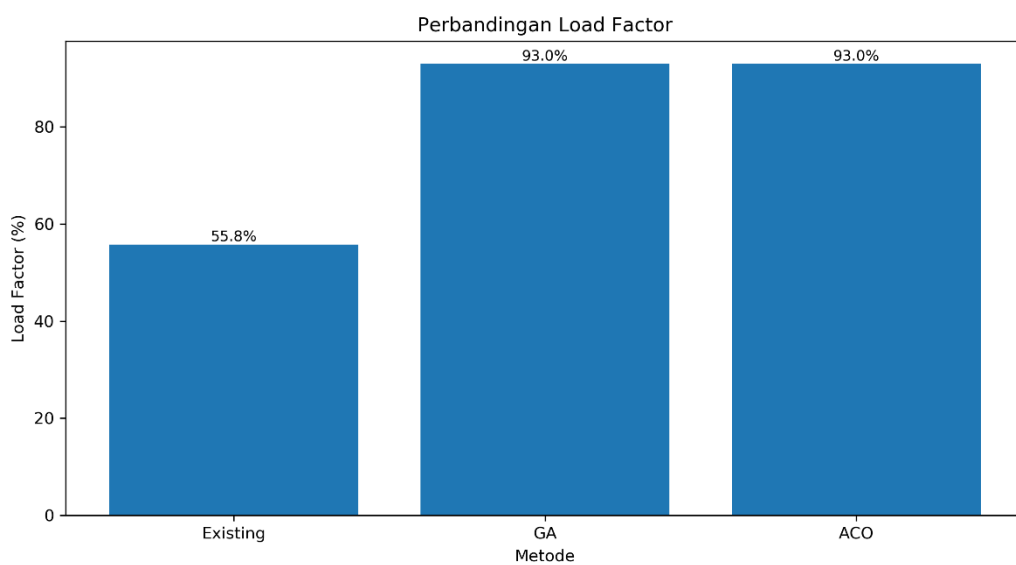
Aspek	Indikator	Existing	GA	ACO	Metode Terbaik
Efisiensi Biaya	Biaya Operasional Harian (Rp)	8.569.020	5.141.412	5.141.412	Sama
Efisiensi Biaya	Penghematan Biaya (%)	0,00	40,00	40,00	Sama
Efisiensi Biaya	Biaya per Penumpang (Rp)	6.399,57	3.839,74	3.839,74	Sama
Operasional Armada	Total Trip PP Harian	60	36	36	Sama
Kualitas Layanan	Waktu Tunggu Total (menit)	42,50	30,00	31,00	GA
Kualitas Layanan	Load Factor (%)	55,79	92,99	92,99	Sama
Efisiensi Armada	Empty Seat (kursi)	1.061	101	101	Sama
Efisiensi Armada	Utilisasi Armada (%)	125,00	112,50	158,33	ACO
Kinerja Algoritma	Nilai Fungsi Objektif	2,2947	1,3339	1,3575	GA
Kinerja Algoritma	Fitness	0,3035	0,4285	0,4242	GA
Kinerja Algoritma	Waktu Komputasi (detik)	-	183,17	223,93	GA

Table 25 Perbandingan Konfigurasi Operasional per Periode

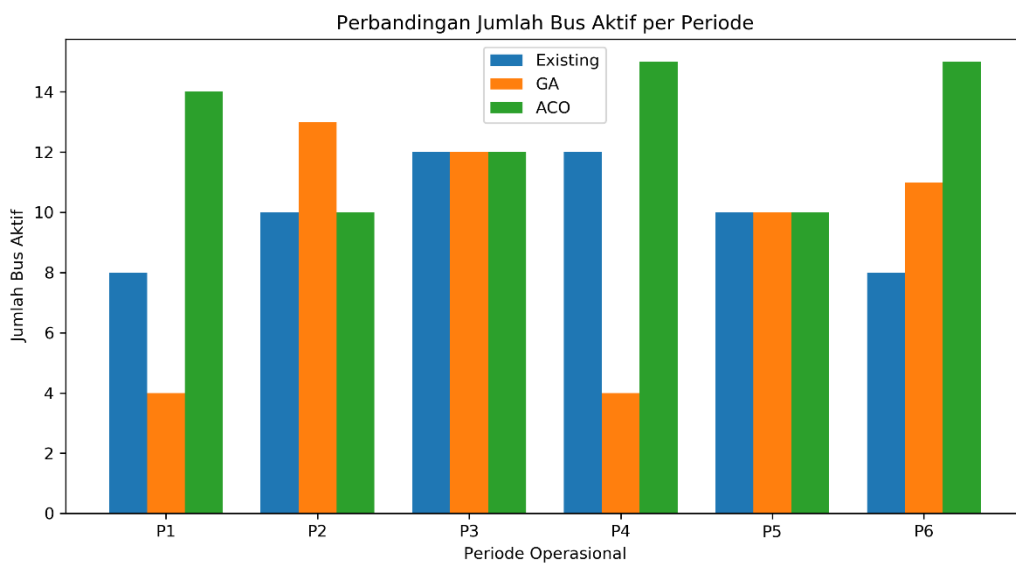
Periode	Demand	Trip Existing	Trip GA	Trip ACO	LF Existing (%)	LF GA (%)	LF ACO (%)
P1	103	8	3	3	32,19	85,83	85,83
P2	371	10	10	10	92,75	92,75	92,75
P3	260	12	7	7	54,17	92,86	92,86
P4	120	12	3	3	25,00	100,00	100,00
P5	312	10	8	8	78,00	97,50	97,50
P6	173	8	5	5	54,06	86,50	86,50



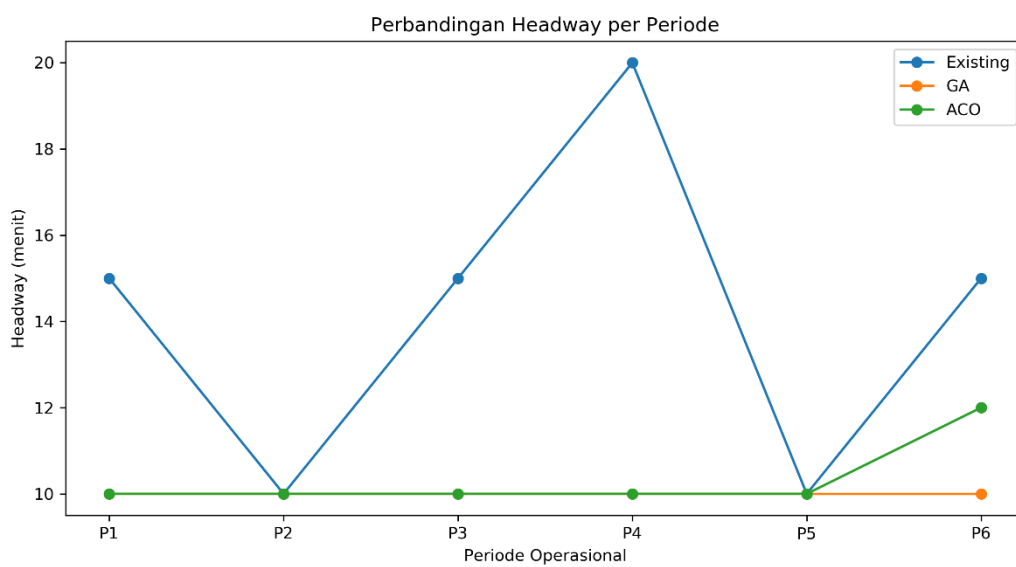
Gambar 9 Perbandingan Biaya Operasional Harian



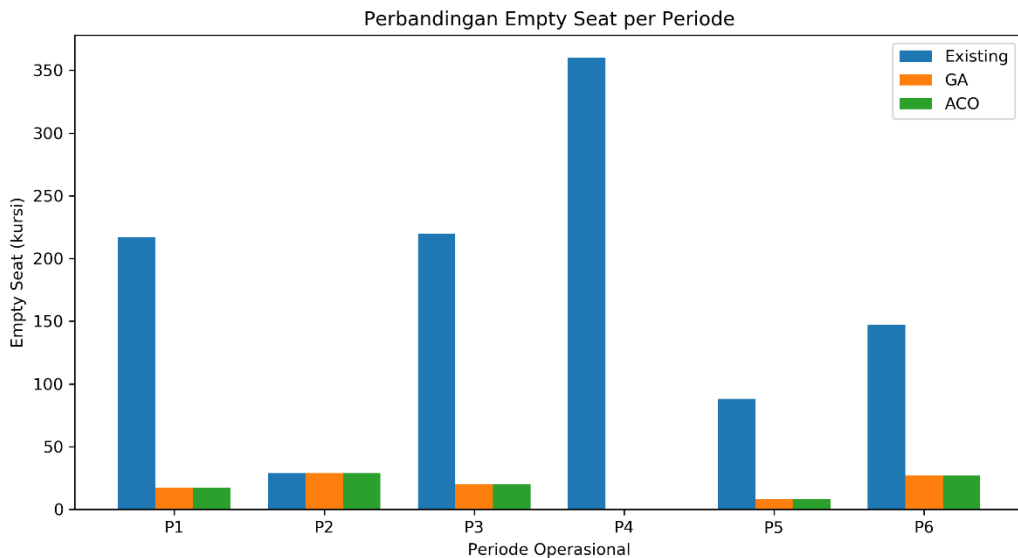
Gambar 10 Perbandingan Load Factor



Gambar 11 Perbandingan Jumlah Bus Aktif per Periode



Gambar 12 Perbandingan Headway per Periode



Gambar 13 Perbandingan Empty Seat per Periode

Baik GA maupun ACO menghasilkan kombinasi trip PP per periode yang identik (jumlah trip pada Tabel 25 sama persis untuk kedua metode), sehingga biaya operasional dan load factor keduanya juga identik. Perbedaan hanya muncul pada nilai fungsi objektif, fitness, waktu tunggu, dan utilisasi armada, yang dipengaruhi oleh perbedaan distribusi bus aktif dan headway pada periode P6. GA memberikan hasil sedikit lebih baik pada nilai fungsi objektif, fitness, waktu tunggu, dan waktu komputasi, sementara ACO unggul pada indikator utilisasi armada. Selisih nilai fungsi objektif antara keduanya hanya sekitar 1,74%, sehingga kedua metode dapat dianggap sama-sama layak diterapkan untuk permasalahan ini.

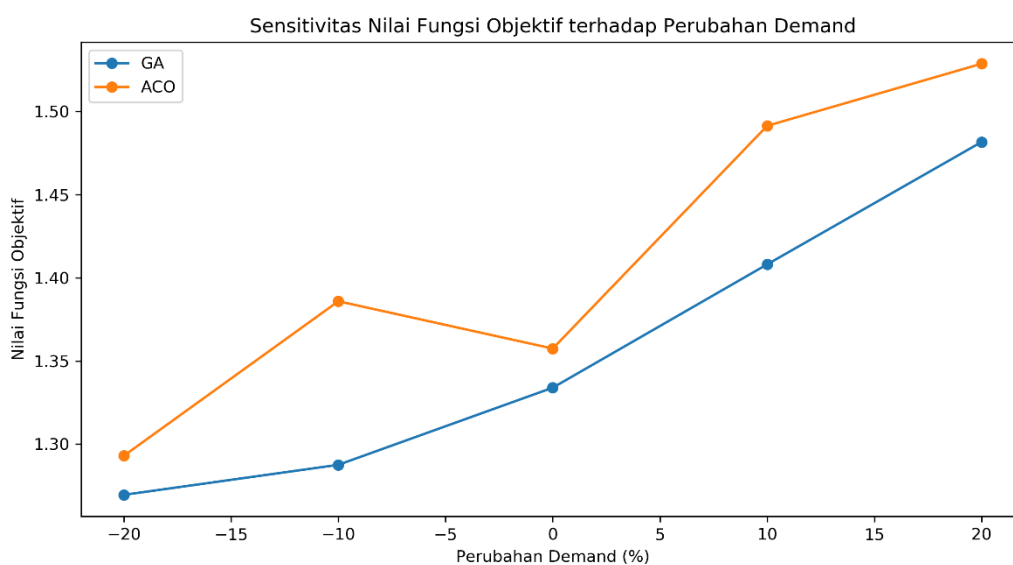
3.2.7 Hasil Analisis Sensitivitas

Analisis sensitivitas dilakukan dengan menguji lima skenario perubahan demand (-20%, -10%, 0%, +10%, +20%) terhadap demand baseline, dan menjalankan ulang GA serta ACO pada setiap skenario. Ringkasan hasil disajikan pada Tabel 26 , Gambar 14 dan Gambar 15.

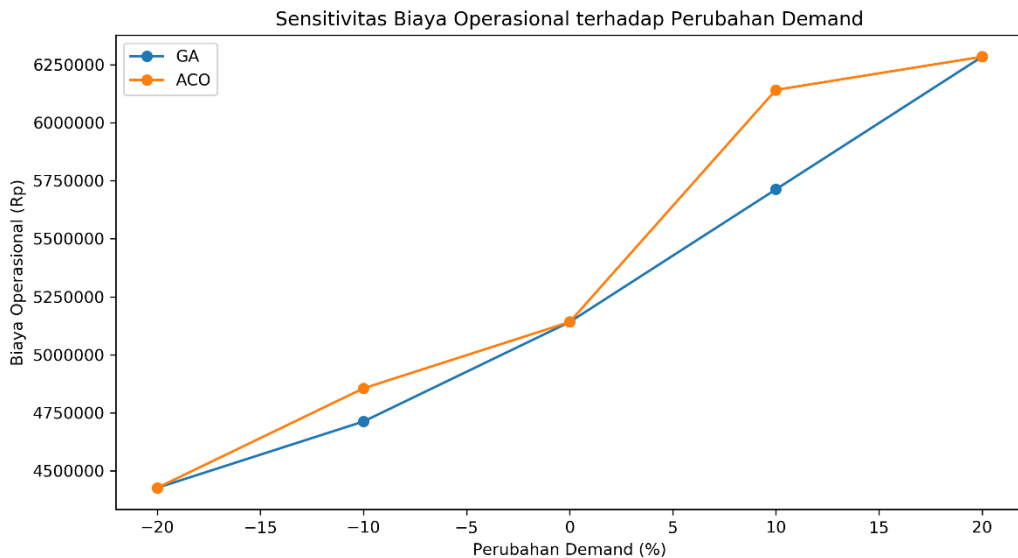
Table 26 Ringkasan Analisis Sensitivitas Perubahan Demand

Skenario	Metode	Nilai Objektif	Biaya Operasional (Rp)	Shortage	Load Factor (%)	Total Trip
Demand -20%	GA	1,2695	4.427.327	0	86,37	31
Demand -20%	ACO	1,2930	4.427.327	0	86,37	31
Demand -10%	GA	1,2875	4.712.961	0	91,36	33

Skenario	Metode	Nilai Objektif	Biaya Operasional (Rp)	Shortage	Load Factor (%)	Total Trip
Demand -10%	ACO	1,3859	4.855.778	0	88,68	34
Demand 0%	GA	1,3339	5.141.412	0	92,99	36
Demand 0%	ACO	1,3575	5.141.412	0	92,99	36
Demand +10%	GA	1,4081	5.712.680	0	92,00	40
Demand +10%	ACO	1,4914	6.141.131	0	85,58	43
Demand +20%	GA	1,4817	6.283.948	0	91,31	44
Demand +20%	ACO	1,5288	6.283.948	0	91,31	44



Gambar 14 Sensitivitas Nilai Fungsi Objektif terhadap Perubahan Demand



Gambar 15 Sensitivitas Biaya Operasional terhadap Perubahan Demand

Berbeda dari evaluasi awal penelitian ini yang menggunakan asumsi load factor lebih tinggi, hasil sensitivitas dengan load factor realistis menunjukkan bahwa shortage tetap 0 penumpang pada seluruh skenario, termasuk pada kenaikan demand +20%. Hal ini terjadi karena total trip PP yang dialokasikan pada skenario +20% (44 trip) masih berada di bawah batas maksimum 60 trip/hari, sehingga armada eksisting (15 bus x 4 trip) masih memiliki kapasitas cadangan yang cukup untuk mengakomodasi pertumbuhan demand hingga +20% tanpa penambahan armada. Biaya operasional meningkat secara proporsional terhadap penambahan trip, dari Rp4.427.327/hari pada skenario -20% hingga Rp6.283.948/hari pada skenario +20%.

3.2.8 Evaluasi Arah Pulang Berbasis Data Lapangan

Untuk memperkuat validitas model gabungan pergi-pulang, dilakukan analisis tambahan menggunakan data lapangan arah pulang, yaitu data halte (37 halte) dan data jadwal aktual (60 trip tercatat).

3.2.8.1 Validasi Jarak Pulang Awal

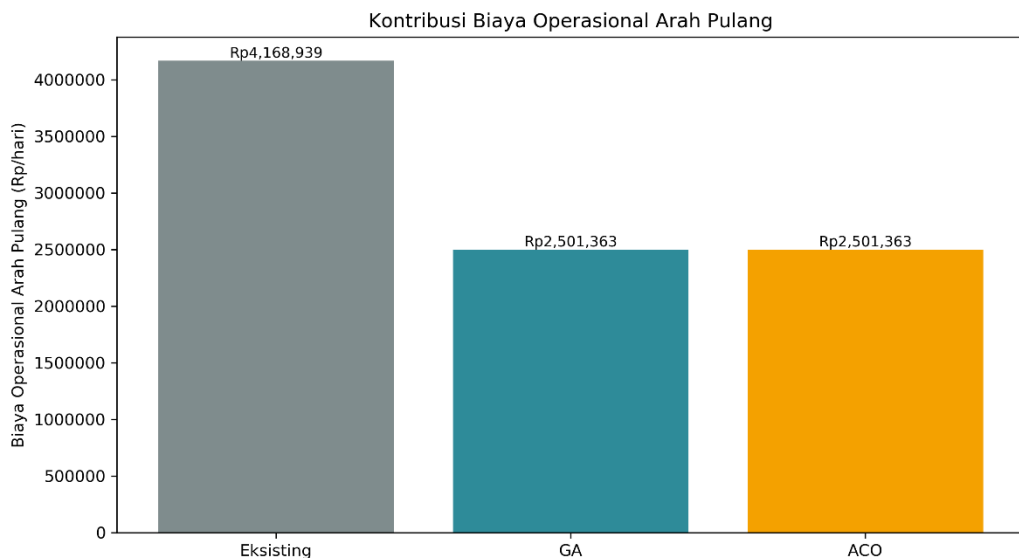
Jarak pulang aktual yang dihitung dari data halte arah pulang adalah 26,369 km, dengan proporsi terhadap jarak pergi-pulang (54,20 km) sebesar 48,65%. Nilai ini konsisten dengan asumsi jarak pulang yang digunakan dalam model optimasi (26,37 km), dengan selisih di bawah 1%. Dengan demikian, hasil analisis ini berfungsi sebagai triangulasi data, bukan koreksi terhadap parameter yang telah digunakan dalam model GA dan ACO.

3.2.8.2 Kontribusi Biaya Operasional Arah Pulang

Berdasarkan proporsi jarak pulang tersebut, kontribusi biaya operasional arah pulang dihitung secara proporsional dari total biaya operasional pada masing-masing skenario. Hasil perhitungan disajikan pada Tabel 27 dan Gambar 16.

Table 27 Kontribusi Biaya Operasional Arah Pulang

Skenario	Biaya Operasional Total (Rp/hari)	Kontribusi Arah Pulang (~48,65% jarak) (Rp/hari)
Eksisting	8.569.020	4.168.938,90
GA	5.141.412	2.501.363,34
ACO	5.141.412	2.501.363,34



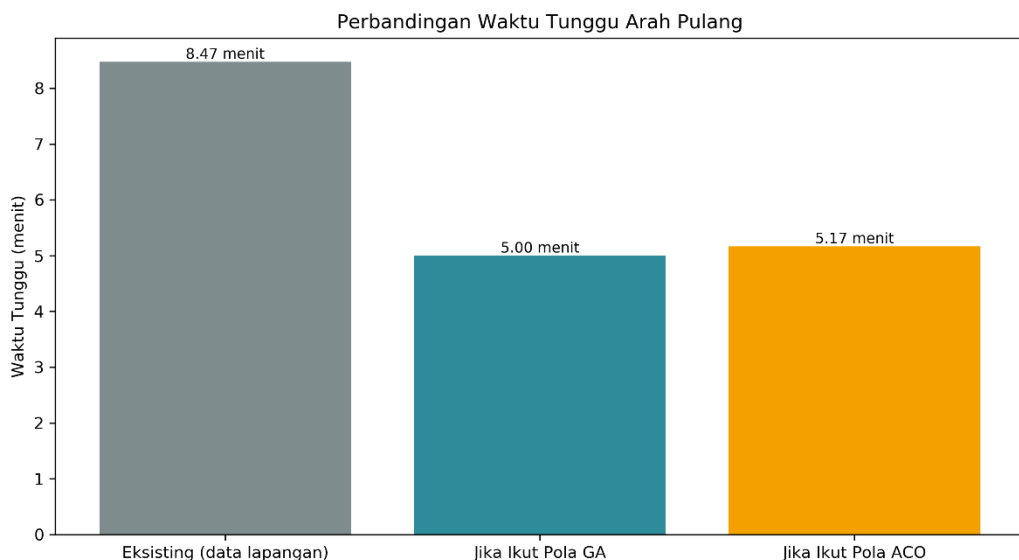
Gambar 16 Kontribusi Biaya Operasional Arah Pulang

3.2.8.3 Waktu Tunggu Arah Pulang

Waktu tunggu arah pulang kondisi eksisting dihitung dari headway aktual di lapangan (rata-rata 16,95 menit, standar deviasi 9,62 menit), kemudian dibandingkan dengan skenario seandainya arah pulang mengikuti pola headway hasil optimasi GA (10,00 menit) maupun ACO (10,33 menit). Hasil perbandingan disajikan pada Tabel 28 dan Gambar 17.

Table 28 Perbandingan Waktu Tunggu Arah Pulang

Skenario	Rata-rata Headway (menit)	Rata-rata Waktu Tunggu (menit)	Penurunan vs Eksisting (%)
Eksisting (data lapangan)	16,95	8,475	0,00
Jika Ikut Pola GA	10,00	5,000	41,00
Jika Ikut Pola ACO	10,33	5,167	39,04

**Gambar 17** Perbandingan Waktu Tunggu Arah Pulang

Data lapangan menunjukkan rata-rata waktu tunggu arah pulang sebesar 8,475 menit. Apabila arah pulang mengikuti pola headway hasil optimasi GA maupun ACO, waktu tunggu dapat ditekan menjadi sekitar 5 menit, atau turun sekitar 39-41% dibandingkan kondisi eksisting lapangan.

3.2.8.4 Catatan Metodologis

Kontribusi biaya arah pulang dihitung secara proporsional berdasarkan rasio jarak terhadap jarak PP, karena optimasi GA dan ACO pada penelitian ini dijalankan pada model gabungan pergi-pulang dan tidak dijalankan ulang secara terpisah untuk masing-masing arah. Nilai kontribusi biaya arah pulang bersifat estimasi alokasi berdasarkan proporsi jarak, bukan hasil optimasi independen untuk arah pulang, karena armada yang beroperasi pada arah pergi dan arah pulang merupakan armada yang sama.

3.3 Pembahasan

3.3.1 Efektivitas Optimasi

Baik GA maupun ACO terbukti mampu menurunkan total trip PP harian dari 60 menjadi 36 trip (turun 40%) dan biaya operasional dari Rp8.569.020 menjadi Rp5.141.412/hari (hemat 40,00%), sekaligus meningkatkan load factor rata-rata dari 55,79% menjadi 92,99%, tanpa menyebabkan shortage. Dengan rentang load factor yang telah disesuaikan terhadap kondisi riil di lapangan, temuan ini menunjukkan bahwa armada eksisting selama ini over-provisioned relatif terhadap demand aktual, sehingga optimasi tidak sekadar meredistribusi trip yang ada, melainkan benar-benar menghasilkan penghematan biaya operasional yang signifikan.

3.3.2 Perbandingan GA dan ACO

GA memberikan hasil sedikit lebih baik pada nilai fungsi objektif, fitness, waktu tunggu, dan waktu komputasi, sementara ACO unggul tipis pada utilisasi armada. Kedua metode menghasilkan kombinasi total trip dan biaya operasional yang identik. GA mencapai konvergensi pada generasi ke-37 dari 300 generasi, sedangkan ACO mencapai konvergensi lebih cepat secara relatif, yaitu pada iterasi ke-13 dari 300 iterasi, meskipun total waktu komputasi ACO (223,93 detik) lebih lama dibandingkan GA (183,17 detik) karena beban komputasi per iterasi ACO (pembentukan solusi oleh 80 semut dan pembaruan feromon) lebih tinggi dibandingkan satu generasi GA. Selisih nilai fungsi objektif antara kedua metode relatif kecil (sekitar 1,74%), sehingga keduanya dapat dianggap sama-sama layak diterapkan; pemilihan metode akhir dapat mempertimbangkan kemudahan implementasi di lapangan.

3.3.3 Keterbatasan Variabel Headway

Pada hampir seluruh periode dan skenario, GA maupun ACO secara konsisten memilih headway minimum (10 menit), dengan sedikit pengecualian pada periode tertentu (misalnya P6 pada solusi ACO sebesar 12 menit). Hal ini terjadi karena formulasi fungsi objektif menghubungkan biaya operasional hanya dengan jumlah trip (qt), sedangkan headway hanya memengaruhi komponen waktu tunggu tanpa konsekuensi biaya tambahan, sehingga headway yang lebih kecil selalu menguntungkan nilai fungsi objektif. Untuk penelitian lanjutan, disarankan mengaitkan headway secara eksplisit dengan jumlah trip yang harus dilayani per bus ($ht = Lt/qt$, sesuai Subbab 2.3.5), agar trade-off antara jumlah trip dan headway tercermin lebih realistis.

3.3.4 Batas Kapasitas Armada terhadap Pertumbuhan Demand

Hasil analisis sensitivitas menunjukkan bahwa dengan load factor realistis, kapasitas 60 trip/hari yang tersedia dari 15 bus mampu menampung pertumbuhan demand hingga +20% tanpa menimbulkan shortage, karena total trip yang dialokasikan pada skenario tersebut (44 trip) masih berada dalam batas kapasitas armada eksisting. Temuan ini berbeda signifikan dari evaluasi awal penelitian (yang menggunakan asumsi load factor lebih tinggi dan sempat menunjukkan risiko shortage pada kenaikan demand), dan

menunjukkan bahwa armada eksisting Koridor 5 saat ini memiliki kapasitas cadangan yang memadai untuk mengantisipasi pertumbuhan demand jangka menengah.

3.3.5 Konsistensi Hasil dengan Kondisi Arah Pulang

Evaluasi tambahan berbasis data lapangan arah pulang (Subbab 3.8) memperkuat validitas parameter jarak yang digunakan dalam model gabungan pergi-pulang, dengan selisih jarak pulang aktual terhadap asumsi awal di bawah 1%. Simulasi penerapan pola headway hasil GA/ACO pada arah pulang menunjukkan potensi penurunan waktu tunggu sekitar 39-41% dibandingkan kondisi eksisting lapangan, memperkuat kesimpulan bahwa penerapan headway yang lebih teratur memberikan manfaat langsung bagi kualitas layanan penumpang.

3.3.6 Implikasi Praktis

Secara keseluruhan, solusi optimasi yang dihasilkan (baik GA maupun ACO) dapat menjadi rekomendasi kebijakan operasional Trans Sulse Koridor 5, khususnya dalam mengurangi jumlah trip PP harian dari 60 menjadi sekitar 36 trip sesuai fluktuasi demand pada setiap periode, guna mencapai efisiensi biaya operasional hingga 40% tanpa mengorbankan kualitas layanan (load factor naik dari 55,79% menjadi 92,99%, tanpa shortage). Evaluasi tambahan pada arah pulang menunjukkan bahwa manfaat tersebut konsisten ketika ditinjau menggunakan data operasional aktual, sehingga memperkuat kelayakan rekomendasi untuk diterapkan secara nyata pada pengelolaan operasional koridor ini.

BAB IV

KESIMPULAN DAN SARAN

4.1 Kesimpulan

Berdasarkan hasil penelitian dan pembahasan yang telah diuraikan pada BAB III, penelitian ini menjawab keempat rumusan masalah yang ditetapkan pada BAB I sebagai berikut.

1. Konfigurasi operasional Bus Trans Sulsel Koridor 5 yang optimal diperoleh dengan menyesuaikan jumlah trip PP yang dijalankan pada setiap periode operasional sesuai fluktuasi permintaan penumpang, alih-alih menjalankan seluruh 60 trip PP harian secara seragam sepanjang hari seperti kondisi eksisting. Baik Genetic Algorithm maupun Ant Colony Optimization merekomendasikan konfigurasi trip PP yang identik, yaitu [3, 10, 7, 3, 8, 5] untuk periode P1 hingga P6 (total 36 trip/hari), dengan headway seragam 10 menit pada hampir seluruh periode. Konfigurasi ini menurunkan biaya operasional harian sekitar 40% (dari Rp8.569.020 menjadi Rp5.141.412 per hari) sekaligus meningkatkan load factor rata-rata dari 55,79% menjadi 92,99%, tanpa menimbulkan shortage pada kondisi demand normal.
2. Algoritma Genetika (GA) menunjukkan kinerja yang baik dalam menyelesaikan permasalahan optimasi ini, ditandai dengan konvergensi nilai fungsi objektif dari kondisi awal populasi acak menuju nilai stabil 1,3339 pada generasi ke-37 dari 300 generasi, dengan waktu komputasi 183,17 detik (populasi 100 individu, crossover 0,80, mutasi 0,10). GA memberikan hasil terbaik pada mayoritas indikator kinerja, yaitu nilai fungsi objektif, fitness, waktu tunggu penumpang, dan waktu komputasi.
3. Ant Colony Optimization (ACO) juga menunjukkan kinerja yang baik, dengan nilai fungsi objektif konvergen ke 1,3575 pada iterasi ke-13 dari 300 iterasi, meskipun total waktu komputasi (223,93 detik) sedikit lebih lama dibandingkan GA karena beban komputasi per iterasi yang lebih tinggi (80 semut, pembaruan feromon setiap iterasi). ACO menghasilkan alokasi trip PP dan biaya operasional yang identik dengan GA, dan sedikit lebih unggul pada indikator utilisasi armada (158,33% berbanding 112,50% pada GA).
4. Berdasarkan perbandingan menyeluruh terhadap sepuluh indikator kinerja, Genetic Algorithm terbukti sedikit lebih efektif dibandingkan Ant Colony Optimization untuk permasalahan optimasi operasional Bus Trans Sulsel Koridor 5 pada penelitian ini, dengan selisih nilai fungsi objektif yang relatif kecil (sekitar 1,74%). Meskipun demikian, karena perbedaan performa keduanya tidak signifikan dan keduanya menghasilkan konfigurasi trip serta biaya operasional yang identik, Ant Colony Optimization tetap dapat dipertimbangkan sebagai alternatif yang layak, terutama apabila utilisasi armada menjadi prioritas utama pengelola.

Selain menjawab keempat rumusan masalah tersebut, penelitian ini juga menghasilkan beberapa temuan tambahan yang memperkaya pembahasan:

5. Hasil analisis sensitivitas menunjukkan bahwa kapasitas maksimum 15 unit armada (60 trip PP/hari) masih mampu menampung pertumbuhan demand hingga 20% di

atas kondisi saat ini tanpa menimbulkan shortage, karena total trip yang dialokasikan pada skenario tersebut (44 trip) masih berada di bawah batas kapasitas armada eksisting. Temuan ini menunjukkan bahwa armada eksisting Koridor 5 memiliki kapasitas cadangan yang memadai untuk mengantisipasi pertumbuhan demand jangka menengah.

6. Evaluasi tambahan terhadap data lapangan arah pulang (jarak dan jadwal aktual) memvalidasi parameter jarak yang digunakan dalam model (selisih di bawah 1% terhadap asumsi awal), serta menunjukkan bahwa penerapan pola headway hasil optimasi berpotensi menurunkan rata-rata waktu tunggu arah pulang sekitar 39-41% dibandingkan kondisi eksisting di lapangan.
7. Ditemukan pula keterbatasan metodologis bahwa headway secara konsisten konvergen ke nilai minimum pada hampir seluruh periode dan skenario (dengan sedikit pengecualian, seperti periode P6 pada solusi ACO sebesar 12 menit), karena formulasi fungsi objektif penelitian ini menghubungkan biaya operasional hanya dengan jumlah trip PP, sedangkan headway hanya memengaruhi komponen waktu tunggu tanpa konsekuensi biaya tambahan, sehingga headway yang lebih kecil selalu menguntungkan nilai fungsi objektif.

4.2 Saran

Berdasarkan kesimpulan di atas serta keterbatasan yang ditemukan selama penelitian, berikut disampaikan beberapa saran bagi pengelola Bus Trans Sulsel maupun bagi pengembangan penelitian selanjutnya.

4.2.1 Saran bagi Pengelola Bus Trans Sulsel

1. Pengelola Bus Trans Sulsel Koridor 5 disarankan mempertimbangkan penerapan konfigurasi jumlah trip PP per periode hasil optimasi (GA maupun ACO), yaitu pengurangan total trip harian dari 60 menjadi sekitar 36 trip sesuai fluktuasi demand pada setiap periode, sebagai kebijakan operasional guna menekan biaya operasional hingga 40% dan meningkatkan load factor tanpa mengorbankan kualitas layanan.
2. Mengingat kapasitas 15 unit armada (60 trip/hari) masih memadai hingga pertumbuhan demand sekitar 20% di atas kondisi saat ini, pengelola tidak perlu segera menambah armada dalam jangka pendek, namun tetap disarankan memantau pertumbuhan demand secara berkala dan menyiapkan strategi antisipatif seperti penambahan armada apabila proyeksi pertumbuhan penumpang diperkirakan melampaui ambang tersebut.
3. Mengingat headway arah pulang di lapangan masih bervariasi cukup besar (standar deviasi 9,62 menit), pengelola disarankan mengevaluasi kemungkinan penerapan headway yang lebih teratur pada arah pulang, mengingat simulasi menunjukkan potensi penurunan waktu tunggu penumpang sekitar 39-41%.

4.2.2 Saran bagi Penelitian Selanjutnya

1. Fungsi objektif pada penelitian selanjutnya disarankan mengaitkan headway dengan jumlah trip yang harus dilayani per bus (sesuai hubungan $ht = Lt/qt$ pada Subbab

2.3.5), sehingga headway yang lebih kecil turut membawa konsekuensi biaya, dan trade-off antara jumlah trip dan headway dapat tercermin secara lebih realistis dalam proses optimasi.

2. Penelitian selanjutnya disarankan menggunakan data historis jumlah penumpang aktual (bukan data simulasi) apabila tersedia, untuk meningkatkan validitas eksternal model terhadap kondisi riil operasional Bus Trans Susel.
3. Penelitian selanjutnya dapat mempertimbangkan pemodelan arah pergi dan arah pulang secara terintegrasi dalam satu optimasi (bukan dievaluasi terpisah secara proporsional seperti pada penelitian ini), mengingat kedua arah dilayani oleh armada yang sama secara bergantian dalam satu siklus operasional harian.
4. Penelitian selanjutnya dapat membandingkan Genetic Algorithm dan Ant Colony Optimization dengan metode metaheuristik lain, seperti Particle Swarm Optimization atau pendekatan hibrida GA-ACO, untuk memperkaya alternatif solusi optimasi operasional transportasi publik.
5. Penelitian selanjutnya dapat mengembangkan fungsi objektif menjadi pendekatan multi-objektif eksplisit (misalnya melalui pencarian solusi Pareto optimal), sehingga trade-off antara biaya operasional, waktu tunggu, dan kualitas layanan dapat dieksplorasi tanpa perlu menetapkan bobot tetap di awal.

DAFTAR PUSTAKA

- Basak, S., Sun, F., Sengupta, S. and Dubey, A., 2019. Data-driven optimization of public transit schedule. arXiv:1912.02574.
- Blum, C., 2005. Ant colony optimization: Introduction and recent trends. *Physics of Life Reviews* 2(4), 353–373. doi: 10.1016/j.plrev.2005.10.001.
- Ceder, A., 2015. *Public Transit Planning and Operation: Modeling, Practice and Behavior*. CRC Press, Boca Raton.
- Dorigo, M., 1992. *Optimization, Learning and Natural Algorithms*. Ph.D. Dissertation, Politecnico di Milano, Milan.
- Dorigo, M. and Gambardella, L.M., 1997. Ant colony system: A cooperative learning approach to the traveling salesman problem. *IEEE Trans. Evol. Comput.* 1(1), 53–66. doi: 10.1109/4235.585892.
- Dorigo, M. and Stützle, T., 2004. *Ant Colony Optimization*. MIT Press, Cambridge, MA
- Goldberg, D.E., 1989. *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley, Reading, MA.
- Holland, J.H., 1975. *Adaptation in Natural and Artificial Systems*. University of Michigan Press, Ann Arbor.
- Korzeń, M. and Kruszyna, M., 2025. Public transport planning using modified ant colony optimization. *Sustainability* 17(6), 2468.
- Law, A.M., 2015. *Simulation Modeling and Analysis*. 5th ed. McGraw-Hill Education, New York.
- Liu, T. and Ceder, A., 2023. Optimizing public transport transfers by integrating timetable coordination and vehicle scheduling. *Transportation Research Record*.
- Mavrovouniotis, M. and Yang, S., 2018. Ant colony optimization for dynamic combinatorial optimization problems. In: *Swarm Intelligence – Volume 1: Principles, Current Algorithms and Methods*, pp. 121–142. Institution of Engineering and Technology (IET).
- Mertens, L., Wolbeck, L.-A., Kliwer, N., Rößler, D. and Xie, L., 2023. An overview of optimization approaches for scheduling and rostering resources in public transportation. arXiv:2310.13425.
- Mitchell, M., 1998. *An Introduction to Genetic Algorithms*. MIT Press, Cambridge, MA.
- PT JAMKRIDA SULSEL, 2025. Rute Bus Trans Sulsel Jangkau Mamminasata, Hadirkan Layanan Terintegrasi Antarwilayah [Online]. Dinas Perhubungan Provinsi Sulawesi Selatan. Available at: <https://www.jamkridasulsel.co.id> [Accessed: 02 Feb 2026].
- Reilly, J. and Levinson, H., 2011. *Public Transport Capacity Analysis Procedures for Developing Cities*. World Bank, Washington, DC.
- Saw, V.L., Chung, N.N., Quek, W.L., Pang, Y.E.I. and Chew, L.Y., 2018. Bus bunching as a synchronisation phenomenon. arXiv:1812.00609.

- Sui, X., Yan, H., Pan, S., Li, X. and Gu, X., 2025. Bus system optimization for timetables, routes, charging, and facilities: a summary. *Digital Transportation and Safety*, 4(1), 1–9. doi: 10.48130/dts-0024-0024.
- Vuchic, V.R., 2007. *Urban Transit Systems and Technology*. John Wiley & Sons, Hoboken.
- Wang, M., Zhang, Y., Li, X. and Chen, J., 2024. Research on bus scheduling optimization considering passenger demand. *Applied Sciences* 14(10), 4126. doi: 10.3390/app14104126.
- Yu, K. and Wang, Z., 2018. Research on survey and calculation of load factor in urban public transportation. *IOP Conf. Ser.: Earth Environ. Sci.*
- Zona Redaksi, 2026. Trans Sulsel Gantikan Layanan Teman Bus, Lanjutkan Koridor 1, 2 dan 5 [Online]. Dinas Perhubungan Provinsi Sulawesi Selatan. Available at: <https://zonaredaksi.id> [Accessed: 02 Feb 2026].

LAMPIRAN

Lampiran 1 Program Python Data Preparation

```

import pandas as pd
import numpy as np
import os

# =====
# PARAMETER TETAP
# =====

JUMLAH_BUS = 15
KAPASITAS_BUS = 40
TRIP_PER_BUS = 4

JARAK_PERGI = 27.83
JARAK_PULANG = 26.37

BIAYA_PER_KM = 2635

# =====
# LOKASI FILE
# =====

DATA_FOLDER = "data"

HALTE_FILE = os.path.join(DATA_FOLDER, "halte.xlsx")
JADWAL_FILE = os.path.join(DATA_FOLDER, "jadwal.xlsx")
ARMADA_FILE = os.path.join(DATA_FOLDER, "armada.xlsx")

OUTPUT_FILE = os.path.join(DATA_FOLDER, "dataset_operasional.xlsx")

# =====
# MEMBACA DATA
# =====

print("="*60)
print("MEMBACA DATA...")
print("="*60)

halte = pd.read_excel(HALTE_FILE)
jadwal = pd.read_excel(JADWAL_FILE)
armada = pd.read_excel(ARMADA_FILE)

# =====
# MEMBERSIHKAN DATA
# =====

halte = halte.dropna(how='all')
jadwal = jadwal.dropna(how='all')
armada = armada.dropna(how='all')

halte.reset_index(drop=True, inplace=True)

```

```

jadwal.reset_index(drop=True, inplace=True)
armada.reset_index(drop=True, inplace=True)

# =====
# MENGHITUNG PARAMETER OPERASIONAL
# =====

jarak_pp = JARAK_PERGI + JARAK_PULANG

biaya_trip = jarak_pp * BIAYA_PER_KM

biaya_bus_harian = biaya_trip * TRIP_PER_BUS

biaya_total_harian = biaya_bus_harian * JUMLAH_BUS

total_trip = JUMLAH_BUS * TRIP_PER_BUS

# =====
# MEMBUAT DATA PARAMETER
# =====

parameter = pd.DataFrame({

"Parameter":[
"Jumlah Bus",
"Kapasitas Bus",
"Trip per Bus",
"Total Trip Harian",
"Jarak Pergi (km)",
"Jarak Pulang (km)",
"Jarak PP (km)",
"Biaya per km",
"Biaya per Trip",
"Biaya per Bus per Hari",
"Total Biaya Harian"],

"Nilai":[
JUMLAH_BUS,
KAPASITAS_BUS,
TRIP_PER_BUS,
total_trip,
JARAK_PERGI,
JARAK_PULANG,
jarak_pp,
BIAYA_PER_KM,
biaya_trip,
biaya_bus_harian,
biaya_total_harian]})

# =====
# MENYIMPAN KE EXCEL
# =====

```

```

with pd.ExcelWriter(OUTPUT_FILE,
engine='openpyxl') as writer:

    halte.to_excel(writer,
    sheet_name="Halte",
    index=False)

    jadwal.to_excel(writer,
    sheet_name="Jadwal",
    index=False)

    armada.to_excel(writer,
    sheet_name="Armada",
    index=False)

    parameter.to_excel(writer,
    sheet_name="Parameter",
    index=False)

    print()
    print("="*60)
    print("DATA PREPARATION SELESAI")
    print("="*60)

    print(f"Jumlah Bus      : {JUMLAH_BUS}")
    print(f"Kapasitas Bus      : {KAPASITAS_BUS}")
    print(f"Trip per Bus         : {TRIP_PER_BUS}")
    print(f"Total Trip Harian    : {total_trip}")

    print()

    print(f"Jarak PP              : {jarak_pp:.2f} km")
    print(f"Biaya per Trip        : Rp {biaya_trip:,.0f}")
    print(f"Biaya per Bus/Hari    : Rp {biaya_bus_harian:,.0f}")
    print(f"Total Biaya Harian    : Rp {biaya_total_harian:,.0f}")

    print()

    print("Dataset disimpan pada :")
    print(OUTPUT_FILE)

```

Lampiran 2 Program Python Demand Generator

```

import os
from typing import List, Tuple

import numpy as np
import pandas as pd

# =====

```

```

# KONFIGURASI FILE
# =====

DATA_FOLDER = "data"

INPUT_FILE = os.path.join(DATA_FOLDER, "dataset_operasional.xlsx")
JADWAL_FILE = os.path.join(DATA_FOLDER, "jadwal.xlsx")
OUTPUT_FILE = os.path.join(DATA_FOLDER, "demand_generated.xlsx")

JUMLAH_BUS_TERSEDIA = 15

# =====
# KONFIGURASI RANDOM
# =====

RANDOM_SEED = 42

# =====
# RANGE LOAD FACTOR BERDASARKAN KATEGORI
# =====

LOAD_FACTOR_RANGE = {
    "Rush Hour": (0.50, 0.625),
    "Normal": (0.25, 0.50),
    "Non Rush Hour": (0.125, 0.25),}

# =====
# DEFINISI PERIODE OPERASIONAL
# =====

PERIODE_OPERASIONAL = [
    {
        "Periode": "P1",
        "Waktu": "05.00-06.30",
        "Kategori": "Non Rush Hour",
        "Headway_Eksisting": 15,
    },
    {
        "Periode": "P2",
        "Waktu": "06.30-09.00",
        "Kategori": "Rush Hour",
        "Headway_Eksisting": 10,
    },
    {
        "Periode": "P3",
        "Waktu": "09.00-12.00",
        "Kategori": "Normal",
        "Headway_Eksisting": 15,
    },
    {
        "Periode": "P4",
        "Waktu": "12.00-15.00",

```

```

        "Kategori": "Non Rush Hour",
        "Headway_Eksisting": 20,
    },
    {
        "Periode": "P5",
        "Waktu": "15.00-18.00",
        "Kategori": "Rush Hour",
        "Headway_Eksisting": 10,
    },
    {
        "Periode": "P6",
        "Waktu": "18.00-21.00",
        "Kategori": "Normal",
        "Headway_Eksisting": 15,
    },
]
# =====
# FUNGSI BANTUAN -- PARAMETER & LOAD FACTOR
# =====

def get_parameter(parameter_df: pd.DataFrame, parameter_name: str) -> float:

    if "Parameter" not in parameter_df.columns or "Nilai" not in parameter_df.columns:
        raise ValueError("Sheet Parameter harus memiliki kolom 'Parameter' dan 'Nilai'.")

    row = parameter_df.loc[parameter_df["Parameter"] == parameter_name]

    if row.empty:
        raise ValueError(f"Parameter '{parameter_name}' tidak ditemukan.")

    return float(row["Nilai"].iloc[0])

def generate_load_factor(category: str) -> float:

    if category not in LOAD_FACTOR_RANGE:
        raise ValueError(f"Kategori '{category}' tidak terdaftar pada LOAD_FACTOR_RANGE.")

    low, high = LOAD_FACTOR_RANGE[category]
    return float(np.random.uniform(low, high))

# =====
# FUNGSI BANTUAN -- PARSING WAKTU & JADWAL AKTUAL
# =====

def _time_to_minutes(value) -> float:

    if hasattr(value, "hour") and hasattr(value, "minute"):
        return value.hour * 60 + value.minute

    if isinstance(value, str):

```

```

cleaned = value.strip().replace(".", ":")
parts = cleaned.split(":")
hour = int(parts[0])
minute = int(parts[1]) if len(parts) > 1 and parts[1] != "" else 0
return hour * 60 + minute

if isinstance(value, (int, float)) and not isinstance(value, bool):
    text = f"{float(value):.2f}"
    hour_str, minute_str = text.split(":")
    return int(hour_str) * 60 + int(minute_str)

raise ValueError(f"Format waktu tidak dikenali pada jadwal.xlsx: {value!r}")

def _parse_periode_range(waktu_str: str) -> Tuple[int, int]:

    start_str, end_str = waktu_str.split("-")
    return int(_time_to_minutes(start_str)), int(_time_to_minutes(end_str))

def _normalize_bus_ids(jadwal_df: pd.DataFrame) -> pd.DataFrame:

    df = jadwal_df.copy()

    if df["Bus"].max() > JUMLAH_BUS_TERSEDIA:
        print(
            "PERINGATAN: kolom 'Bus' pada jadwal.xlsx bernilai lebih dari "
            f"{JUMLAH_BUS_TERSEDIA} (armada sebenarnya), yaitu sampai "
            f"{int(df['Bus'].max())}. Ini kemungkinan bug penomoran baris "
            "(bukan ID armada fisik yang berulang). Menerapkan koreksi "
            "Bus_asli = ((Bus - 1) % 15) + 1. PERIKSA ULANG apakah hasil "
            "koreksi ini benar-benar merepresentasikan bus fisik yang sama "
            "sebelum dipakai sebagai baseline resmi -- bandingkan dengan "
            "data lapangan/armada.xlsx jika ragu.")
        df["Bus"] = ((df["Bus"] - 1) % JUMLAH_BUS_TERSEDIA) + 1

    return df

def compute_actual_trips_and_buses(jadwal_df: pd.DataFrame) -> pd.DataFrame:

    required_columns = {"Bus", "Berangkat"}
    missing = required_columns - set(jadwal_df.columns)
    if missing:
        raise KeyError(
            f"Kolom {sorted(missing)} tidak ditemukan pada jadwal.xlsx. "
            "Dibutuhkan kolom 'Bus' dan 'Berangkat' untuk menghitung "
            "Jumlah_Trip_Eksisting (q_t) dan Jumlah_Bus_Eksisting (x_t) "
            "per periode.")

    departure_minutes = jadwal_df["Berangkat"].apply(_time_to_minutes)

    records = []
    for item in PERIODE_OPERASIONAL:

```

```

start_minute, end_minute = _parse_periode_range(item["Waktu"])

is_last_period = item is PERIODE_OPERASIONAL[-1]
if is_last_period:
    in_period = (departure_minutes >= start_minute) & (departure_minutes <=
end_minute)
else:
    in_period = (departure_minutes >= start_minute) & (departure_minutes <
end_minute)

trips_in_period = jadwal_df.loc[in_period]

records.append({
    "Periode": item["Periode"],
    "Jumlah_Trip_Eksisting": int(len(trips_in_period)),
    "Jumlah_Bus_Eksisting": int(trips_in_period["Bus"].nunique()),})

result_df = pd.DataFrame(records)

total_trips_terbagi = int(result_df["Jumlah_Trip_Eksisting"].sum())
total_baris_jadwal = len(jadwal_df)

if total_trips_terbagi != total_baris_jadwal:
    print(
        "PERINGATAN: total trip yang berhasil dipetakan ke P1-P5 "
        f"({total_trips_terbagi}) tidak sama dengan total baris "
        f"jadwal.xlsx ({total_baris_jadwal}). Ada keberangkatan yang "
        "jatuh di luar rentang waktu P1-P5 -- periksa kembali "
        "PERIODE_OPERASIONAL atau data jam keberangkatan.")

return result_df

# =====
# PEMBENTUKAN DATASET DEMAND
# =====

def build_demand_dataset(
    jumlah_bus: int,
    kapasitas_bus: int,
    biaya_per_trip: float,
    actual_df: pd.DataFrame = None,) -> pd.DataFrame:

    records = []

    if actual_df is not None:
        actual_lookup = actual_df.set_index("Periode")
    else:
        print(
            "PERINGATAN: jadwal.xlsx tidak tersedia -- Jumlah_Bus_Eksisting "
            "dan Jumlah_Trip_Eksisting DIASUMSIKAN sama dengan parameter "
            "'Jumlah Bus' di setiap periode. Ini TIDAK merepresentasikan "
            "kondisi aktual dan akan membuat biaya eksisting keliru ")

```



```

        "(Sum q_t bisa melebihi 15 bus x 4 trip = 60). Sediakan "
        "jadwal.xlsx agar q_t dan x_t dihitung dari data sebenarnya.")
    actual_lookup = None

    for item in PERIODE_OPERASIONAL:
        category = item["Kategori"]
        lf_random = generate_load_factor(category)

        demand = int(round(lf_random * kapasitas_bus * jumlah_bus))
        kapasitas_maksimum = kapasitas_bus * jumlah_bus

        if actual_lookup is not None:
            periode_row = actual_lookup.loc[item["Periode"]]
            jumlah_trip_eksisting = int(periode_row["Jumlah_Trip_Eksisting"])
            jumlah_bus_eksisting = int(periode_row["Jumlah_Bus_Eksisting"])
        else:
            jumlah_trip_eksisting = jumlah_bus
            jumlah_bus_eksisting = jumlah_bus

        records.append({
            "Periode": item["Periode"],
            "Waktu": item["Waktu"],
            "Kategori": category,
            "LF_Min": LOAD_FACTOR_RANGE[category][0],
            "LF_Max": LOAD_FACTOR_RANGE[category][1],
            "LF_Acak": round(lf_random, 4),
            "Jumlah_Bus_Eksisting": jumlah_bus_eksisting,
            "Jumlah_Trip_Eksisting": jumlah_trip_eksisting,
            "Kapasitas_Bus": kapasitas_bus,
            "Kapasitas_Maksimum": kapasitas_maksimum,
            "Demand": demand,
            "Headway_Eksisting": item["Headway_Eksisting"],
            "Biaya_per_Trip": biaya_per_trip,})

    return pd.DataFrame(records)

# =====
# PROGRAM UTAMA
# =====

def main() -> None:
    print("=" * 70)
    print("02 DEMAND GENERATOR (REVISI)")
    print("=" * 70)

    if RANDOM_SEED is not None:
        np.random.seed(RANDOM_SEED)

    if not os.path.exists(INPUT_FILE):
        raise FileNotFoundError(
            f"File input tidak ditemukan: {INPUT_FILE}\n"
            "Jalankan 01_data_preparation.py terlebih dahulu.")

```

```

print(f"Membaca file input: {INPUT_FILE}")

parameter_df = pd.read_excel(INPUT_FILE, sheet_name="Parameter")

jumlah_bus = int(get_parameter(parameter_df, "Jumlah Bus"))
kapasitas_bus = int(get_parameter(parameter_df, "Kapasitas Bus"))
biaya_per_trip = get_parameter(parameter_df, "Biaya per Trip")

actual_df = None
if os.path.exists(JADWAL_FILE):
    print(f"Membaca jadwal aktual: {JADWAL_FILE}")
    jadwal_df = pd.read_excel(JADWAL_FILE)
    jadwal_df = _normalize_bus_ids(jadwal_df)
    actual_df = compute_actual_trips_and_buses(jadwal_df)

    print()
    print("x_t (bus aktif) dan q_t (trip PP) aktual per periode:")
    print(actual_df.to_string(index=False))
    print(f"Total trip (Sum q_t) : {actual_df['Jumlah_Trip_Eksisting'].sum()}")
else:
    print(f"File jadwal tidak ditemukan: {JADWAL_FILE}")

demand_df = build_demand_dataset(
    jumlah_bus=jumlah_bus,
    kapasitas_bus=kapasitas_bus,
    biaya_per_trip=biaya_per_trip,
    actual_df=actual_df,)

summary_df = pd.DataFrame({
    "Keterangan": [
        "Jumlah Bus Tersedia (parameter)",
        "Kapasitas per Bus",
        "Kapasitas Maksimum per Periode",
        "Jumlah Periode",
        "Total Demand Harian",
        "Rata-rata Load Factor Bangkitan",
        "Total Trip Eksisting (Sum q_t)."],
    "Nilai": [
        jumlah_bus,
        kapasitas_bus,
        jumlah_bus * kapasitas_bus,
        len(demand_df),
        int(demand_df["Demand"].sum()),
        round(float(demand_df["LF_Acak"].mean()), 4),
        int(demand_df["Jumlah_Trip_Eksisting"].sum()),.]})

os.makedirs(DATA_FOLDER, exist_ok=True)

with pd.ExcelWriter(OUTPUT_FILE, engine="openpyxl") as writer:
    demand_df.to_excel(writer, sheet_name="Demand", index=False)
    summary_df.to_excel(writer, sheet_name="Ringkasan", index=False)

```

```

print()
print("Demand berhasil dibangkitkan.")
print(demand_df[["Periode", "Waktu", "Kategori", "LF_Acak", "Demand",
    "Jumlah_Bus_Eksisting", "Jumlah_Trip_Eksisting"]])
print()
print(f"Output disimpan pada: {OUTPUT_FILE}")
print("=" * 70)

if __name__ == "__main__":
    main()

```

Lampiran 3 Program Python Objective Function

```

import os
from typing import Dict, List
import pandas as pd

# =====
# KONFIGURASI FILE
# =====

DATA_FOLDER = "data"

DEMAND_FILE = os.path.join(DATA_FOLDER, "demand_generated.xlsx")
OUTPUT_FILE = os.path.join(DATA_FOLDER, "objective_function_test.xlsx")

JUMLAH_BUS_TERSEDIA = 15
TRIP_PER_BUS_PER_HARI = 4
TOTAL_TRIP_HARIAN_MAKS = JUMLAH_BUS_TERSEDIA *
TRIP_PER_BUS_PER_HARI # = 60

WEIGHT_COST = 1.0
WEIGHT_WAITING_TIME = 1.0
WEIGHT_EMPTY_SEAT = 1.0

PENALTY_PER_SHORTAGE_PASSENGER = 1000.0

# =====
# MEMBACA DATA DEMAND
# =====

def load_demand_data(file_path: str = DEMAND_FILE) -> pd.DataFrame:

    if not os.path.exists(file_path):
        raise FileNotFoundError(
            f"File demand tidak ditemukan: {file_path}\n"
            "Jalankan 02_demand_generator.py terlebih dahulu.")

    demand_df = pd.read_excel(file_path, sheet_name="Demand")

```

```

if "Jumlah_Trip_Eksisting" not in demand_df.columns:
    raise KeyError(
        "Kolom 'Jumlah_Trip_Eksisting' tidak ditemukan pada
demand_generated.xlsx.\n"
        "Kolom ini wajib ada (jumlah trip PP aktual per periode, dari jadwal.xlsx) "
        "agar biaya operasional dihitung dari q_t, bukan dari x_t. "
        "Perbarui 02_demand_generator.py terlebih dahulu.")

return demand_df

# =====
# KONSTANTA NORMALISASI
# =====

def _compute_normalization_constants(demand_df: pd.DataFrame) -> Dict[str, float]:

    baseline_cost = float(
        (demand_df["Biaya_per_Trip"] * demand_df["Jumlah_Trip_Eksisting"]).sum())
    baseline_waiting_time = float((demand_df["Headway_Eksisting"] / 2).sum())
    baseline_capacity = float(demand_df["Kapasitas_Maksimum"].sum())

    return {
        "cost": max(baseline_cost, 1.0),
        "waiting_time": max(baseline_waiting_time, 1.0),
        "empty_seat": max(baseline_capacity, 1.0),}

# =====
# EVALUASI SOLUSI
# =====

def evaluate_solution(
    active_buses: List[int],
    headways: List[int],
    demand_df: pd.DataFrame,
    trips: List[int] = None,) -> Dict[str, float]:

    number_of_periods = len(demand_df)

    if trips is None:
        # Kompatibilitas sementara -- sebaiknya selalu isi `trips` eksplisit.
        trips = list(active_buses)

    if (
        len(active_buses) != number_of_periods
        or len(headways) != number_of_periods
        or len(trips) != number_of_periods):
        raise ValueError(
            "Panjang active_buses, headways, dan trips harus sama dengan jumlah
periode.")

    total_trips = sum(trips)

```

```

if total_trips > TOTAL_TRIP_HARIAN_MAKS:
    raise ValueError(
        f"Total trip (Sum q_t = {total_trips}) melebihi kapasitas armada harian "
        f"({JUMLAH_BUS_TERSEDIA} bus x {TRIP_PER_BUS_PER_HARI} trip = {TOTAL_TRIP_HARIAN_MAKS}).")

norm = _compute_normalization_constants(demand_df)

total_cost = 0.0
total_waiting_time = 0.0
total_empty_seat = 0.0
total_shortage = 0.0
total_capacity = 0.0
total_demand_served = 0.0
total_demand = 0.0

for idx in range(number_of_periods):
    bus = active_buses[idx]
    trip = trips[idx]
    headway = headways[idx]

    demand = float(demand_df["Demand"].iloc[idx])
    capacity_bus = float(demand_df["Kapasitas_Bus"].iloc[idx])
    cost_per_trip = float(demand_df["Biaya_per_Trip"].iloc[idx])

    capacity_period = trip * capacity_bus
    empty_seat = max(0.0, capacity_period - demand)
    shortage = max(0.0, demand - capacity_period)
    demand_served = min(demand, capacity_period)

    total_cost += cost_per_trip * trip

    total_waiting_time += headway / 2
    total_empty_seat += empty_seat
    total_shortage += shortage
    total_capacity += capacity_period
    total_demand_served += demand_served
    total_demand += demand

penalty = total_shortage * PENALTY_PER_SHORTAGE_PASSENGER

objective_value = (
    WEIGHT_COST * (total_cost / norm["cost"])
    + WEIGHT_WAITING_TIME * (total_waiting_time / norm["waiting_time"])
    + WEIGHT_EMPTY_SEAT * (total_empty_seat / norm["empty_seat"])
    + penalty)

fitness = 1.0 / (1.0 + objective_value)

load_factor = (
    total_demand_served / total_capacity if total_capacity > 0 else 0.0)

```

```

existing_bus_baseline = float(demand_df["Jumlah_Bus_Eksisting"].iloc[0])
fleet_utilization = (
    sum(active_buses) / (number_of_periods * existing_bus_baseline)
    if existing_bus_baseline > 0
    else 0.0)

return {
    "objective_value": objective_value,
    "fitness": fitness,
    "operational_cost": total_cost,
    "waiting_time": total_waiting_time,
    "empty_seat": total_empty_seat,
    "shortage": total_shortage,
    "load_factor": load_factor,
    "fleet_utilization": fleet_utilization,
    "penalty": penalty,
    "total_demand": total_demand,
    "total_capacity": total_capacity,
    "total_trips": total_trips,}

# =====
# DETAIL PER PERIODE
# =====

def build_period_detail(
    active_buses: List[int],
    headways: List[int],
    demand_df: pd.DataFrame,
    trips: List[int] = None,) -> pd.DataFrame:

    if trips is None:
        trips = list(active_buses)

    records = []

    for idx in range(len(demand_df)):
        bus = active_buses[idx]
        trip = trips[idx]
        headway = headways[idx]

        demand = float(demand_df["Demand"].iloc[idx])
        capacity_bus = float(demand_df["Kapasitas_Bus"].iloc[idx])
        cost_per_trip = float(demand_df["Biaya_per_Trip"].iloc[idx])
        capacity_period = trip * capacity_bus

        empty_seat = max(0.0, capacity_period - demand)
        shortage = max(0.0, demand - capacity_period)
        load_factor = (
            min(demand, capacity_period) / capacity_period
            if capacity_period > 0
            else 0.0)

```

```

records.append({
    "Periode": demand_df["Periode"].iloc[idx],
    "Waktu": demand_df["Waktu"].iloc[idx],
    "Kategori": demand_df["Kategori"].iloc[idx],
    "Demand": demand,
    "Bus_Aktif": bus,
    "Trip_PP": trip,
    "Headway": headway,
    "Kapasitas_Periode": capacity_period,
    "Load_Factor": load_factor,
    "Empty_Seat": empty_seat,
    "Shortage": shortage,
    "Biaya_Periode": cost_per_trip * trip,})

return pd.DataFrame(records)

# =====
# EVALUASI KONDISI EKSISTING (BASELINE)
# =====

def evaluate_existing(demand_df: pd.DataFrame) -> Dict[str, object]:

    active_buses = demand_df["Jumlah_Bus_Eksisting"].astype(int).tolist()
    trips = demand_df["Jumlah_Trip_Eksisting"].astype(int).tolist()
    headways = demand_df["Headway_Eksisting"].astype(int).tolist()

    evaluation = evaluate_solution(active_buses, headways, demand_df, trips=trips)
    detail_df = build_period_detail(active_buses, headways, demand_df, trips=trips)

    return {
        "active_buses": active_buses,
        "trips": trips,
        "headways": headways,
        "evaluation": evaluation,
        "detail_df": detail_df,}

# =====
# SIMPAN HASIL BASELINE
# =====

def save_existing_results(
    result: Dict[str, object],
    output_file: str = OUTPUT_FILE,) -> None:

    os.makedirs(DATA_FOLDER, exist_ok=True)

    evaluation = result["evaluation"]

    indicator_keys = [
        "objective_value",
        "fitness",
        "operational_cost",

```

```

"waiting_time",
"empty_seat",
"shortage",
"load_factor",
"fleet_utilization",
"penalty",
"total_demand",
"total_capacity",
"total_trips",]

summary_df = pd.DataFrame({
    "Indikator": indicator_keys + ["Computation Time (seconds)"],
    "Nilai": [evaluation[key] for key in indicator_keys] + [0.0],})

n_periods = len(result["active_buses"])
best_solution_df = pd.DataFrame({
    "Komponen": (
        [f"Bus_P{i+1}" for i in range(n_periods)]
        + [f"Trip_P{i+1}" for i in range(n_periods)]
        + [f"Headway_P{i+1}" for i in range(n_periods)]),
    "Nilai": result["active_buses"] + result["trips"] + result["headways"],})

with pd.ExcelWriter(output_file, engine="openpyxl") as writer:
    best_solution_df.to_excel(writer, sheet_name="Best_Solution", index=False)
    result["detail_df"].to_excel(writer, sheet_name="Detail_Per_Periode",
index=False)
    summary_df.to_excel(writer, sheet_name="Ringkasan_Evaluasi", index=False)

# =====
# PROGRAM UTAMA
# =====

def main() -> None:
    print("=" * 70)
    print("OBJECTIVE FUNCTION (REVISI) - EVALUASI KONDISI EKSISTING
(BASELINE)")
    print("=" * 70)

    demand_df = load_demand_data()

    print("Data demand berhasil dibaca.")
    print(demand_df[["Periode", "Kategori", "Demand", "Jumlah_Trip_Eksisting",
"Headway_Eksisting"]])
    print()

    result = evaluate_existing(demand_df)
    save_existing_results(result)

    print("Hasil evaluasi kondisi eksisting:")
    print("Bus aktif (x_t) :", result["active_buses"])
    print("Trip PP (q_t) :", result["trips"], " -> total:", sum(result["trips"]))
    print("Headway      :", result["headways"])

```



```

print()

for key, value in result["evaluation"].items():
    print(f"{key:20s}: {value:,.4f}")

print()
print(f"Output disimpan pada: {OUTPUT_FILE}")
print("=" * 70)

if __name__ == "__main__":
    main()

```

Lampiran 4 Program Python Genetic Algorithm

```

import os
import time
from typing import Dict, List, Tuple

import numpy as np
import pandas as pd

from objective_function import (
    load_demand_data,
    evaluate_solution,
    build_period_detail,
    TOTAL_TRIP_HARIAN_MAKS,)

# =====
# KONFIGURASI FILE
# =====

DATA_FOLDER = "data"
OUTPUT_FILE = os.path.join(DATA_FOLDER, "ga_result.xlsx")

# =====
# PARAMETER GENETIC ALGORITHM
# =====

RANDOM_SEED = 123

POPULATION_SIZE = 100
MAX_GENERATIONS = 300

CROSSOVER_RATE = 0.80
MUTATION_RATE = 0.10

TOURNAMENT_SIZE = 3
ELITE_SIZE = 2

# =====
# BATAS KEPUTUSAN

```

```

# =====

MIN_BUS = 1
MAX_BUS = 15

HEADWAY_OPTIONS = [10, 12, 15, 18, 20, 25]

# =====
# FUNGSI PEMBENTUKAN INDIVIDU
# =====

def create_individual(number_of_periods: int) -> np.ndarray:

    buses = np.random.randint(MIN_BUS, MAX_BUS + 1, size=number_of_periods)
    # trip_t dibangkitkan di rentang [1, bus_t] agar valid sejak awal
    # (satu bus paling banyak 1 trip PP per periode).
    trips = np.array([np.random.randint(1, bus + 1) for bus in buses])
    headways = np.random.choice(HEADWAY_OPTIONS, size=number_of_periods)

    individual = np.concatenate([buses, trips, headways]).astype(int)
    return repair_individual(individual, number_of_periods)

def split_individual(
    individual: np.ndarray, number_of_periods: int) -> Tuple[List[int], List[int], List[int]]:

    m = number_of_periods
    active_buses = individual[0:m].astype(int).tolist()
    trips = individual[m:2 * m].astype(int).tolist()
    headways = individual[2 * m:3 * m].astype(int).tolist()

    return active_buses, trips, headways

def create_population(population_size: int, number_of_periods: int) -> np.ndarray:

    return np.array([
        create_individual(number_of_periods)
        for _ in range(population_size)])

# =====
# FUNGSI EVALUASI POPULASI
# =====

def evaluate_individual(individual: np.ndarray, demand_df: pd.DataFrame) -> Dict[str, float]:

    number_of_periods = len(demand_df)
    active_buses, trips, headways = split_individual(individual, number_of_periods)

    return evaluate_solution(
        active_buses=active_buses,
        headways=headways,
        demand_df=demand_df,

```

```

trips=trips,)

def evaluate_population(population: np.ndarray, demand_df: pd.DataFrame) ->
pd.DataFrame:

    records = []

    for idx, individual in enumerate(population):
        evaluation = evaluate_individual(individual, demand_df)

        records.append({
            "index": idx,
            "objective_value": evaluation["objective_value"],
            "fitness": evaluation["fitness"],
            "operational_cost": evaluation["operational_cost"],
            "waiting_time": evaluation["waiting_time"],
            "empty_seat": evaluation["empty_seat"],
            "shortage": evaluation["shortage"],
            "load_factor": evaluation["load_factor"],
            "fleet_utilization": evaluation["fleet_utilization"],
            "penalty": evaluation["penalty"],
            "total_trips": evaluation["total_trips"],})

    return pd.DataFrame(records)

# =====
# OPERATOR GENETIC ALGORITHM
# =====

def tournament_selection(population: np.ndarray, fitness_values: np.ndarray) ->
np.ndarray:

    candidate_indices = np.random.choice(
        len(population),
        size=TOURNAMENT_SIZE,
        replace=False,)

    best_candidate_index = candidate_indices[
        np.argmax(fitness_values[candidate_indices])]

    return population[best_candidate_index].copy()

def crossover(parent1: np.ndarray, parent2: np.ndarray) -> Tuple[np.ndarray,
np.ndarray]:

    if np.random.rand() > Crossover_RATE:
        return parent1.copy(), parent2.copy()

    chromosome_length = len(parent1)

    # Titik crossover tidak boleh 0 atau panjang kromosom.
    point = np.random.randint(1, chromosome_length)

```

```

child1 = np.concatenate([parent1[:point], parent2[point:]])
child2 = np.concatenate([parent2[:point], parent1[point:]])

return child1.astype(int), child2.astype(int)

def mutate(individual: np.ndarray, number_of_periods: int) -> np.ndarray:

    mutated = individual.copy()
    m = number_of_periods

    for i in range(len(mutated)):
        if np.random.rand() < MUTATION_RATE:
            if i < m:
                mutated[i] = np.random.randint(MIN_BUS, MAX_BUS + 1)
            elif i < 2 * m:
                mutated[i] = np.random.randint(1, MAX_BUS + 1)
            else:
                mutated[i] = np.random.choice(HEADWAY_OPTIONS)

    return mutated.astype(int)

def _repair_trip_budget(trips: np.ndarray) -> np.ndarray:

    repaired = trips.copy()

    while repaired.sum() > TOTAL_TRIP_HARIAN_MAKS:
        reducible_idx = np.where(repaired > 1)[0]
        if len(reducible_idx) == 0:
            # Semua trip sudah minimum (1) tapi total masih > 60 --
            # ini hanya bisa terjadi jika jumlah periode > 60, jadi aman
            # untuk kasus penelitian ini (5 periode).
            break
        idx_to_reduce = reducible_idx[np.argmax(repaired[reducible_idx])]
        repaired[idx_to_reduce] -= 1

    return repaired

def repair_individual(individual: np.ndarray, number_of_periods: int) -> np.ndarray:

    repaired = individual.copy()
    m = number_of_periods

    repaired[0:m] = np.clip(repaired[0:m], MIN_BUS, MAX_BUS)

    buses = repaired[0:m]
    trips = repaired[m:2 * m]
    trips = np.clip(trips, 1, buses)
    trips = _repair_trip_budget(trips)
    repaired[m:2 * m] = trips

    for i in range(2 * m, 3 * m):

```

```

        if repaired[i] not in HEADWAY_OPTIONS:
            repaired[i] = min(HEADWAY_OPTIONS, key=lambda x: abs(x - repaired[i]))

    return repaired.astype(int)

def get_elites(population: np.ndarray, evaluation_df: pd.DataFrame) -> np.ndarray:

    elite_indices = evaluation_df.sort_values(
        by="fitness",
        ascending=False,
    )["index"].head(ELITE_SIZE).to_numpy()

    return population[elite_indices].copy()

# =====
# ALGORITMA UTAMA
# =====

def run_genetic_algorithm(demand_df: pd.DataFrame) -> Dict[str, object]:

    number_of_periods = len(demand_df)

    population = create_population(
        population_size=POPULATION_SIZE,
        number_of_periods=number_of_periods,)

    history = []
    best_individual = None
    best_evaluation = None

    start_time = time.time()

    for generation in range(1, MAX_GENERATIONS + 1):
        evaluation_df = evaluate_population(population, demand_df)

        best_row = evaluation_df.sort_values(
            by="fitness",
            ascending=False,).iloc[0]

        current_best_individual = population[int(best_row["index"])].copy()
        current_best_evaluation = evaluate_individual(
            current_best_individual,
            demand_df,)

        if best_evaluation is None or current_best_evaluation["fitness"] >
best_evaluation["fitness"]:
            best_individual = current_best_individual.copy()
            best_evaluation = current_best_evaluation.copy()

        history.append({
            "generation": generation,
            "best_fitness": best_evaluation["fitness"],

```

```

    "best_objective_value": best_evaluation["objective_value"],
    "current_best_fitness": current_best_evaluation["fitness"],
    "current_best_objective_value": current_best_evaluation["objective_value"],
    "mean_fitness": evaluation_df["fitness"].mean(),
    "mean_objective_value": evaluation_df["objective_value"].mean(),})

elites = get_elites(population, evaluation_df)
fitness_values = evaluation_df["fitness"].to_numpy()

new_population = []

for elite in elites:
    new_population.append(elite)

while len(new_population) < POPULATION_SIZE:
    parent1 = tournament_selection(population, fitness_values)
    parent2 = tournament_selection(population, fitness_values)

    child1, child2 = crossover(parent1, parent2)

    child1 = mutate(child1, number_of_periods)
    child2 = mutate(child2, number_of_periods)

    child1 = repair_individual(child1, number_of_periods)
    child2 = repair_individual(child2, number_of_periods)

    new_population.append(child1)

    if len(new_population) < POPULATION_SIZE:
        new_population.append(child2)

population = np.array(new_population)

computation_time = time.time() - start_time

active_buses, trips, headways = split_individual(
    best_individual,
    number_of_periods,)

detail_df = build_period_detail(
    active_buses=active_buses,
    headways=headways,
    demand_df=demand_df,
    trips=trips,)

history_df = pd.DataFrame(history)

summary_df = pd.DataFrame({
    "Indikator": [
        "objective_value",
        "fitness",
        "operational_cost",

```

```

        "waiting_time",
        "empty_seat",
        "shortage",
        "load_factor",
        "fleet_utilization",
        "penalty",
        "total_demand",
        "total_capacity",
        "total_trips",
        "Computation Time (seconds)",],
    "Nilai": [
        best_evaluation["objective_value"],
        best_evaluation["fitness"],
        best_evaluation["operational_cost"],
        best_evaluation["waiting_time"],
        best_evaluation["empty_seat"],
        best_evaluation["shortage"],
        best_evaluation["load_factor"],
        best_evaluation["fleet_utilization"],
        best_evaluation["penalty"],
        best_evaluation["total_demand"],
        best_evaluation["total_capacity"],
        best_evaluation["total_trips"],
        computation_time,],})

return {
    "best_individual": best_individual,
    "active_buses": active_buses,
    "trips": trips,
    "headways": headways,
    "best_evaluation": best_evaluation,
    "detail_df": detail_df,
    "history_df": history_df,
    "summary_df": summary_df,
    "computation_time": computation_time,}

# =====
# SIMPAN HASIL
# =====

def save_results(results: Dict[str, object], output_file: str = OUTPUT_FILE) -> None:

    os.makedirs(DATA_FOLDER, exist_ok=True)

    n = len(results["active_buses"])
    best_solution_df = pd.DataFrame({
        "Komponen": (
            [f"Bus_P{i+1}" for i in range(n)]
            + [f"Trip_P{i+1}" for i in range(n)]
            + [f"Headway_P{i+1}" for i in range(n)]),
        "Nilai": results["active_buses"] + results["trips"] + results["headways"],})

```

```

with pd.ExcelWriter(output_file, engine="openpyxl") as writer:
    best_solution_df.to_excel(
        writer,
        sheet_name="Best_Solution",
        index=False,)

    results["detail_df"].to_excel(
        writer,
        sheet_name="Detail_Per_Periode",
        index=False,)

    results["summary_df"].to_excel(
        writer,
        sheet_name="Ringkasan_Evaluasi",
        index=False,)

    results["history_df"].to_excel(
        writer,
        sheet_name="Konvergensi_GA",
        index=False,)

# =====
# PROGRAM UTAMA
# =====

def main() -> None:
    print("=" * 70)
    print("04 GENETIC ALGORITHM (REVISI)")
    print("=" * 70)

    if RANDOM_SEED is not None:
        np.random.seed(RANDOM_SEED)

    demand_df = load_demand_data()

    print("Data demand berhasil dibaca.")
    print(demand_df[["Periode", "Kategori", "Demand", "Headway_Eksisting"]])
    print()

    results = run_genetic_algorithm(demand_df)
    save_results(results)

    print("Hasil terbaik Genetic Algorithm:")
    print("Bus aktif (x_t) :", results["active_buses"])
    print("Trip PP (q_t) :", results["trips"], " -> total:", sum(results["trips"]))
    print("Headway      :", results["headways"])
    print()

    for key, value in results["best_evaluation"].items():
        print(f'{key:20s}: {value:,.4f}')

    print(f'computation_time : {results["computation_time"]:,.4f} seconds')

```



```

print()
print(f"Output disimpan pada: {OUTPUT_FILE}")
print("=" * 70)

if __name__ == "__main__":
    main()

```

Lampiran 5 Program Python Ant Colony Optimization

```

import os
import time
import importlib.util
from typing import Dict, List, Tuple
import numpy as np
import pandas as pd

# =====
# MEMUAT MODUL OBJECTIVE FUNCTION
# =====

try:
    BASE_DIR = os.path.dirname(os.path.abspath(__file__))
except NameError:
    BASE_DIR = os.getcwd()

OBJECTIVE_FUNCTION_PATH = os.path.join(BASE_DIR, "objective_function.py")

if not os.path.exists(OBJECTIVE_FUNCTION_PATH):
    OBJECTIVE_FUNCTION_PATH = "objective_function.py"

spec = importlib.util.spec_from_file_location(
    "objective_function_module",
    OBJECTIVE_FUNCTION_PATH,)
objective_function_module = importlib.util.module_from_spec(spec)
spec.loader.exec_module(objective_function_module)

load_demand_data = objective_function_module.load_demand_data
evaluate_solution = objective_function_module.evaluate_solution
build_period_detail = objective_function_module.build_period_detail
TOTAL_TRIP_HARIAN_MAKS =
objective_function_module.TOTAL_TRIP_HARIAN_MAKS

# =====
# KONFIGURASI FILE
# =====

DATA_FOLDER = "data"
OUTPUT_FILE = os.path.join(DATA_FOLDER, "aco_result.xlsx")

# =====
# PARAMETER ANT COLONY OPTIMIZATION

```

```

# =====

RANDOM_SEED = 456

NUM_ANTS = 80
MAX_ITERATIONS = 300

ALPHA = 1.0    # pengaruh pheromone
BETA = 2.0     # pengaruh heuristic information
RHO = 0.50     # evaporation rate
Q = 1.0        # konstanta pheromone deposit
TAU0 = 1.0     # pheromone awal

# =====
# BATAS KEPUTUSAN
# =====

MIN_BUS = 1
MAX_BUS = 15

HEADWAY_OPTIONS = [10, 12, 15, 18, 20, 25]

# =====
# RUANG SOLUSI
# =====

def build_decision_options() -> List[Tuple[int, int, int]]:

    options = []

    for bus in range(MIN_BUS, MAX_BUS + 1):
        for trip in range(1, bus + 1):
            for headway in HEADWAY_OPTIONS:
                options.append((bus, trip, headway))

    return options

# =====
# HEURISTIC INFORMATION
# =====

def calculate_local_heuristic(
    bus: int,
    trip: int,
    headway: int,
    demand: float,
    capacity_bus: float,
    cost_per_trip: float,) -> float:

    service_capacity = trip * capacity_bus

    local_cost = cost_per_trip * trip

```

```

local_waiting_time = headway / 2
local_empty_seat = max(0, service_capacity - demand)
local_shortage = max(0, demand - service_capacity)

local_score = (
    (local_cost / 8_568_000)
    + (local_waiting_time / 100)
    + (local_empty_seat / 2400)
    + (local_shortage * 1000))

return 1.0 / (1.0 + local_score)

def build_heuristic_matrix(
    demand_df: pd.DataFrame,
    decision_options: List[Tuple[int, int, int]],) -> np.ndarray:

    number_of_periods = len(demand_df)
    number_of_options = len(decision_options)

    heuristic = np.zeros((number_of_periods, number_of_options), dtype=float)

    capacity_bus = float(demand_df["Kapasitas_Bus"].iloc[0])
    cost_per_trip = float(demand_df["Biaya_per_Trip"].iloc[0])

    for period_idx in range(number_of_periods):
        demand = float(demand_df["Demand"].iloc[period_idx])

        for option_idx, (bus, trip, headway) in enumerate(decision_options):
            heuristic[period_idx, option_idx] = calculate_local_heuristic(
                bus=bus,
                trip=trip,
                headway=headway,
                demand=demand,
                capacity_bus=capacity_bus,
                cost_per_trip=cost_per_trip,)

    return heuristic

# =====
# PEMBENTUKAN SOLUSI OLEH SEMUT
# =====

def choose_option(
    pheromone_row: np.ndarray,
    heuristic_row: np.ndarray,) -> int:

    numerator = (pheromone_row ** ALPHA) * (heuristic_row ** BETA)
    total = np.sum(numerator)

    if total <= 0 or np.isnan(total):
        probabilities = np.ones_like(numerator) / len(numerator)
    else:

```

```

        probabilities = numerator / total

    return int(np.random.choice(len(probabilities), p=probabilities))

def _repair_trip_budget(trips: List[int], buses: List[int]) -> List[int]:

    repaired = list(trips)

    while sum(repaired) > TOTAL_TRIP_HARIAN_MAKS:
        reducible_idx = [i for i, v in enumerate(repaired) if v > 1]
        if not reducible_idx:
            break
        idx_to_reduce = max(reducible_idx, key=lambda i: repaired[i])
        repaired[idx_to_reduce] -= 1

    return repaired

def construct_solution(
    pheromone: np.ndarray,
    heuristic: np.ndarray,
    decision_options: List[Tuple[int, int, int]],) -> Tuple[List[int], List[int], List[int],
List[int]]:

    number_of_periods = pheromone.shape[0]

    active_buses = []
    trips = []
    headways = []
    chosen_indices = []

    for period_idx in range(number_of_periods):
        option_idx = choose_option(
            pheromone_row=pheromone[period_idx],
            heuristic_row=heuristic[period_idx],)

        bus, trip, headway = decision_options[option_idx]

        active_buses.append(bus)
        trips.append(trip)
        headways.append(headway)
        chosen_indices.append(option_idx)

    trips = _repair_trip_budget(trips, active_buses)

    return active_buses, trips, headways, chosen_indices

# =====
# UPDATE PHEROMONE
# =====

def update_pheromone(
    pheromone: np.ndarray,

```

```

ant_solutions: List[Dict[str, object]],) -> np.ndarray:

pheromone = (1.0 - RHO) * pheromone

best_solution = min(
    ant_solutions,
    key=lambda item: item["evaluation"]["objective_value"],)

best_objective = best_solution["evaluation"]["objective_value"]

deposit_amount = Q / (1.0 + best_objective)

for period_idx, option_idx in enumerate(best_solution["chosen_indices"]):
    pheromone[period_idx, option_idx] += deposit_amount

pheromone = np.maximum(pheromone, 1e-12)

return pheromone

# =====
# ALGORITMA UTAMA
# =====

def run_ant_colony_optimization(demand_df: pd.DataFrame) -> Dict[str, object]:

    decision_options = build_decision_options()

    number_of_periods = len(demand_df)
    number_of_options = len(decision_options)

    heuristic = build_heuristic_matrix(
        demand_df=demand_df,
        decision_options=decision_options,)

    pheromone = np.full(
        shape=(number_of_periods, number_of_options),
        fill_value=TAU0,
        dtype=float,)

    best_active_buses = None
    best_trips = None
    best_headways = None
    best_chosen_indices = None
    best_evaluation = None

    history = []

    start_time = time.time()

    for iteration in range(1, MAX_ITERATIONS + 1):
        ant_solutions = []

```

```

for _ in range(NUM_ANTS):
    active_buses, trips, headways, chosen_indices = construct_solution(
        pheromone=pheromone,
        heuristic=heuristic,
        decision_options=decision_options,)

    evaluation = evaluate_solution(
        active_buses=active_buses,
        headways=headways,
        demand_df=demand_df,
        trips=trips,)

    ant_solutions.append({
        "active_buses": active_buses,
        "trips": trips,
        "headways": headways,
        "chosen_indices": chosen_indices,
        "evaluation": evaluation,})

iteration_best = min(
    ant_solutions,
    key=lambda item: item["evaluation"]["objective_value"],)

if (
    best_evaluation is None
    or iteration_best["evaluation"]["objective_value"] <
best_evaluation["objective_value"]):
    best_active_buses = iteration_best["active_buses"]
    best_trips = iteration_best["trips"]
    best_headways = iteration_best["headways"]
    best_chosen_indices = iteration_best["chosen_indices"]
    best_evaluation = iteration_best["evaluation"].copy()

pheromone = update_pheromone(
    pheromone=pheromone,
    ant_solutions=ant_solutions,)

objective_values = [
    item["evaluation"]["objective_value"]
    for item in ant_solutions]

fitness_values = [
    item["evaluation"]["fitness"]
    for item in ant_solutions]

history.append({
    "iteration": iteration,
    "best_objective_value": best_evaluation["objective_value"],
    "best_fitness": best_evaluation["fitness"],
    "current_best_objective_value":
iteration_best["evaluation"]["objective_value"],
    "current_best_fitness": iteration_best["evaluation"]["fitness"],

```

```

        "mean_objective_value": float(np.mean(objective_values)),
        "mean_fitness": float(np.mean(fitness_values)),})

computation_time = time.time() - start_time

detail_df = build_period_detail(
    active_buses=best_active_buses,
    headways=best_headways,
    demand_df=demand_df,
    trips=best_trips,)

history_df = pd.DataFrame(history)

summary_df = pd.DataFrame({
    "Indikator": [
        "objective_value",
        "fitness",
        "operational_cost",
        "waiting_time",
        "empty_seat",
        "shortage",
        "load_factor",
        "fleet_utilization",
        "penalty",
        "total_demand",
        "total_capacity",
        "total_trips",
        "Computation Time (seconds)",],
    "Nilai": [
        best_evaluation["objective_value"],
        best_evaluation["fitness"],
        best_evaluation["operational_cost"],
        best_evaluation["waiting_time"],
        best_evaluation["empty_seat"],
        best_evaluation["shortage"],
        best_evaluation["load_factor"],
        best_evaluation["fleet_utilization"],
        best_evaluation["penalty"],
        best_evaluation["total_demand"],
        best_evaluation["total_capacity"],
        best_evaluation["total_trips"],
        computation_time,]})

pheromone_records = []

for period_idx in range(number_of_periods):
    for option_idx, (bus, trip, headway) in enumerate(decision_options):
        pheromone_records.append({
            "Periode": demand_df["Periode"].iloc[period_idx],
            "Option_Index": option_idx,
            "Bus": bus,
            "Trip": trip,

```

```

        "Headway": headway,
        "Pheromone": pheromone[period_idx, option_idx],
        "Heuristic": heuristic[period_idx, option_idx],})

pheromone_df = pd.DataFrame(pheromone_records)

return {
    "active_buses": best_active_buses,
    "trips": best_trips,
    "headways": best_headways,
    "chosen_indices": best_chosen_indices,
    "best_evaluation": best_evaluation,
    "detail_df": detail_df,
    "history_df": history_df,
    "summary_df": summary_df,
    "pheromone_df": pheromone_df,
    "computation_time": computation_time,}

# =====
# SIMPAN HASIL
# =====

def save_results(results: Dict[str, object], output_file: str = OUTPUT_FILE) -> None:

    os.makedirs(DATA_FOLDER, exist_ok=True)

    n = len(results["active_buses"])
    best_solution_df = pd.DataFrame({
        "Komponen": (
            [f"Bus_P{i+1}" for i in range(n)]
            + [f"Trip_P{i+1}" for i in range(n)]
            + [f"Headway_P{i+1}" for i in range(n)]),
        "Nilai": results["active_buses"] + results["trips"] + results["headways"],})

    with pd.ExcelWriter(output_file, engine="openpyxl") as writer:
        best_solution_df.to_excel(
            writer,
            sheet_name="Best_Solution",
            index=False,)

        results["detail_df"].to_excel(
            writer,
            sheet_name="Detail_Per_Periode",
            index=False,)

        results["summary_df"].to_excel(
            writer,
            sheet_name="Ringkasan_Evaluasi",
            index=False,)

        results["history_df"].to_excel(
            writer,

```



```

    sheet_name="Konvergensi_ACO",
    index=False,)

    results["pheromone_df"].to_excel(
        writer,
        sheet_name="Pheromone_Final",
        index=False,)

# =====
# PROGRAM UTAMA
# =====

def main() -> None:
    print("=" * 70)
    print("05 ANT COLONY OPTIMIZATION (REVISI)")
    print("=" * 70)

    if RANDOM_SEED is not None:
        np.random.seed(RANDOM_SEED)

    demand_df = load_demand_data()

    print("Data demand berhasil dibaca.")
    print(demand_df[["Periode", "Kategori", "Demand", "Headway_Eksisting"]])
    print()

    results = run_ant_colony_optimization(demand_df)
    save_results(results)

    print("Hasil terbaik Ant Colony Optimization:")
    print("Bus aktif (x_t) :", results["active_buses"])
    print("Trip PP (q_t) :", results["trips"], " -> total:", sum(results["trips"]))
    print("Headway      :", results["headways"])
    print()

    for key, value in results["best_evaluation"].items():
        print(f"{key:20s}: {value:,.4f}")

    print(f"computation_time : {results['computation_time']:,.4f} seconds")
    print()
    print(f"Output disimpan pada: {OUTPUT_FILE}")
    print("=" * 70)

if __name__ == "__main__":
    main()

```

Lampiran 6 Program Python Performance Evaluation

```

import os
from typing import Dict

```

```

import numpy as np
import pandas as pd

# =====
# KONFIGURASI FILE
# =====

DATA_FOLDER = "data"

EXISTING_FILE = os.path.join(DATA_FOLDER, "objective_function_test.xlsx")
GA_FILE = os.path.join(DATA_FOLDER, "ga_result.xlsx")
ACO_FILE = os.path.join(DATA_FOLDER, "aco_result.xlsx")

OUTPUT_FILE = os.path.join(DATA_FOLDER, "performance_evaluation.xlsx")

DEFAULT_COST_PER_TRIP = 142_817.0

# =====
# FUNGSI PEMBACA HASIL
# =====

def read_summary(file_path: str, method_name: str) -> Dict[str, float]:

    if not os.path.exists(file_path):
        raise FileNotFoundError(
            f"File tidak ditemukan: {file_path}\n"
            f"Pastikan program untuk {method_name} sudah dijalankan.")

    df = pd.read_excel(file_path, sheet_name="Ringkasan_Evaluasi")

    if "Indikator" not in df.columns or "Nilai" not in df.columns:
        raise ValueError(
            f"Sheet Ringkasan_Evaluasi pada {file_path} harus memiliki kolom "
            "'Indikator' dan 'Nilai'.")

    result = dict(zip(df["Indikator"], df["Nilai"]))
    result["Metode"] = method_name

    return result

def read_detail(file_path: str, sheet_name: str = "Detail_Per_Periode") ->
pd.DataFrame:
    """
    Membaca detail per periode dari file hasil.
    """
    if not os.path.exists(file_path):
        raise FileNotFoundError(f"File tidak ditemukan: {file_path}")

    return pd.read_excel(file_path, sheet_name=sheet_name)

# =====
# PEMBENTUKAN TABEL PERBANDINGAN

```

```
# =====

def build_comparison_table(
    existing: Dict[str, float],
    ga: Dict[str, float],
    aco: Dict[str, float],) -> pd.DataFrame:

    indicators = [
        "objective_value",
        "operational_cost",
        "waiting_time",
        "empty_seat",
        "shortage",
        "load_factor",
        "fleet_utilization",
        "fitness",
        "total_demand",
        "total_capacity",
        "total_trips",]

    rows = []

    for indicator in indicators:
        rows.append({
            "Indikator": indicator,
            "Existing": existing.get(indicator, np.nan),
            "GA": ga.get(indicator, np.nan),
            "ACO": aco.get(indicator, np.nan),})

    comparison_df = pd.DataFrame(rows)

    return comparison_df

def build_operational_summary(
    existing: Dict[str, float],
    ga: Dict[str, float],
    aco: Dict[str, float],) -> pd.DataFrame:

    existing_cost = float(existing.get("operational_cost", np.nan))
    ga_cost = float(ga.get("operational_cost", np.nan))
    aco_cost = float(aco.get("operational_cost", np.nan))

    total_demand = float(existing.get("total_demand", np.nan))

    def saving_percent(new_cost: float) -> float:
        if existing_cost == 0 or np.isnan(existing_cost):
            return np.nan
        return ((existing_cost - new_cost) / existing_cost) * 100

    def cost_per_passenger(cost: float) -> float:
        if total_demand == 0 or np.isnan(total_demand):
            return np.nan
```

```
return cost / total_demand
```

```
rows = [
    {
        "Aspek": "Efisiensi Biaya",
        "Indikator": "Biaya Operasional Harian",
        "Satuan": "Rp/hari",
        "Existing": existing_cost,
        "GA": ga_cost,
        "ACO": aco_cost,
        "Keterangan": "Semakin kecil semakin baik",},
    {
        "Aspek": "Efisiensi Biaya",
        "Indikator": "Penghematan Biaya",
        "Satuan": "%",
        "Existing": 0,
        "GA": saving_percent(ga_cost),
        "ACO": saving_percent(aco_cost),
        "Keterangan": "Dibandingkan kondisi eksisting",},
    {
        "Aspek": "Efisiensi Biaya",
        "Indikator": "Biaya per Penumpang",
        "Satuan": "Rp/penumpang",
        "Existing": cost_per_passenger(existing_cost),
        "GA": cost_per_passenger(ga_cost),
        "ACO": cost_per_passenger(aco_cost),
        "Keterangan": "Semakin kecil semakin baik",},
    {
        "Aspek": "Operasional Armada",
        "Indikator": "Total Trip PP Harian",
        "Satuan": "trip",
        "Existing": existing.get("total_trips", np.nan),
        "GA": ga.get("total_trips", np.nan),
        "ACO": aco.get("total_trips", np.nan),
        "Keterangan": "Batas maksimum 15 bus x 4 trip = 60",},
    {
        "Aspek": "Kualitas Layanan",
        "Indikator": "Waktu Tunggu Total",
        "Satuan": "menit",
        "Existing": existing.get("waiting_time", np.nan),
        "GA": ga.get("waiting_time", np.nan),
        "ACO": aco.get("waiting_time", np.nan),
        "Keterangan": "Semakin kecil semakin baik",},
    {
        "Aspek": "Kualitas Layanan",
        "Indikator": "Load Factor",
        "Satuan": "%",
        "Existing": existing.get("load_factor", np.nan) * 100,
        "GA": ga.get("load_factor", np.nan) * 100,
        "ACO": aco.get("load_factor", np.nan) * 100,
        "Keterangan": "Mengukur keterisian kendaraan",},
    {
```

```

        "Aspek": "Efisiensi Armada",
        "Indikator": "Empty Seat",
        "Satuan": "kursi",
        "Existing": existing.get("empty_seat", np.nan),
        "GA": ga.get("empty_seat", np.nan),
        "ACO": aco.get("empty_seat", np.nan),
        "Keterangan": "Semakin kecil semakin baik",},
    {
        "Aspek": "Efisiensi Armada",
        "Indikator": "Utilisasi Armada",
        "Satuan": "%",
        "Existing": existing.get("fleet_utilization", np.nan) * 100,
        "GA": ga.get("fleet_utilization", np.nan) * 100,
        "ACO": aco.get("fleet_utilization", np.nan) * 100,
        "Keterangan": "Proporsi penggunaan armada",},
    {
        "Aspek": "Kinerja Algoritma",
        "Indikator": "Nilai Fungsi Objektif",
        "Satuan": "-",
        "Existing": existing.get("objective_value", np.nan),
        "GA": ga.get("objective_value", np.nan),
        "ACO": aco.get("objective_value", np.nan),
        "Keterangan": "Semakin kecil semakin baik",},
    {
        "Aspek": "Kinerja Algoritma",
        "Indikator": "Fitness",
        "Satuan": "-",
        "Existing": existing.get("fitness", np.nan),
        "GA": ga.get("fitness", np.nan),
        "ACO": aco.get("fitness", np.nan),
        "Keterangan": "Semakin besar semakin baik",},
    {
        "Aspek": "Kinerja Algoritma",
        "Indikator": "Waktu Komputasi",
        "Satuan": "detik",
        "Existing": 0,
        "GA": ga.get("Computation Time (seconds)", ga.get("computation_time",
np.nan)),
        "ACO": aco.get("Computation Time (seconds)", aco.get("computation_time",
np.nan)),
        "Keterangan": "Semakin kecil semakin baik",},]

    return pd.DataFrame(rows)

def determine_best_method(operational_summary: pd.DataFrame) ->
pd.DataFrame:

    df = operational_summary.copy()

    best_methods = []

    for _, row in df.iterrows():

```

```

indicator = row["Indikator"]

ga_value = row["GA"]
aco_value = row["ACO"]

if pd.isna(ga_value) or pd.isna(aco_value):
    best_methods.append("-")
    continue

higher_is_better = [
    "Load Factor",
    "Utilisasi Armada",
    "Fitness",
    "Penghematan Biaya",]

if indicator in higher_is_better:
    if ga_value > aco_value:
        best_methods.append("GA")
    elif aco_value > ga_value:
        best_methods.append("ACO")
    else:
        best_methods.append("Sama")
else:
    if ga_value < aco_value:
        best_methods.append("GA")
    elif aco_value < ga_value:
        best_methods.append("ACO")
    else:
        best_methods.append("Sama")

df["Metode Terbaik"] = best_methods

return df

# =====
# PERBANDINGAN DETAIL PER PERIODE
# =====

def build_period_comparison(
    existing_detail: pd.DataFrame,
    ga_detail: pd.DataFrame,
    aco_detail: pd.DataFrame,) -> pd.DataFrame:

    columns_base = ["Periode", "Waktu", "Kategori", "Demand"]

    result = existing_detail[columns_base].copy()

    result["Bus_Existing"] = existing_detail["Bus_Aktif"]
    result["Bus_GA"] = ga_detail["Bus_Aktif"]
    result["Bus_ACO"] = aco_detail["Bus_Aktif"]

    result["Trip_Existing"] = (

```

```

    existing_detail["Trip_PP"] if "Trip_PP" in existing_detail.columns
    else existing_detail["Bus_Aktif"])
result["Trip_GA"] = (
    ga_detail["Trip_PP"] if "Trip_PP" in ga_detail.columns
    else ga_detail["Bus_Aktif"])
result["Trip_ACO"] = (
    aco_detail["Trip_PP"] if "Trip_PP" in aco_detail.columns
    else aco_detail["Bus_Aktif"])

result["Headway_Existing"] = existing_detail["Headway"]
result["Headway_GA"] = ga_detail["Headway"]
result["Headway_ACO"] = aco_detail["Headway"]

result["Load_Factor_Existing"] = existing_detail["Load_Factor"] * 100
result["Load_Factor_GA"] = ga_detail["Load_Factor"] * 100
result["Load_Factor_ACO"] = aco_detail["Load_Factor"] * 100

result["Empty_Seat_Existing"] = existing_detail["Empty_Seat"]
result["Empty_Seat_GA"] = ga_detail["Empty_Seat"]
result["Empty_Seat_ACO"] = aco_detail["Empty_Seat"]

# Biaya per periode (dihitung dari trip, bukan bus) -- fallback
# dihitung manual jika file lama belum punya kolom Biaya_Periode.
if "Biaya_per_Trip" in existing_detail.columns:
    cost_per_trip = float(existing_detail["Biaya_per_Trip"].iloc[0])
else:
    cost_per_trip = DEFAULT_COST_PER_TRIP

result["Biaya_Existing"] = (
    existing_detail["Biaya_Periode"] if "Biaya_Periode" in existing_detail.columns
    else result["Trip_Existing"] * cost_per_trip)
result["Biaya_GA"] = (
    ga_detail["Biaya_Periode"] if "Biaya_Periode" in ga_detail.columns
    else result["Trip_GA"] * cost_per_trip)
result["Biaya_ACO"] = (
    aco_detail["Biaya_Periode"] if "Biaya_Periode" in aco_detail.columns
    else result["Trip_ACO"] * cost_per_trip)

return result

# =====
# PROGRAM UTAMA
# =====

def main() -> None:
    print("=" * 70)
    print("06 PERFORMANCE EVALUATION (REVISI)")
    print("=" * 70)

    existing = read_summary(EXISTING_FILE, "Existing")
    ga = read_summary(GA_FILE, "GA")
    aco = read_summary(ACO_FILE, "ACO")

```

```

existing_detail = read_detail(EXISTING_FILE)
ga_detail = read_detail(GA_FILE)
aco_detail = read_detail(ACO_FILE)

comparison_df = build_comparison_table(existing, ga, aco)
operational_summary_df = build_operational_summary(existing, ga, aco)
operational_summary_df = determine_best_method(operational_summary_df)

period_comparison_df = build_period_comparison(
    existing_detail=existing_detail,
    ga_detail=ga_detail,
    aco_detail=aco_detail,)

os.makedirs(DATA_FOLDER, exist_ok=True)

with pd.ExcelWriter(OUTPUT_FILE, engine="openpyxl") as writer:
    comparison_df.to_excel(
        writer,
        sheet_name="Comparison_Raw",
        index=False,)

    operational_summary_df.to_excel(
        writer,
        sheet_name="Ringkasan_Perbandingan",
        index=False,)

    period_comparison_df.to_excel(
        writer,
        sheet_name="Perbandingan_Per_Periode",
        index=False,)

print("Evaluasi kinerja selesai.")
print()
print("Ringkasan perbandingan:")
print(operational_summary_df)
print()
print(f"Output disimpan pada: {OUTPUT_FILE}")
print("=" * 70)

if __name__ == "__main__":
    main()

```

Lampiran 7 Program Python Sensitivity Analysis

```

import os
import time
import importlib.util
from typing import Dict, List
import numpy as np
import pandas as pd

```



```

# =====
# KONFIGURASI FILE
# =====

DATA_FOLDER = "data"

DEMAND_FILE = os.path.join(DATA_FOLDER, "demand_generated.xlsx")
OUTPUT_FILE = os.path.join(DATA_FOLDER, "sensitivity_analysis.xlsx")

# =====
# KONFIGURASI SKENARIO
# =====

DEMAND_SCENARIOS = {
    "Demand_-20%": -0.20,
    "Demand_-10%": -0.10,
    "Demand_0%": 0.00,
    "Demand_+10%": 0.10,
    "Demand_+20%": 0.20,}

# =====
# KONFIGURASI RANDOM
# =====

RANDOM_SEED = 789

# =====
# FUNGSI IMPORT MODUL DARI FILE
# =====

def import_module_from_file(module_name: str, file_name: str):
    try:
        base_dir = os.path.dirname(os.path.abspath(__file__))
    except NameError:
        base_dir = os.getcwd()

    module_path = os.path.join(base_dir, file_name)

    if not os.path.exists(module_path):
        module_path = file_name

    if not os.path.exists(module_path):
        raise FileNotFoundError(f"File modul tidak ditemukan: {file_name}")

    spec = importlib.util.spec_from_file_location(module_name, module_path)
    module = importlib.util.module_from_spec(spec)
    spec.loader.exec_module(module)

    return module

ga_module = import_module_from_file(

```

```

module_name="ga_module",
file_name="04_genetic_algorithm.py",)

aco_module = import_module_from_file(
    module_name="aco_module",
    file_name="05_ant_colony_optimization.py",)

# =====
# FUNGSI PEMBACA DATA
# =====

def load_base_demand(file_path: str = DEMAND_FILE) -> pd.DataFrame:

    if not os.path.exists(file_path):
        raise FileNotFoundError(
            f"File demand tidak ditemukan: {file_path}\n"
            "Jalankan 02_demand_generator.py terlebih dahulu.")

    return pd.read_excel(file_path, sheet_name="Demand")

# =====
# FUNGSI PEMBENTUKAN SKENARIO DEMAND
# =====

def create_demand_scenario(
    base_demand_df: pd.DataFrame,
    scenario_name: str,
    change_rate: float,) -> pd.DataFrame:

    df = base_demand_df.copy()

    df["Demand_Base"] = df["Demand"]
    df["Scenario"] = scenario_name
    df["Demand_Change_Rate"] = change_rate
    df["Demand"] = np.round(df["Demand"] * (1 + change_rate)).astype(int)

    return df

# =====
# FUNGSI EKSTRAK RINGKASAN HASIL
# =====

def extract_summary(
    method: str,
    scenario_name: str,
    change_rate: float,
    results: Dict[str, object],) -> Dict[str, float]:

    evaluation = results["best_evaluation"]

    return {
        "Scenario": scenario_name,

```

```

    "Demand_Change_Rate": change_rate,
    "Method": method,
    "Objective_Value": evaluation["objective_value"],
    "Fitness": evaluation["fitness"],
    "Operational_Cost": evaluation["operational_cost"],
    "Waiting_Time": evaluation["waiting_time"],
    "Empty_Seat": evaluation["empty_seat"],
    "Shortage": evaluation["shortage"],
    "Load_Factor": evaluation["load_factor"],
    "Fleet_Utilization": evaluation["fleet_utilization"],
    "Penalty": evaluation["penalty"],
    "Total_Demand": evaluation["total_demand"],
    "Total_Capacity": evaluation["total_capacity"],
    "Computation_Time": results["computation_time"],}

def add_solution_columns(
    scenario_name: str,
    method: str,
    results: Dict[str, object],
    demand_df: pd.DataFrame,) -> pd.DataFrame:

    detail = results["detail_df"].copy()

    detail.insert(0, "Scenario", scenario_name)
    detail.insert(1, "Method", method)

    if "Demand_Base" in demand_df.columns:
        detail["Demand_Base"] = demand_df["Demand_Base"].values

    if "Demand_Change_Rate" in demand_df.columns:
        detail["Demand_Change_Rate"] = demand_df["Demand_Change_Rate"].values

    return detail

# =====
# FUNGSI ANALISIS SENSITIVITAS
# =====

def run_sensitivity_analysis(
    base_demand_df: pd.DataFrame,) -> Dict[str, pd.DataFrame]:

    summary_records = []
    detail_records = []
    convergence_records = []

    for scenario_name, change_rate in DEMAND_SCENARIOS.items():
        print("-" * 70)
        print(f"Menjalankan skenario: {scenario_name}")

        scenario_df = create_demand_scenario(
            base_demand_df=base_demand_df,

```

```

scenario_name=scenario_name,
change_rate=change_rate,)

# Agar hasil relatif replikatif untuk setiap skenario,
# seed dibedakan berdasarkan urutan skenario.
scenario_index = list(DEMAND_SCENARIOS.keys()).index(scenario_name)

np.random.seed(RANDOM_SEED + scenario_index)
ga_results = ga_module.run_genetic_algorithm(scenario_df)

np.random.seed(RANDOM_SEED + 100 + scenario_index)
aco_results = aco_module.run_ant_colony_optimization(scenario_df)

summary_records.append(
    extract_summary(
        method="GA",
        scenario_name=scenario_name,
        change_rate=change_rate,
        results=ga_results,))

summary_records.append(
    extract_summary(
        method="ACO",
        scenario_name=scenario_name,
        change_rate=change_rate,
        results=aco_results,))

ga_detail = add_solution_columns(
    scenario_name=scenario_name,
    method="GA",
    results=ga_results,
    demand_df=scenario_df,)

aco_detail = add_solution_columns(
    scenario_name=scenario_name,
    method="ACO",
    results=aco_results,
    demand_df=scenario_df,)

detail_records.append(ga_detail)
detail_records.append(aco_detail)

ga_history = ga_results["history_df"].copy()
ga_history.insert(0, "Scenario", scenario_name)
ga_history.insert(1, "Method", "GA")

aco_history = aco_results["history_df"].copy()
aco_history.insert(0, "Scenario", scenario_name)
aco_history.insert(1, "Method", "ACO")

convergence_records.append(ga_history)
convergence_records.append(aco_history)

```

```

print(f"GA Objective : {ga_results['best_evaluation']['objective_value']:.6f}")
print(f"ACO Objective : {aco_results['best_evaluation']['objective_value']:.6f}")

summary_df = pd.DataFrame(summary_records)
detail_df = pd.concat(detail_records, ignore_index=True)
convergence_df = pd.concat(convergence_records, ignore_index=True)

return {
    "summary_df": summary_df,
    "detail_df": detail_df,
    "convergence_df": convergence_df,}

# =====
# TABEL PIVOT UNTUK BAB IV
# =====

def build_pivot_tables(summary_df: pd.DataFrame) -> Dict[str, pd.DataFrame]:

    pivot_tables = {}

    indicators = [
        "Objective_Value",
        "Operational_Cost",
        "Waiting_Time",
        "Empty_Seat",
        "Load_Factor",
        "Fleet_Utilization",
        "Computation_Time",]

    for indicator in indicators:
        pivot_tables[indicator] = summary_df.pivot_table(
            index="Scenario",
            columns="Method",
            values=indicator,
            aggfunc="mean",).reset_index()

    return pivot_tables

# =====
# PROGRAM UTAMA
# =====

def main() -> None:
    print("=" * 70)
    print("07 SENSITIVITY ANALYSIS")
    print("=" * 70)

    base_demand_df = load_base_demand()

    print("Data demand dasar berhasil dibaca.")
    print(base_demand_df[["Periode", "Kategori", "Demand"]])

```

```

print()

results = run_sensitivity_analysis(base_demand_df)

summary_df = results["summary_df"]
detail_df = results["detail_df"]
convergence_df = results["convergence_df"]
pivot_tables = build_pivot_tables(summary_df)

os.makedirs(DATA_FOLDER, exist_ok=True)

with pd.ExcelWriter(OUTPUT_FILE, engine="openpyxl") as writer:
    summary_df.to_excel(
        writer,
        sheet_name="Ringkasan_Sensitivitas",
        index=False,)

    detail_df.to_excel(
        writer,
        sheet_name="Detail_Per_Periode",
        index=False,)

    convergence_df.to_excel(
        writer,
        sheet_name="Konvergensi",
        index=False,)

    for indicator, pivot_df in pivot_tables.items():
        sheet_name = indicator[:31]
        pivot_df.to_excel(
            writer,
            sheet_name=sheet_name,
            index=False,)

print()
print("Analisis sensitivitas selesai.")
print(summary_df)
print()
print(f"Output disimpan pada: {OUTPUT_FILE}")
print("=" * 70)

if __name__ == "__main__":
    main()

```

Lampiran 8 Program Python Visualization

```

import os
from typing import Optional
import pandas as pd
import matplotlib
matplotlib.use("Agg")

```

```

import matplotlib.pyplot as plt

# =====
# KONFIGURASI FILE DAN FOLDER
# =====

DATA_FOLDER = "data"
OUTPUT_FOLDER = "output"
FIGURE_FOLDER = os.path.join(OUTPUT_FOLDER, "figures")

DEMAND_FILE = os.path.join(DATA_FOLDER, "demand_generated.xlsx")
GA_FILE = os.path.join(DATA_FOLDER, "ga_result.xlsx")
ACO_FILE = os.path.join(DATA_FOLDER, "aco_result.xlsx")
PERFORMANCE_FILE = os.path.join(DATA_FOLDER,
    "performance_evaluation.xlsx")
SENSITIVITY_FILE = os.path.join(DATA_FOLDER, "sensitivity_analysis.xlsx")
EVALUASI_PULANG_FILE = os.path.join(DATA_FOLDER, "evaluasi_pulang.xlsx")

# =====
# KONFIGURASI UMUM GRAFIK
# =====

DPI = 300
FIGSIZE = (9, 5)

# =====
# FUNGSI UMUM
# =====

def ensure_output_folder() -> None:
    os.makedirs(FIGURE_FOLDER, exist_ok=True)

def save_figure(file_name: str) -> None:
    ensure_output_folder()
    path = os.path.join(FIGURE_FOLDER, file_name)
    plt.tight_layout()
    plt.savefig(path, dpi=DPI, bbox_inches="tight")
    plt.close()
    print(f"Grafik disimpan: {path}")

def check_file_exists(file_path: str) -> bool:
    if not os.path.exists(file_path):
        print(f"File tidak ditemukan, dilewati: {file_path}")
        return False
    return True

# =====
# 1. GRAFIK DEMAND PER PERIODE
# =====

def plot_demand_per_period() -> None:
    if not check_file_exists(DEMAND_FILE):

```

```

    return

    df = pd.read_excel(DEMAND_FILE, sheet_name="Demand")

    plt.figure(figsize=FIGSIZE)
    plt.bar(df["Periode"], df["Demand"])
    plt.xlabel("Periode Operasional")
    plt.ylabel("Demand Penumpang")
    plt.title("Demand Penumpang per Periode Operasional")

    for i, value in enumerate(df["Demand"]):
        plt.text(i, value, str(int(value)), ha="center", va="bottom", fontsize=9)

    save_figure("01_demand_per_period.png")

# =====
# 2. GRAFIK LOAD FACTOR BANGKITAN
# =====

def plot_generated_load_factor() -> None:
    if not check_file_exists(DEMAND_FILE):
        return

    df = pd.read_excel(DEMAND_FILE, sheet_name="Demand")

    plt.figure(figsize=FIGSIZE)
    plt.bar(df["Periode"], df["LF_Acak"] * 100)
    plt.xlabel("Periode Operasional")
    plt.ylabel("Load Factor (%)")
    plt.title("Load Factor Bangkitan per Periode")

    for i, value in enumerate(df["LF_Acak"] * 100):
        plt.text(i, value, f"{value:.1f}%", ha="center", va="bottom", fontsize=9)

    save_figure("02_generated_load_factor.png")

# =====
# 3. KONVERGENSI GENETIC ALGORITHM
# =====

def plot_ga_convergence() -> None:
    if not check_file_exists(GA_FILE):
        return

    df = pd.read_excel(GA_FILE, sheet_name="Konvergensi_GA")

    plt.figure(figsize=FIGSIZE)
    plt.plot(df["generation"], df["best_objective_value"], marker="o", markersize=2)
    plt.xlabel("Generasi")
    plt.ylabel("Nilai Fungsi Objektif Terbaik")
    plt.title("Konvergensi Genetic Algorithm")

```



```

save_figure("03_ga_convergence.png")

# =====
# 4. KONVERGENSI ANT COLONY OPTIMIZATION
# =====

def plot_aco_convergence() -> None:
    if not check_file_exists(ACO_FILE):
        return

    df = pd.read_excel(ACO_FILE, sheet_name="Konvergensi_ACO")

    plt.figure(figsize=FIGSIZE)
    plt.plot(df["iteration"], df["best_objective_value"], marker="o", markersize=2)
    plt.xlabel("Iterasi")
    plt.ylabel("Nilai Fungsi Objektif Terbaik")
    plt.title("Konvergensi Ant Colony Optimization")

    save_figure("04_aco_convergence.png")

# =====
# 5. PERBANDINGAN BIAYA OPERASIONAL
# =====

def plot_operational_cost_comparison() -> None:
    if not check_file_exists(PERFORMANCE_FILE):
        return

    df = pd.read_excel(PERFORMANCE_FILE,
sheet_name="Ringkasan_Perbandingan")

    row = df.loc[df["Indikator"] == "Biaya Operasional Harian"]

    if row.empty:
        print("Indikator Biaya Operasional Harian tidak ditemukan.")
        return

    values = [
        float(row["Existing"].iloc[0]),
        float(row["GA"].iloc[0]),
        float(row["ACO"].iloc[0]),]

    labels = ["Existing", "GA", "ACO"]

    plt.figure(figsize=FIGSIZE)
    plt.bar(labels, values)
    plt.xlabel("Metode")
    plt.ylabel("Biaya Operasional (Rp/hari)")
    plt.title("Perbandingan Biaya Operasional Harian")

    for i, value in enumerate(values):
        plt.text(i, value, f"Rp{value:,.0f}", ha="center", va="bottom", fontsize=9)

```

```

save_figure("05_operational_cost_comparison.png")

# =====
# 6. PERBANDINGAN LOAD FACTOR
# =====

def plot_load_factor_comparison() -> None:
    if not check_file_exists(PERFORMANCE_FILE):
        return

    df = pd.read_excel(PERFORMANCE_FILE,
sheet_name="Ringkasan_Perbandingan")

    row = df.loc[df["Indikator"] == "Load Factor"]

    if row.empty:
        print("Indikator Load Factor tidak ditemukan.")
        return

    values = [
        float(row["Existing"].iloc[0]),
        float(row["GA"].iloc[0]),
        float(row["ACO"].iloc[0]),]

    labels = ["Existing", "GA", "ACO"]

    plt.figure(figsize=FIGSIZE)
    plt.bar(labels, values)
    plt.xlabel("Metode")
    plt.ylabel("Load Factor (%)")
    plt.title("Perbandingan Load Factor")

    for i, value in enumerate(values):
        plt.text(i, value, f"{value:.1f}%", ha="center", va="bottom", fontsize=9)

    save_figure("06_load_factor_comparison.png")

# =====
# 7. PERBANDINGAN BUS AKTIF PER PERIODE
# =====

def plot_active_bus_per_period() -> None:
    if not check_file_exists(PERFORMANCE_FILE):
        return

    df = pd.read_excel(PERFORMANCE_FILE,
sheet_name="Perbandingan_Per_Periode")

    x = range(len(df))
    width = 0.25

```

```

plt.figure(figsize=FIGSIZE)
plt.bar([i - width for i in x], df["Bus_Existing"], width=width, label="Existing")
plt.bar(x, df["Bus_GA"], width=width, label="GA")
plt.bar([i + width for i in x], df["Bus_ACO"], width=width, label="ACO")

plt.xlabel("Periode Operasional")
plt.ylabel("Jumlah Bus Aktif")
plt.title("Perbandingan Jumlah Bus Aktif per Periode")
plt.xticks(list(x), df["Periode"])
plt.legend()

save_figure("07_active_bus_per_period.png")

# =====
# 8. PERBANDINGAN HEADWAY PER PERIODE
# =====

def plot_headway_per_period() -> None:
    if not check_file_exists(PERFORMANCE_FILE):
        return

    df = pd.read_excel(PERFORMANCE_FILE,
sheet_name="Perbandingan_Per_Periode")

    plt.figure(figsize=FIGSIZE)
    plt.plot(df["Periode"], df["Headway_Existing"], marker="o", label="Existing")
    plt.plot(df["Periode"], df["Headway_GA"], marker="o", label="GA")
    plt.plot(df["Periode"], df["Headway_ACO"], marker="o", label="ACO")

    plt.xlabel("Periode Operasional")
    plt.ylabel("Headway (menit)")
    plt.title("Perbandingan Headway per Periode")
    plt.legend()

    save_figure("08_headway_per_period.png")

# =====
# 9. PERBANDINGAN EMPTY SEAT PER PERIODE
# =====

def plot_empty_seat_per_period() -> None:
    if not check_file_exists(PERFORMANCE_FILE):
        return

    df = pd.read_excel(PERFORMANCE_FILE,
sheet_name="Perbandingan_Per_Periode")

    x = range(len(df))
    width = 0.25

    plt.figure(figsize=FIGSIZE)

```

```

plt.bar([i - width for i in x], df["Empty_Seat_Existing"], width=width,
label="Existing")
plt.bar(x, df["Empty_Seat_GA"], width=width, label="GA")
plt.bar([i + width for i in x], df["Empty_Seat_ACO"], width=width, label="ACO")

plt.xlabel("Periode Operasional")
plt.ylabel("Empty Seat (kursi)")
plt.title("Perbandingan Empty Seat per Periode")
plt.xticks(list(x), df["Periode"])
plt.legend()

save_figure("09_empty_seat_per_period.png")

# =====
# 10. SENSITIVITAS DEMAND
# =====

def plot_sensitivity_objective() -> None:
    if not check_file_exists(SENSITIVITY_FILE):
        return

    df = pd.read_excel(SENSITIVITY_FILE, sheet_name="Ringkasan_Sensitivitas")

    pivot = df.pivot_table(
        index="Demand_Change_Rate",
        columns="Method",
        values="Objective_Value",
        aggfunc="mean",).reset_index()

    pivot["Demand_Change_Rate"] = pivot["Demand_Change_Rate"] * 100

    plt.figure(figsize=FIGSIZE)

    if "GA" in pivot.columns:
        plt.plot(pivot["Demand_Change_Rate"], pivot["GA"], marker="o", label="GA")

    if "ACO" in pivot.columns:
        plt.plot(pivot["Demand_Change_Rate"], pivot["ACO"], marker="o", label="ACO")

    plt.xlabel("Perubahan Demand (%)")
    plt.ylabel("Nilai Fungsi Objektif")
    plt.title("Sensitivitas Nilai Fungsi Objektif terhadap Perubahan Demand")
    plt.legend()

    save_figure("10_sensitivity_objective.png")

# =====
# 11. SENSITIVITAS DEMAND
# =====

def plot_sensitivity_operational_cost() -> None:
    if not check_file_exists(SENSITIVITY_FILE):

```

```

return

df = pd.read_excel(SENSITIVITY_FILE, sheet_name="Ringkasan_Sensitivitas")

pivot = df.pivot_table(
    index="Demand_Change_Rate",
    columns="Method",
    values="Operational_Cost",
    aggfunc="mean",).reset_index()

pivot["Demand_Change_Rate"] = pivot["Demand_Change_Rate"] * 100

plt.figure(figsize=FIGSIZE)

if "GA" in pivot.columns:
    plt.plot(pivot["Demand_Change_Rate"], pivot["GA"], marker="o", label="GA")

if "ACO" in pivot.columns:
    plt.plot(pivot["Demand_Change_Rate"], pivot["ACO"], marker="o", label="ACO")

plt.xlabel("Perubahan Demand (%)")
plt.ylabel("Biaya Operasional (Rp)")
plt.title("Sensitivitas Biaya Operasional terhadap Perubahan Demand")
plt.legend()

save_figure("11_sensitivity_operational_cost.png")

# =====
# 12. PERBANDINGAN WAKTU TUNGGU ARAH PULANG
# =====

def plot_pulang_waktu_tunggu() -> None:

    if not check_file_exists(EVALUASI_PULANG_FILE):
        return

    df = pd.read_excel(EVALUASI_PULANG_FILE,
        sheet_name="Waktu_Tunggu_Pulang")

    labels = df["Skenario"].tolist()
    values = df["Rata-rata Waktu Tunggu (menit)"].tolist()

    plt.figure(figsize=FIGSIZE)
    plt.bar(labels, values, color=["#7f8c8d", "#2E8B99", "#F4A100"])
    plt.ylabel("Waktu Tunggu (menit)")
    plt.title("Perbandingan Waktu Tunggu Arah Pulang")

    for i, value in enumerate(values):
        plt.text(i, value, f"{value:.2f} menit", ha="center", va="bottom", fontsize=9)

    save_figure("12_pulang_waktu_tunggu.png")

```

```

# =====
# 13. PERBANDINGAN BIAYA OPERASIONAL KONTRIBUSI ARAH PULANG
# =====

def plot_pulang_biaya() -> None:

    if not check_file_exists(EVALUASI_PULANG_FILE):
        return

    df = pd.read_excel(EVALUASI_PULANG_FILE,
sheet_name="Biaya_Operasional_Pulang")

    kolom_pulang = [c for c in df.columns if c.startswith("Kontribusi Arah Pulang")][0]

    labels = df["Skenario"].tolist()
    values = df[kolom_pulang].tolist()

    plt.figure(figsize=FIGSIZE)
    plt.bar(labels, values, color=["#7f8c8d", "#2E8B99", "#F4A100"])
    plt.ylabel("Biaya Operasional Arah Pulang (Rp/hari)")
    plt.title("Kontribusi Biaya Operasional Arah Pulang")

    for i, value in enumerate(values):
        plt.text(i, value, f"Rp{value:,.0f}", ha="center", va="bottom", fontsize=9)

    save_figure("13_pulang_biaya_operasional.png")

# =====
# PROGRAM UTAMA
# =====

def main() -> None:
    print("=" * 70)
    print("08 VISUALIZATION")
    print("=" * 70)

    ensure_output_folder()

    plot_demand_per_period()
    plot_generated_load_factor()
    plot_ga_convergence()
    plot_aco_convergence()
    plot_operational_cost_comparison()
    plot_load_factor_comparison()
    plot_active_bus_per_period()
    plot_headway_per_period()
    plot_empty_seat_per_period()
    plot_sensitivity_objective()
    plot_sensitivity_operational_cost()
    plot_pulang_waktu_tunggu()
    plot_pulang_biaya()

```

```

print()
print("Visualisasi selesai.")
print(f"Semua grafik disimpan pada folder: {FIGURE_FOLDER}")
print("=" * 70)

if __name__ == "__main__":
    main()

```

Lampiran 9 Program Python Analisis Rute Pulang

```

import os
import pandas as pd
import numpy as np

# =====
# KONFIGURASI FILE
# =====

DATA_FOLDER = "data"

HALTE_PULANG_FILE = os.path.join(DATA_FOLDER, "halte_pulang.xlsx")
JADWAL_PULANG_FILE = os.path.join(DATA_FOLDER, "jadwal_pulang.xlsx")
OUTPUT_FILE = os.path.join(DATA_FOLDER, "analisis_pulang.xlsx")

# Nama kolom pada file input.
KOLOM_NAMA_HALTE = "NAMA HALTE"
KOLOM_JARAK_METER = "Jarak_dari_Awal(km)" # isinya meter, lihat catatan di atas

KOLOM_BERANGKAT = "Berangkat"
KOLOM_TIBA = "Tiba"
KOLOM_HEADWAY = "Headway"
KOLOM_TRIP = "Trip"

# =====
# MEMBACA & MEMBERSIHKAN DATA
# =====

def load_halte_pulang(file_path: str = HALTE_PULANG_FILE) -> pd.DataFrame:
    if not os.path.exists(file_path):
        raise FileNotFoundError(f"File tidak ditemukan: {file_path}")

    df = pd.read_excel(file_path)
    df = df.dropna(how="all").reset_index(drop=True)

    if KOLOM_NAMA_HALTE not in df.columns or KOLOM_JARAK_METER not in df.columns:
        raise ValueError(
            f"File {file_path} harus memiliki kolom "
            f"'{KOLOM_NAMA_HALTE}' dan '{KOLOM_JARAK_METER}'. "
            f"Kolom yang ditemukan: {list(df.columns)}")

```

```

df["Jarak_km"] = df[KOLOM_JARAK_METER] / 1000.0
df = df.sort_values(by="Jarak_km").reset_index(drop=True)
return df

def load_jadwal_pulang(file_path: str = JADWAL_PULANG_FILE) -> pd.DataFrame:
    if not os.path.exists(file_path):
        raise FileNotFoundError(f"File tidak ditemukan: {file_path}")

    df = pd.read_excel(file_path)
    df = df.dropna(how="all").reset_index(drop=True)

    kolom_wajib = [KOLOM_BERANGKAT, KOLOM_TIBA, KOLOM_HEADWAY]
    hilang = [k for k in kolom_wajib if k not in df.columns]
    if hilang:
        raise ValueError(
            f"File {file_path} tidak memiliki kolom: {hilang}. "
            f"Kolom yang ditemukan: {list(df.columns)}")

    df[KOLOM_BERANGKAT] =
pd.to_datetime(df[KOLOM_BERANGKAT].astype(str), format="%H:%M:%S")
    df[KOLOM_TIBA] = pd.to_datetime(df[KOLOM_TIBA].astype(str),
format="%H:%M:%S")

    return df

# =====
# PERHITUNGAN PARAMETER
# =====

def hitung_jarak_pulang_aktual(halte_df: pd.DataFrame) -> float:
    """Jarak pulang aktual = jarak halte terjauh dari titik awal (km)."""
    return float(halte_df["Jarak_km"].max())

def hitung_waktu_tempuh(jadwal_df: pd.DataFrame) -> pd.DataFrame:
    """Waktu tempuh (menit) tiap trip = Tiba - Berangkat."""
    df = jadwal_df.copy()
    delta = df[KOLOM_TIBA] - df[KOLOM_BERANGKAT]
    df["Waktu_Tempuh_Menit"] = delta.dt.total_seconds() / 60
    return df

def validasi_headway(jadwal_df: pd.DataFrame) -> pd.DataFrame:

    df = jadwal_df.copy()
    headway_hitung = [0.0]
    for i in range(1, len(df)):
        selisih = (df[KOLOM_BERANGKAT].iloc[i] - df[KOLOM_BERANGKAT].iloc[i -
1]).total_seconds() / 60
        headway_hitung.append(selisih)
    df["Headway_Hitung_Ulang"] = headway_hitung
    df["Selisih_vs_Sumber"] = df["Headway_Hitung_Ulang"] - df[KOLOM_HEADWAY]

```



```
return df
```

```
# =====
# PROGRAM UTAMA
# =====
```

```
def main() -> None:
```

```
    print("=" * 70)
    print("01b ANALISIS DATA ARAH PULANG")
    print("=" * 70)
```

```
    halte_df = load_halte_pulang()
    jadwal_df = load_jadwal_pulang()
```

```
    jarak_pulang_aktual = hitung_jarak_pulang_aktual(halte_df)
```

```
    jadwal_df = hitung_waktu_tempuh(jadwal_df)
    jadwal_df = validasi_headway(jadwal_df)
```

```
    headway_valid = jadwal_df[KOLOM_HEADWAY].iloc[1:]
```

```
    rata_waktu_tempuh = float(jadwal_df["Waktu_Tempuh_Menit"].mean())
    rata_headway = float(headway_valid.mean())
    min_headway = float(headway_valid.min())
    max_headway = float(headway_valid.max())
    std_headway = float(headway_valid.std())
    selisih_maksimum = float(jadwal_df["Selisih_vs_Sumber"].abs().max())
```

```
    parameter_pulang = pd.DataFrame({
        "Parameter": [
            "Jumlah Halte Pulang",
            "Jarak Pulang Aktual (km)",
            "Jumlah Trip Tercatat",
            "Rata-rata Waktu Tempuh (menit)",
            "Rata-rata Headway Eksisting (menit)",
            "Headway Minimum (menit)",
            "Headway Maksimum (menit)",
            "Standar Deviasi Headway (menit)",
            "Selisih Maksimum Validasi Headway (menit)"
        ],
        "Nilai": [
            len(halte_df),
            round(jarak_pulang_aktual, 3),
            len(jadwal_df),
            round(rata_waktu_tempuh, 2),
            round(rata_headway, 2),
            round(min_headway, 2),
            round(max_headway, 2),
            round(std_headway, 2),
            round(selisih_maksimum, 4)
        ]
    })
```

```
    ringkasan_trip = None
    if KOLOM_TRIP in jadwal_df.columns:
```

```

ringkasan_trip = jadwal_df.groupby(KOLOM_TRIP).agg(
    Jumlah_Bus=("Bus", "count"),
    Rata_Waktu_Tempuh_Menit=("Waktu_Tempuh_Menit", "mean"),
    Rata_Headway_Menit=(KOLOM_HEADWAY, "mean"),
    Min_Headway_Menit=(KOLOM_HEADWAY, "min"),
    Max_Headway_Menit=(KOLOM_HEADWAY, "max"),).reset_index().round(2)

output_jadwal = jadwal_df.copy()
output_jadwal[KOLOM_BERANGKAT] =
output_jadwal[KOLOM_BERANGKAT].dt.strftime("%H:%M")
output_jadwal[KOLOM_TIBA] =
output_jadwal[KOLOM_TIBA].dt.strftime("%H:%M")

os.makedirs(DATA_FOLDER, exist_ok=True)

with pd.ExcelWriter(OUTPUT_FILE, engine="openpyxl") as writer:
    halte_df.to_excel(writer, sheet_name="Halte_Pulang", index=False)
    output_jadwal.to_excel(writer, sheet_name="Jadwal_Pulang", index=False)
    parameter_pulang.to_excel(writer, sheet_name="Parameter_Pulang",
index=False)
    if ringkasan_trip is not None:
        ringkasan_trip.to_excel(writer, sheet_name="Ringkasan_per_Trip",
index=False)

print()
print(parameter_pulang.to_string(index=False))
if ringkasan_trip is not None:
    print()
    print("Ringkasan per kelompok Trip:")
    print(ringkasan_trip.to_string(index=False))

print()
print("Catatan penting:")
print(f"- Jarak pulang aktual hasil perhitungan: {jarak_pulang_aktual:.3f} km")
print(" Bandingkan dengan JARAK_PULANG di 01_data_preparation.py (26.37
km).")
print(f"- Validasi headway: selisih maksimum antara kolom Headway sumber")
print(f" dan hasil hitung ulang = {selisih_maksimum:.4f} menit (0 berarti
konsisten).")
print(f"- Headway eksisting pulang sangat bervariasi (std = {std_headway:.2f}
menit).")
print(" konsisten dengan pengamatan bahwa jadwal pulang tidak beraturan.")
print()
print(f"Output disimpan pada: {OUTPUT_FILE}")
print("=" * 70)

if __name__ == "__main__":
    main()

```

Lampiran 10 Program Python Evaluasi Rute Pulang

```

import os
import pandas as pd

# =====
# KONFIGURASI FILE
# =====

DATA_FOLDER = "data"

ANALISIS_PULANG_FILE = os.path.join(DATA_FOLDER, "analisis_pulang.xlsx")
DATASET_OPERASIONAL_FILE = os.path.join(DATA_FOLDER,
"dataset_operasional.xlsx")
EXISTING_FILE = os.path.join(DATA_FOLDER, "objective_function_test.xlsx")
GA_FILE = os.path.join(DATA_FOLDER, "ga_result.xlsx")
ACO_FILE = os.path.join(DATA_FOLDER, "aco_result.xlsx")

OUTPUT_FILE = os.path.join(DATA_FOLDER, "evaluasi_pulang.xlsx")

# =====
# FUNGSI BANTUAN
# =====

def _cek_file(path: str, sumber: str) -> None:
    if not os.path.exists(path):
        raise FileNotFoundError(
            f"File tidak ditemukan: {path}\n"
            f"Jalankan {sumber} terlebih dahulu.")

def ambil_parameter(df: pd.DataFrame, nama_parameter: str,
kolom_nama="Parameter", kolom_nilai="Nilai") -> float:
    row = df.loc[df[kolom_nama] == nama_parameter]
    if row.empty:
        raise ValueError(f"Parameter '{nama_parameter}' tidak ditemukan.")
    return float(row[kolom_nilai].iloc[0])

def ambil_indikator(df: pd.DataFrame, nama_indikator: str) -> float:
    return ambil_parameter(df, nama_indikator, kolom_nama="Indikator")

def rata_rata_headway_solusi(best_solution_df: pd.DataFrame) -> float:

    headway_rows =
best_solution_df[best_solution_df["Komponen"].str.startswith("Headway_")]
    return float(headway_rows["Nilai"].mean())

# =====
# PROGRAM UTAMA
# =====

def main() -> None:

```

```

print("=" * 70)
print("01c EVALUASI ARAH PULANG (TANPA MENJALANKAN ULANG
GA/ACO)")
print("=" * 70)

for f, sumber in [
    (ANALISIS_PULANG_FILE, "01b_analisis_pulang.py"),
    (DATASET_OPERASIONAL_FILE, "01_data_preparation.py"),
    (EXISTING_FILE, "objective_function.py"),
    (GA_FILE, "04_genetic_algorithm.py"),
    (ACO_FILE, "05_ant_colony_optimization.py"),]:
    _cek_file(f, sumber)

# -----
# 1. Baca parameter pulang (hasil 01b)
# -----
param_pulang = pd.read_excel(ANALISIS_PULANG_FILE,
sheet_name="Parameter_Pulang")
jarak_pulang_aktual = ambil_parameter(param_pulang, "Jarak Pulang Aktual
(km)")
headway_pulang_lapangan = ambil_parameter(param_pulang, "Rata-rata
Headway Eksisting (menit)")
std_headway_lapangan = ambil_parameter(param_pulang, "Standar Deviasi
Headway (menit)")

# -----
# 2. Baca parameter dasar (hasil 01)
# -----
param_dasar = pd.read_excel(DATASET_OPERASIONAL_FILE,
sheet_name="Parameter")
jarak_pergi = ambil_parameter(param_dasar, "Jarak Pergi (km)")
jarak_pulang_asumsi = ambil_parameter(param_dasar, "Jarak Pulang (km)")
jarak_pp = ambil_parameter(param_dasar, "Jarak PP (km)")

proporsi_pulang = jarak_pulang_aktual / jarak_pp

# -----
# 3. Baca biaya operasional Existing / GA / ACO
# -----
ringkasan_existing = pd.read_excel(EXISTING_FILE,
sheet_name="Ringkasan_Evaluasi")
ringkasan_ga = pd.read_excel(GA_FILE, sheet_name="Ringkasan_Evaluasi")
ringkasan_aco = pd.read_excel(ACO_FILE, sheet_name="Ringkasan_Evaluasi")

biaya_existing = ambil_indikator(ringkasan_existing, "operational_cost")
biaya_ga = ambil_indikator(ringkasan_ga, "operational_cost")
biaya_aco = ambil_indikator(ringkasan_aco, "operational_cost")

biaya_pulang_existing = proporsi_pulang * biaya_existing
biaya_pulang_ga = proporsi_pulang * biaya_ga
biaya_pulang_aco = proporsi_pulang * biaya_aco

```

```

# -----
# 4. Baca headway optimal GA / ACO (rata-rata seluruh periode)
# -----
best_solution_ga = pd.read_excel(GA_FILE, sheet_name="Best_Solution")
best_solution_aco = pd.read_excel(ACO_FILE, sheet_name="Best_Solution")

headway_optimal_ga = rata_rata_headway_solusi(best_solution_ga)
headway_optimal_aco = rata_rata_headway_solusi(best_solution_aco)

waktu_tunggu_lapangan = headway_pulang_lapangan / 2
waktu_tunggu_ga = headway_optimal_ga / 2
waktu_tunggu_aco = headway_optimal_aco / 2

penurunan_waktu_tunggu_ga = (waktu_tunggu_lapangan - waktu_tunggu_ga) /
waktu_tunggu_lapangan * 100
penurunan_waktu_tunggu_aco = (waktu_tunggu_lapangan - waktu_tunggu_aco) /
waktu_tunggu_lapangan * 100

# -----
# 5. Susun tabel ringkasan
# -----
tabel_biaya = pd.DataFrame({
    "Skenario": ["Eksisting", "GA", "ACO"],
    "Biaya Operasional Total (Rp/hari)": [biaya_existing, biaya_ga, biaya_aco],
    f"Kontribusi Arah Pulang (~{proporsi_pulang*100:.2f}% jarak) (Rp/hari)": [
        biaya_pulang_existing, biaya_pulang_ga, biaya_pulang_aco],})

tabel_waktu_tunggu = pd.DataFrame({
    "Skenario": ["Eksisting (data lapangan)", "Jika Ikut Pola GA", "Jika Ikut Pola
ACO"],
    "Rata-rata Headway (menit)": [headway_pulang_lapangan,
headway_optimal_ga, headway_optimal_aco],
    "Rata-rata Waktu Tunggu (menit)": [waktu_tunggu_lapangan, waktu_tunggu_ga,
waktu_tunggu_aco],
    "Penurunan Waktu Tunggu vs Eksisting (%)": [0.0,
penurunan_waktu_tunggu_ga, penurunan_waktu_tunggu_aco],})

catatan = pd.DataFrame({
    "Keterangan": [
        "Jarak pulang aktual (km)",
        "Jarak PP / pergi-pulang (km)",
        "Proporsi jarak pulang terhadap PP (%)",
        "Standar deviasi headway lapangan (menit)",
        "Metode perhitungan biaya pulang",
        "Metode perhitungan waktu tunggu",],
    "Nilai / Penjelasan": [
        round(jarak_pulang_aktual, 3),
        round(jarak_pp, 2),
        round(proporsi_pulang * 100, 2),
        round(std_headway_lapangan, 2),
        "Biaya total (Existing/GA/ACO) x proporsi jarak pulang terhadap PP "
        "(karena biaya trip dihitung dari jarak PP gabungan, bukan per arah)",

```

```

    "Headway rata-rata / 2 (asumsi kedatangan penumpang acak/random
arrival)"),,})

os.makedirs(DATA_FOLDER, exist_ok=True)

with pd.ExcelWriter(OUTPUT_FILE, engine="openpyxl") as writer:
    tabel_biaya.to_excel(writer, sheet_name="Biaya_Operasional_Pulang",
index=False)
    tabel_waktu_tunggu.to_excel(writer, sheet_name="Waktu_Tunggu_Pulang",
index=False)
    catatan.to_excel(writer, sheet_name="Catatan_Metodologi", index=False)

print()
print("Tabel Biaya Operasional (kontribusi arah pulang):")
print(tabel_biaya.to_string(index=False))
print()
print("Tabel Waktu Tunggu Arah Pulang:")
print(tabel_waktu_tunggu.to_string(index=False))
print()
print(f"Output disimpan pada: {OUTPUT_FILE}")
print("=" * 70)

if __name__ == "__main__":
    main()

```